Securing Your Python Supply Chain

From Dependencies to Deployment

Nine things I discovered in my own project

Bernát Gábor · PyBeach · October 24, 2026

bernat.tech · github.com/gaborbernat



Slides

Bloomberg
©2026 Bloomberg Finance L.P.
All rights reserved.

[00:00–00:21]

You add a security scanner to your release pipeline. Which credentials can that scanner read? In March, a compromised security tool helped turn widely used Python packages into credential stealers. I discovered nine problems in my own project. This talk is those nine, and what each one would have cost.

Bernát Gábor



Source: Bernát Gábor. License: CC BY 4.0.

PyPA / tox-dev maintainer

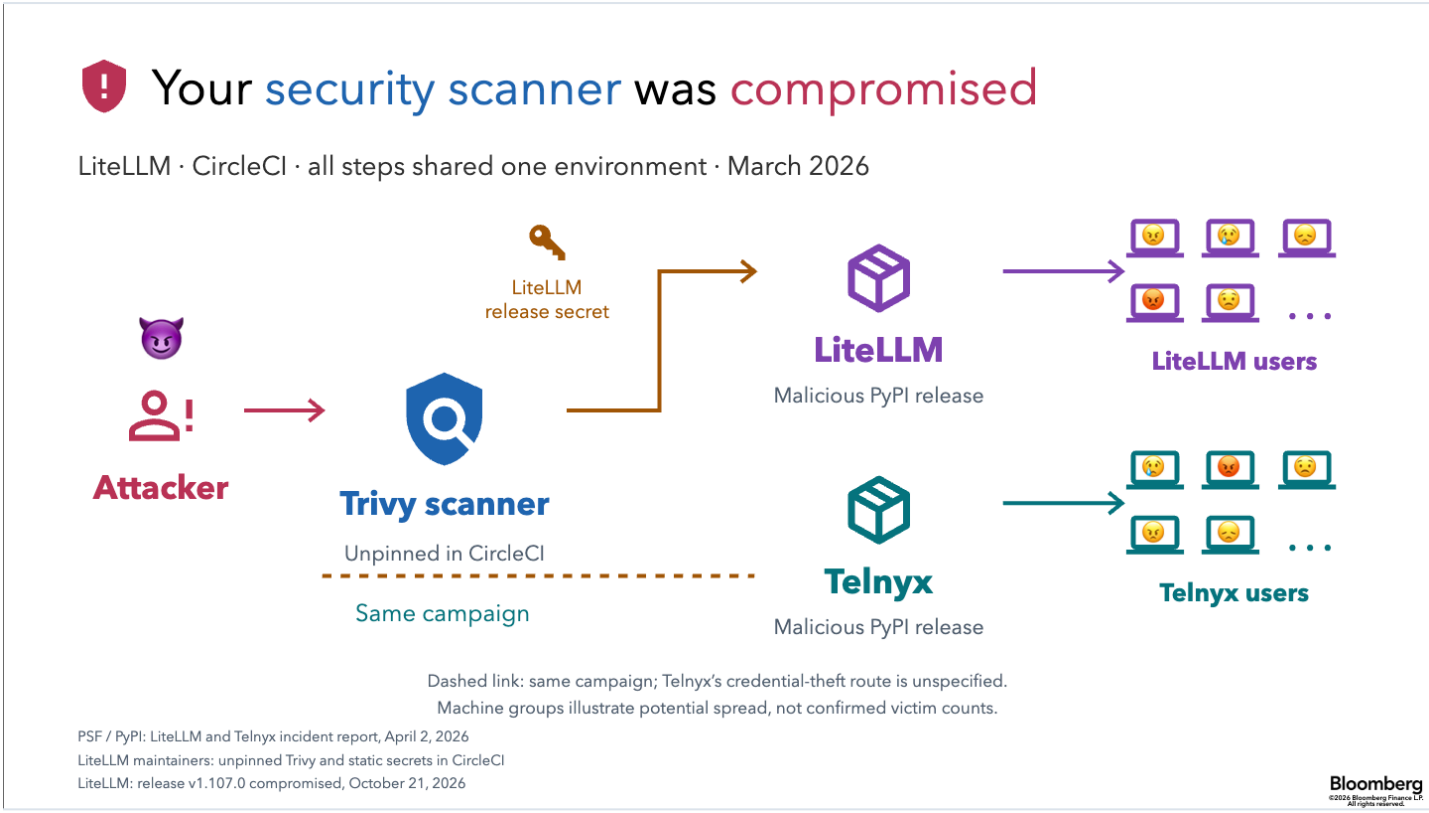
virtualenv • **tox** • **pipx** • **platformdirs** • **filelock**

Bloomberg, Developer Experience

Bloomberg
©2024 Bloomberg Finance L.P.
All rights reserved.

[00:21–00:39]

I'm Bernát. I work on developer experience at Bloomberg and maintain virtualenv, tox, pipx, and a few other packaging tools. virtualenv creates Python environments. It gets about a hundred million downloads a month, which is why the nine things matter to more people than me.



[00:39–01:27]

Hands up if your CI runs a security scanner. Keep them up if you know which secrets that job can read.

[Watch the hands drop. Give it three seconds.]

If your hand went down, this talk is for you.

[Click: Follow the credential to LiteLLM.]

LiteLLM ran on CircleCI; all steps shared one environment. Its maintainers reported an unpinned Trivy scanner that could read static release secrets, including the PyPI credential. Their account links that access to the malicious releases. The scanner was there to make the release safer. It had the keys to the release.

[Click: Follow the releases to users.]

Telnyx was hit in the same campaign. Both packages carried credential-stealing code to everyone who installed them. And before anyone relaxes: GitHub Actions steps in one job share a runner too. This is not a CircleCI story.

And on Wednesday, attackers compromised LiteLLM again, through a different door.



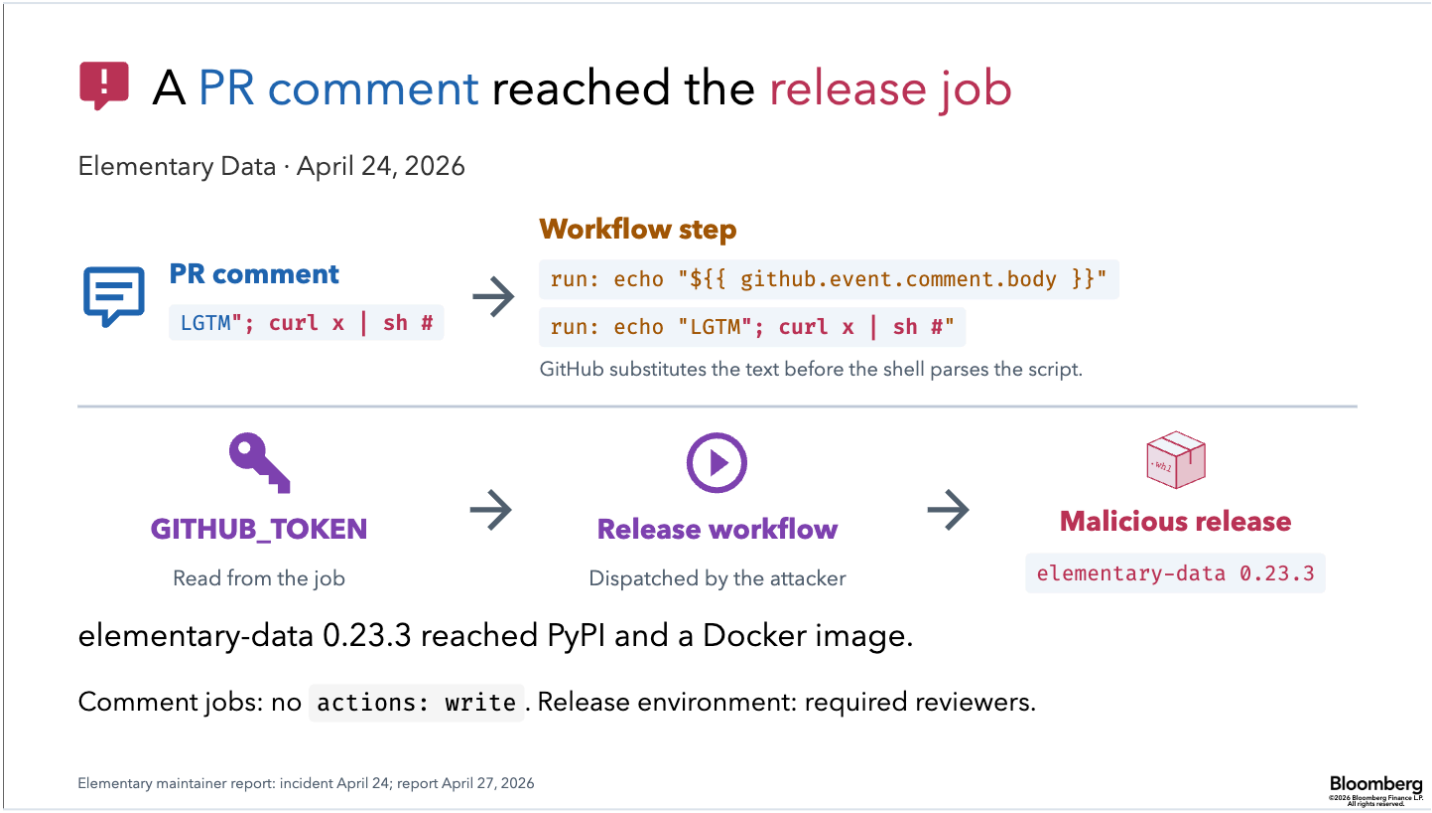
[01:27–02:03]

LiteLLM normally saw roughly 1,700 downloads a minute. In March, the affected versions received over 119,000 downloads before quarantine, two hours and thirty-two minutes later. That's downloads, not compromised machines, but every one of them was a chance. Downloads kept coming the whole time people were responding.

You might be thinking, 'I'd never make that mistake.' Then I checked mine.

[Click: Reveal the changelog's release authority. Pause for recognition.]

There it was, in changelog generation. The tools writing our release notes ran in a job with access to the release credential. Not a scanner, a Markdown generator, with the same reach. That's the first one; keep count.



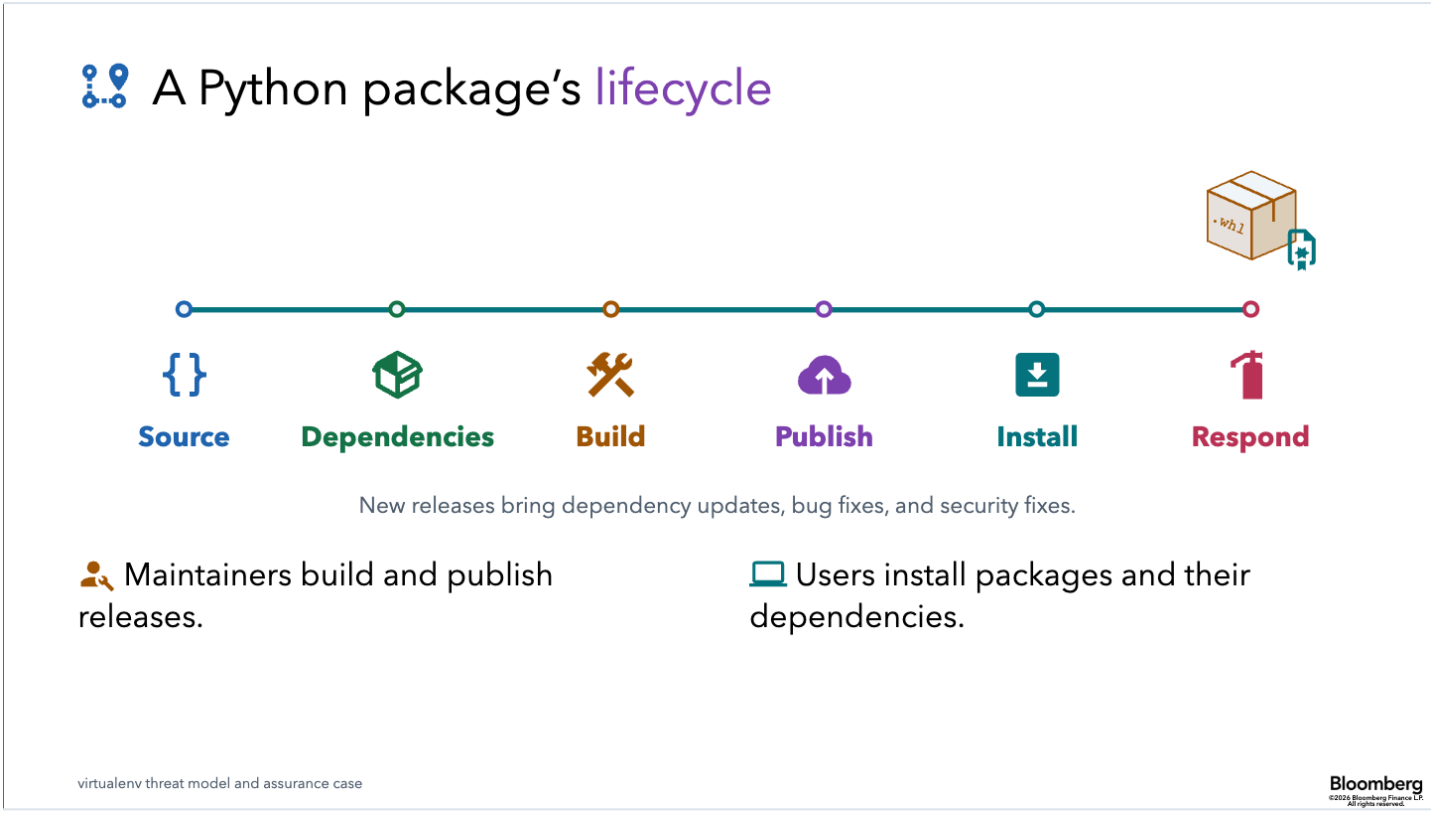
[02:03–02:40]

Before we open mine, the other way in. In April, a PR comment on Elementary's repository became shell code inside a workflow. Not a dependency, not a stolen file: text, pasted into a script by the workflow itself. The attacker got the workflow's GitHub token and used it to dispatch the release workflow.

[Click: Reveal the affected release.]

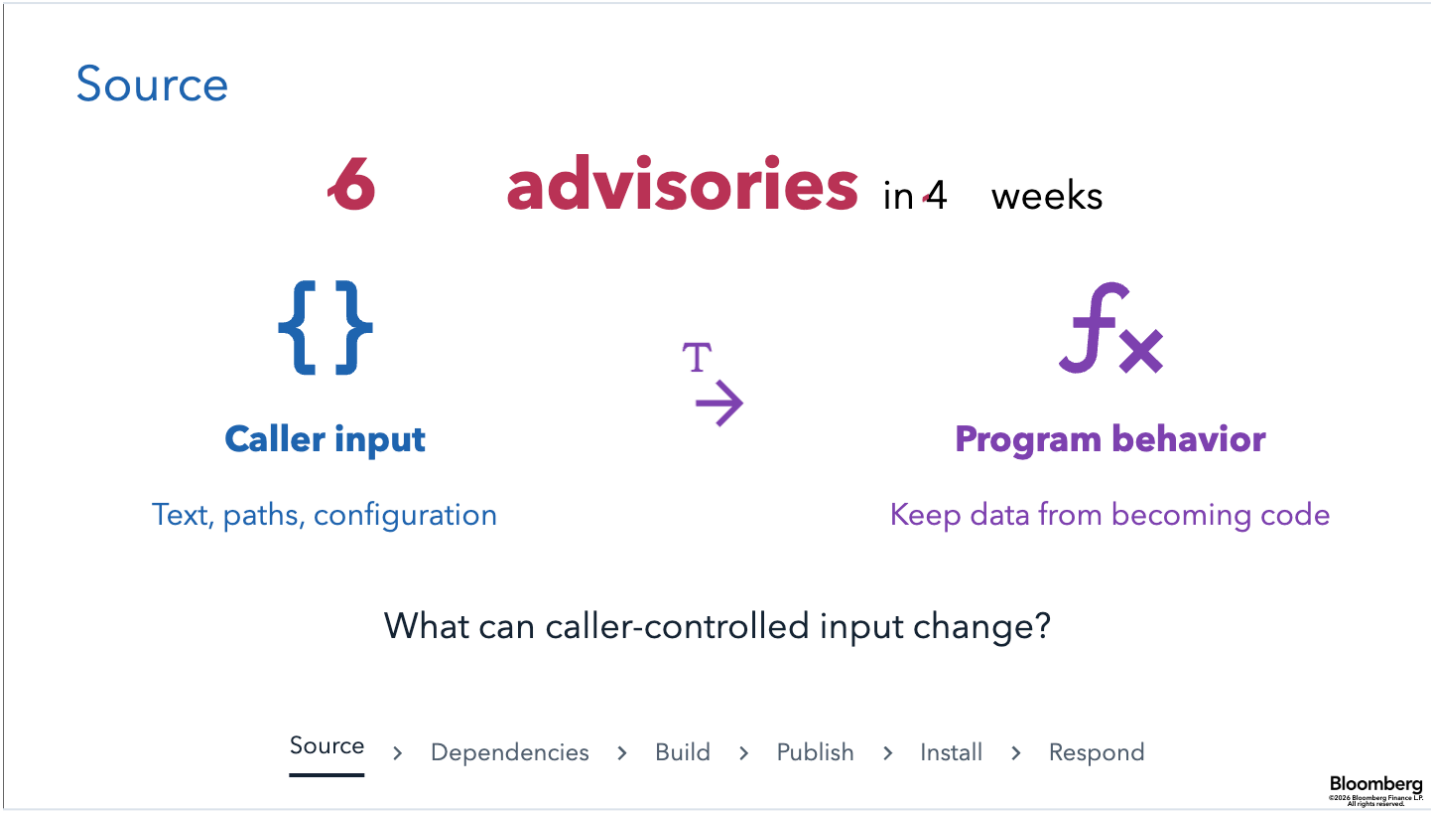
A malicious CLI release reached PyPI and Docker, and stayed there for eleven and a half hours.

Two incidents, two routes: a secret that a tool could read, and a string that a workflow executed. Hold on to those two, because seven of my nine are one of them.



[02:40–03:01]

Here's the map, so you know where we are. Source, dependencies, build, publish, install, respond. Maintainers own source through publish; the people installing own install; both own respond. The nine findings land on four of these stops. I'll go through the stops in this order, and close most of them with a field guide: the settings and commands on one slide, to read later.

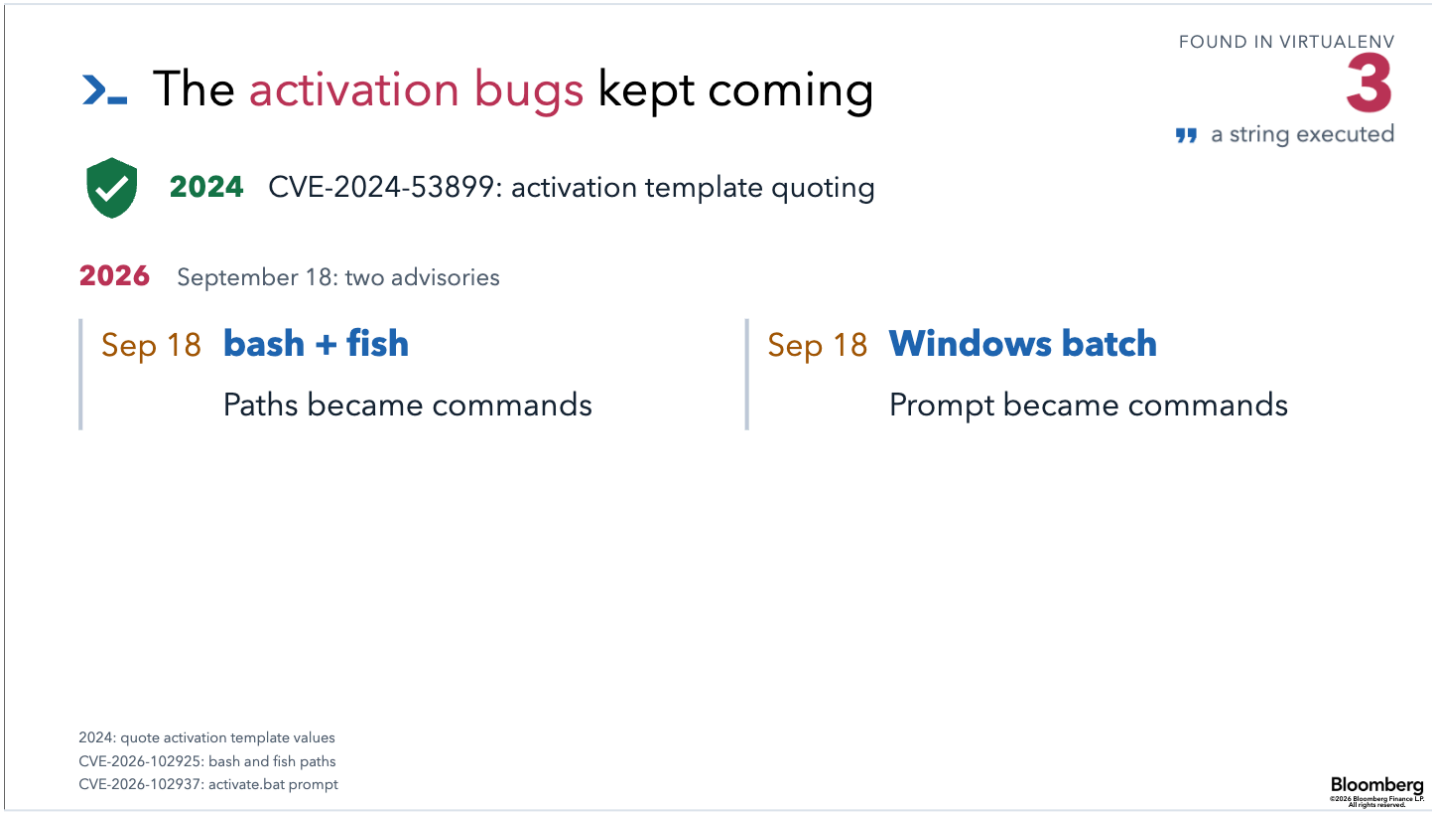


[03:01–03:18]

First stop is the code we ship, so I checked mine: the activation scripts, the files virtualenv writes for every environment, in six shell languages.

[Click: Strike the six; twenty-one slams in.]

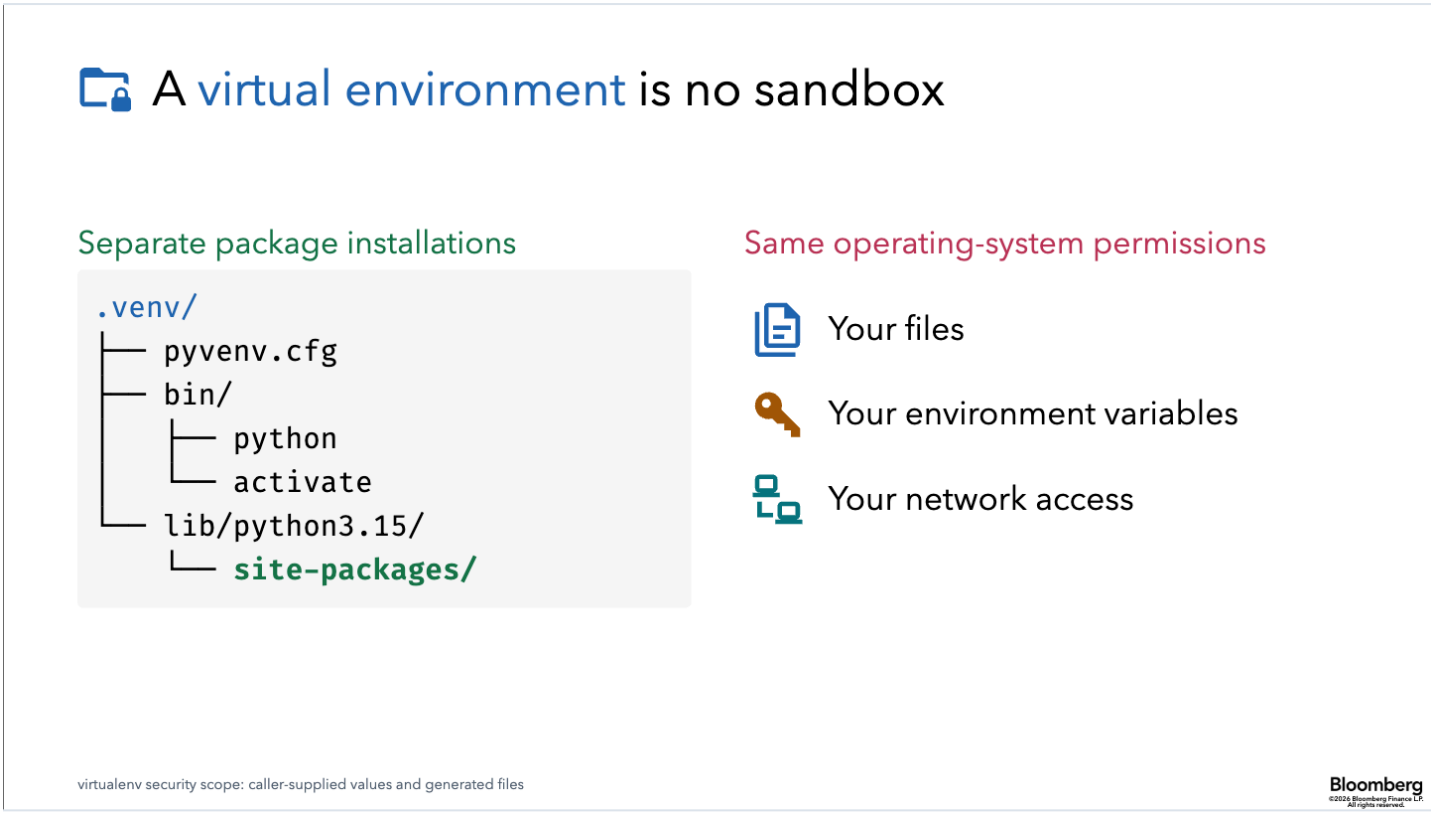
That number didn't hold. Counting every virtualenv advisory published since September 17, it's twenty-one in five weeks.



[03:18–03:55]

Creating an environment writes shell scripts containing its path and prompt. If either value becomes a command, activating runs it, with your permissions, on your machine. We fixed one quoting bug in 2024. In September, we fixed two more.

The bash and fish scripts quoted a value and then placed it inside a second pair of quotes, so a path with shell punctuation closed the string early and ran as code. The Windows batch activator didn't escape the prompt at all; a double quote in it was live cmd.exe syntax. Two and three, both mine: a value that should have stayed data and didn't.

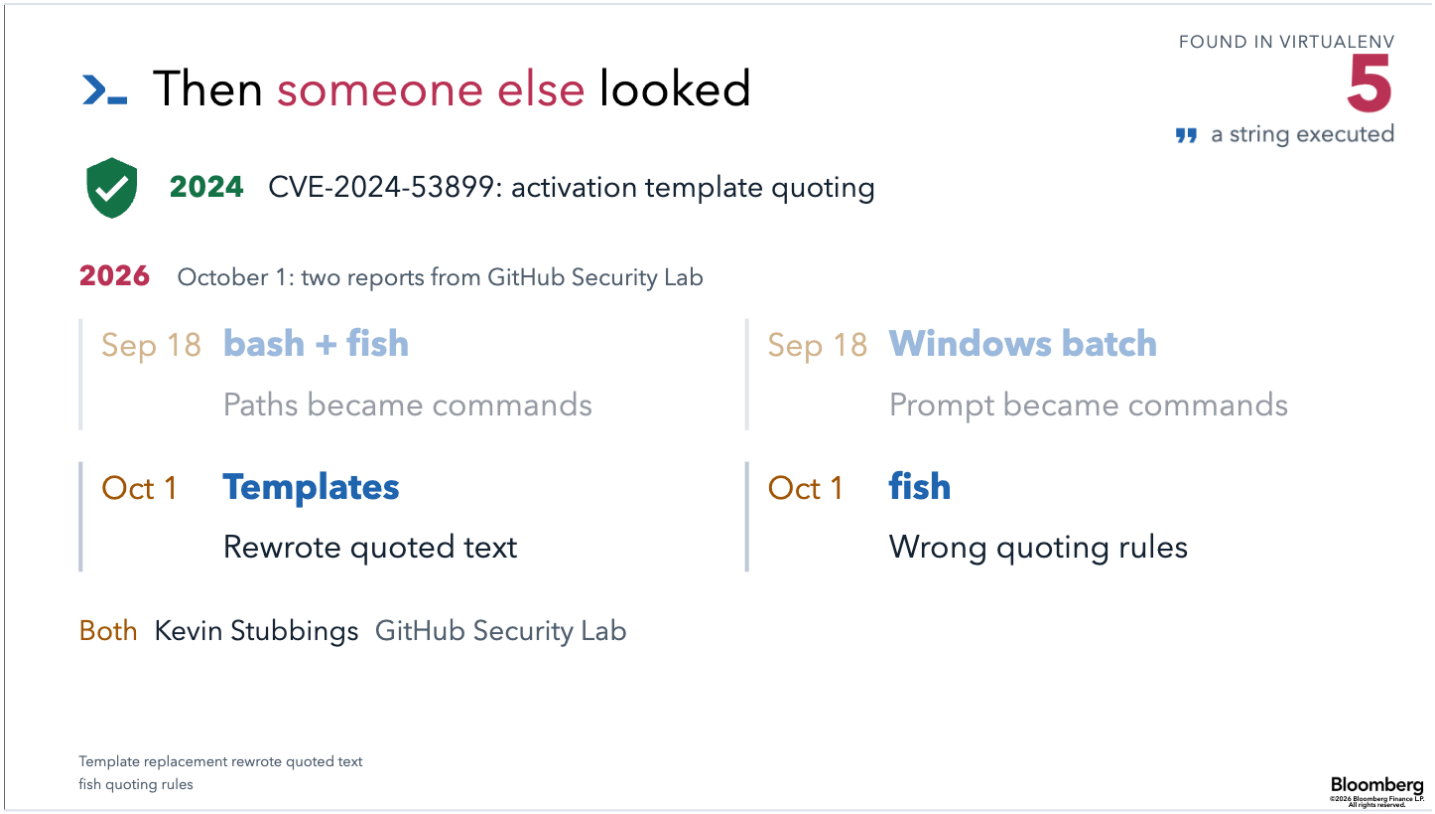


[03:55–04:08]

Why does that matter? A virtual environment separates packages, not permissions.

[Click: Reveal the shared operating-system permissions.]

Code in it runs as you: your files, your environment variables, your network. Activating changes none of that.

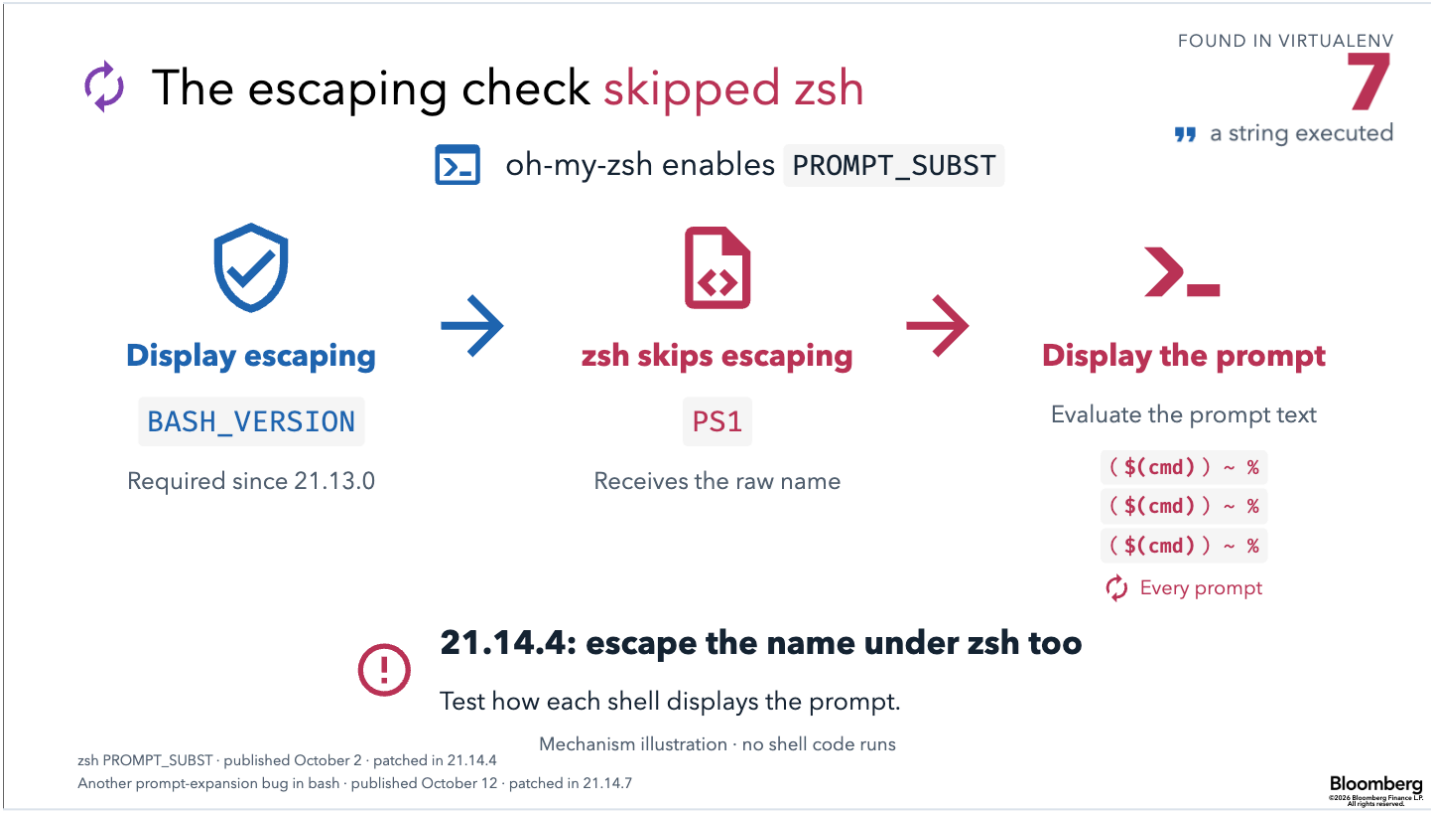


[04:08–04:44]

Two weeks later Kevin Stubbings from GitHub Security Lab reported two more. The template renderer replaced placeholders one at a time across the whole script, so a directory named after a placeholder got rewritten after it had already been quoted. The bug wasn't in quote; it was in what happened to quote's output. And fish reads a backslash inside single quotes differently from POSIX, so the quoting rules we'd borrowed from the shell library were the wrong rules for that shell. Four and five.

[Pause.]

I had CodeQL on this code, configured, with custom queries. We'll get to why it saw neither.

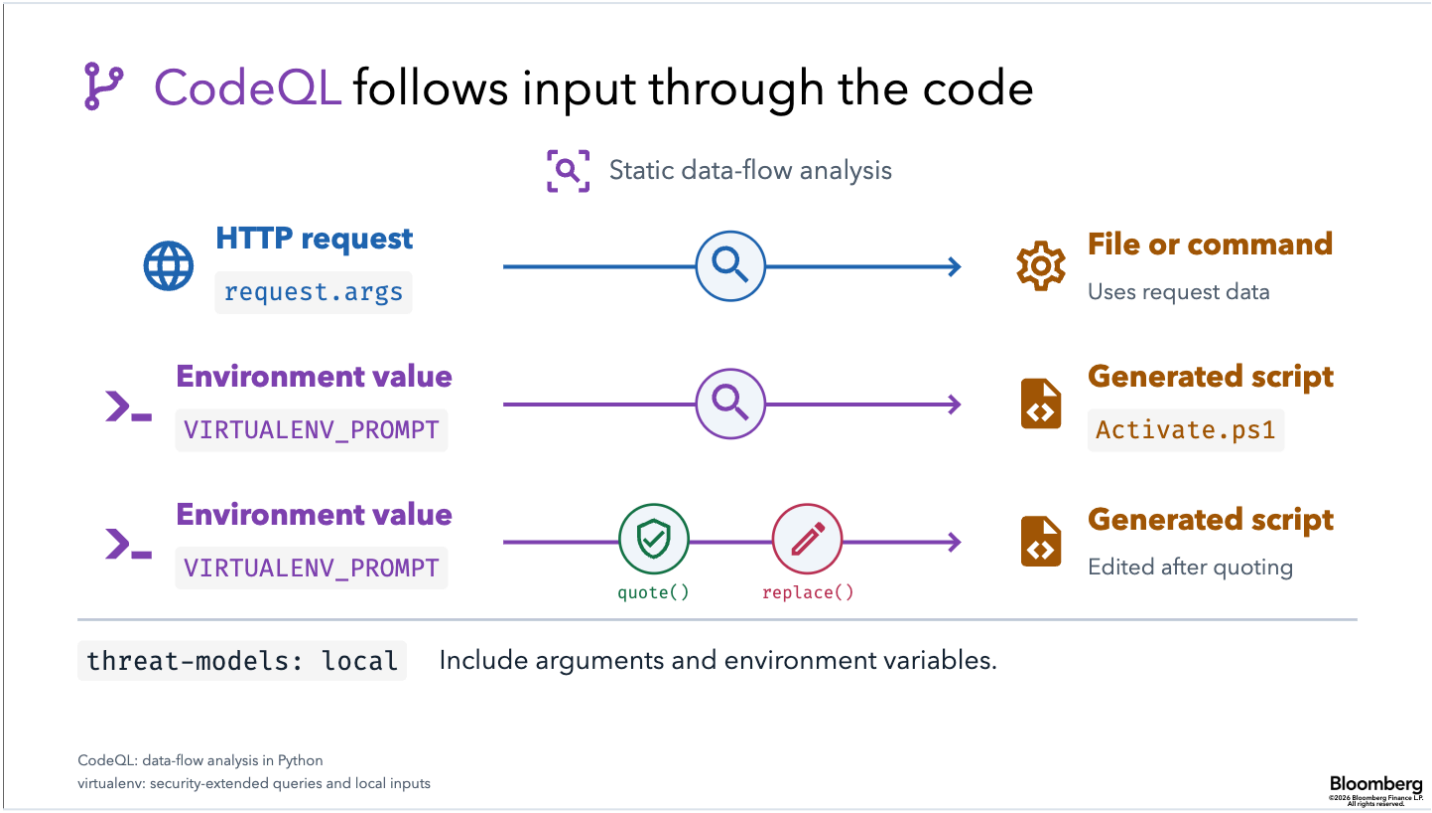


[04:44–05:28]

Six and seven are the same bug twice. Since 21.13.0 we escaped the environment name for prompt display, but only when BASH_VERSION was set. zsh got the raw value, and zsh users run the bash activate script.

[Click: Follow the value to prompt display; let the repeat icon turn.]

oh-my-zsh turns on PROMPT_SUBST, so zsh evaluates the prompt text every time it draws it. An environment name became a command, once per prompt, forever. Kaya Emre Arikan reported it; the fix went out October 2. On October 12 we published the bash version of the same thing: bash makes two passes over the prompt, and we'd escaped for one of them. Quoting the assignment was never the question. What the shell does with the value afterwards is, and every shell does something different.



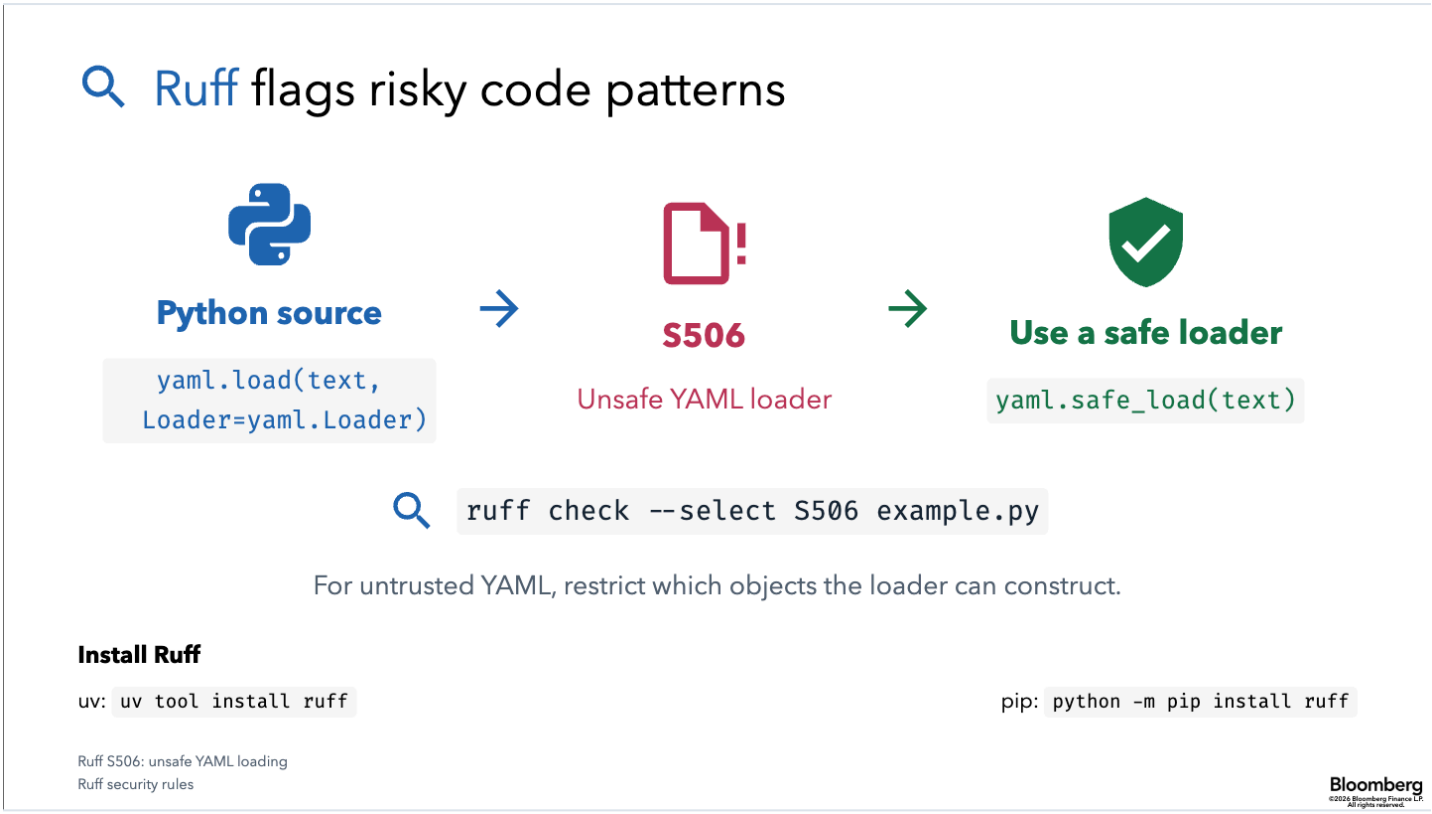
[05:28–06:12]

So why didn't CodeQL catch the October bugs? It follows a value from an input to the operation that uses it, and we count arguments and environment variables as input, not just HTTP requests.

[Trace the request through the scanner. Click: reveal the environment value.]

[Click: The value turns safe at the quote gate, then a replacement edits it.]

Our query treats a call to quote as a barrier: tainted value goes in, safe value comes out. That catches a missing call. It can't tell that the quoting rules are wrong for fish, or that a later replacement edits the result. The scanner modelled what we believed about the code. The bugs were in what we believed. So keep examples that must be flagged and examples that must pass, and when a report surprises you, add it before you fix it.



[06:12–06:34]

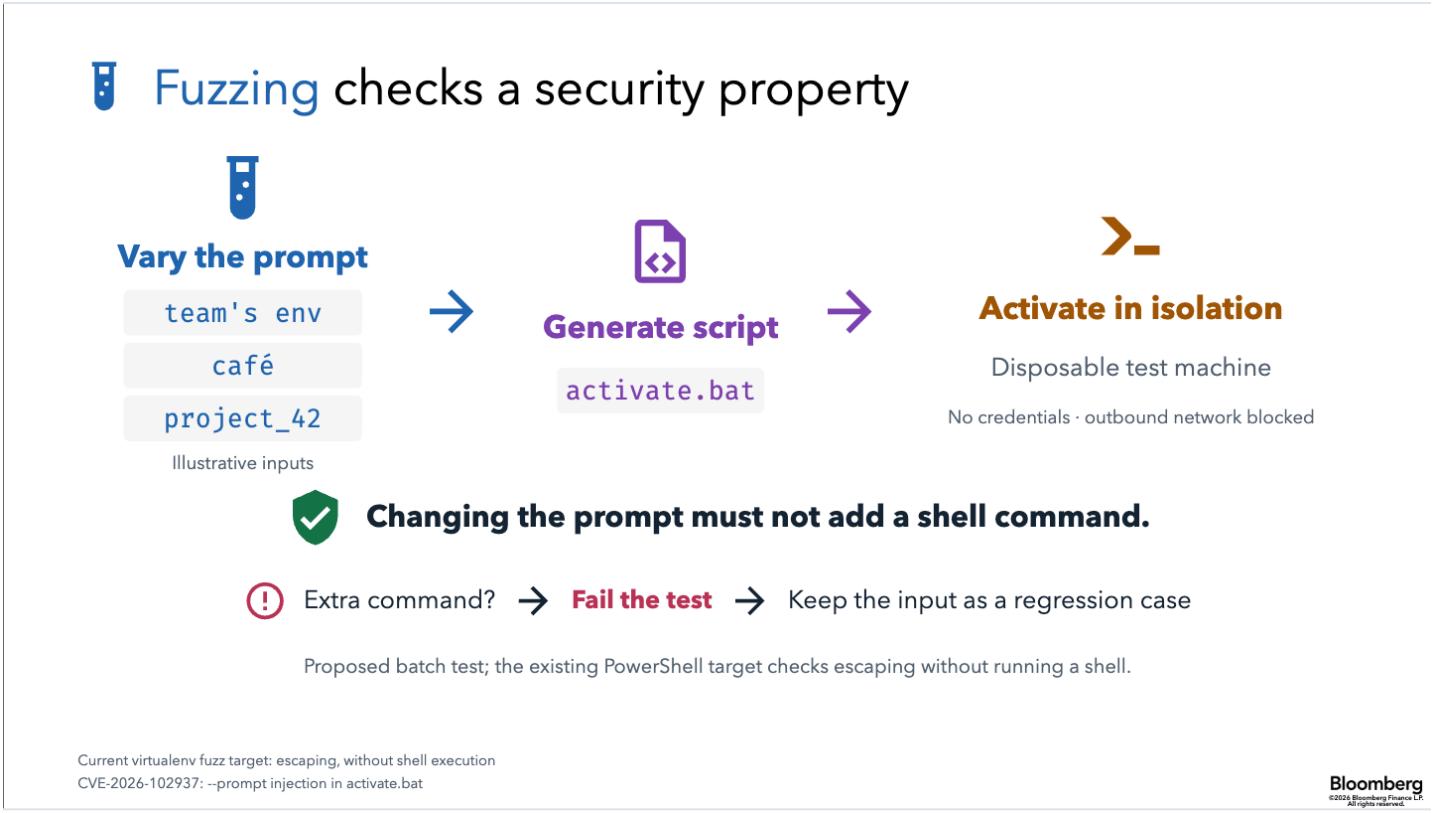
The other scanner here is Ruff. It flags a risky call without running anything: a YAML loader that builds arbitrary Python objects.

[Click: Reveal S506.]

S506 catches it.

[Click: Reveal safe_load.]

safe_load fixes it. Ruff sees the call, never where the input came from. Cheap, fast, and for a different bug than ours.



[06:34–07:05]

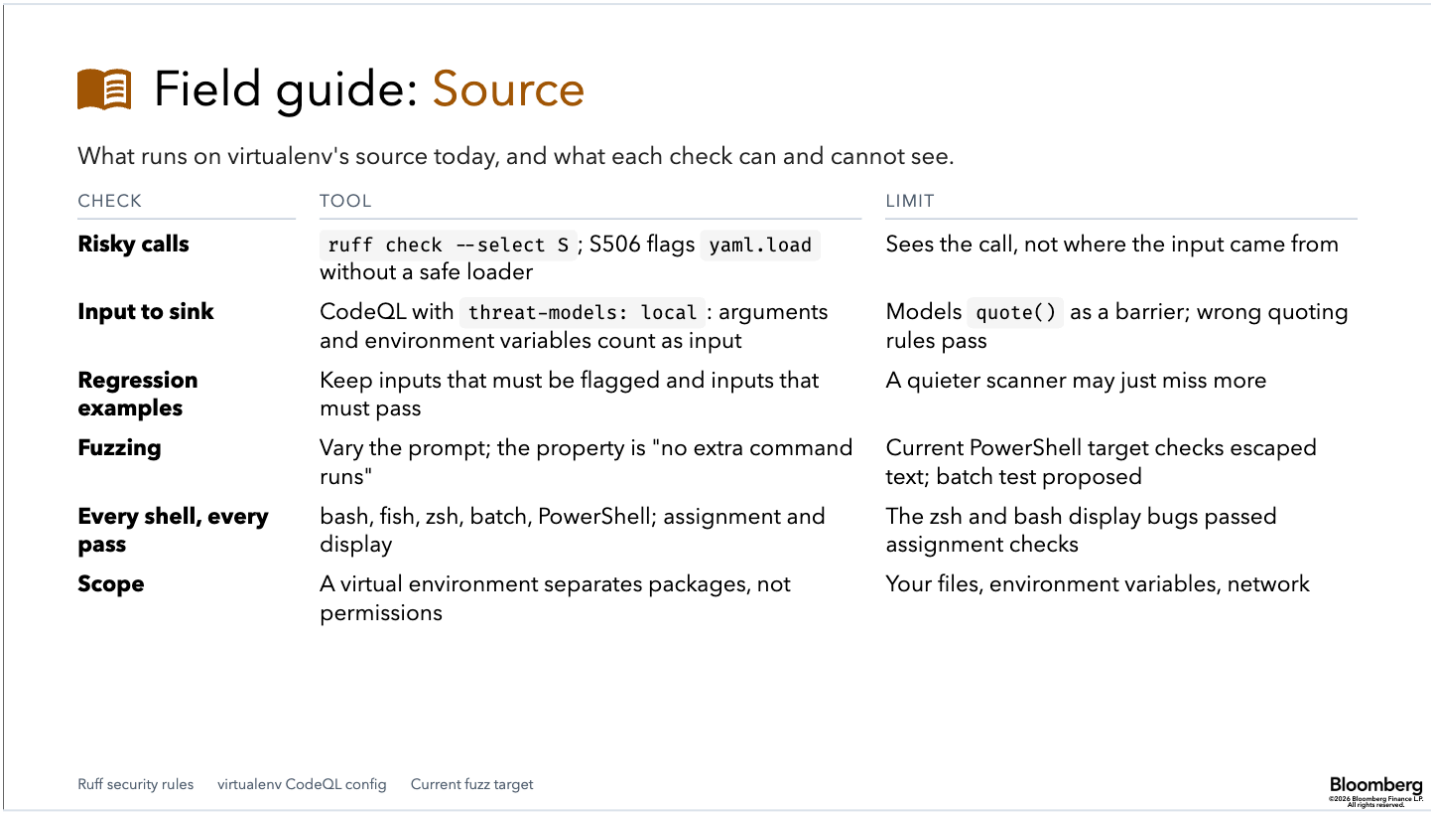
The batch bug is the one a fuzzer finds: vary the prompt; the rule is no extra command runs.

[Click: Follow the script into an isolated test.]

That means a disposable machine with no credentials and no network, and drawing the prompt, because two of the nine only fired at display time.

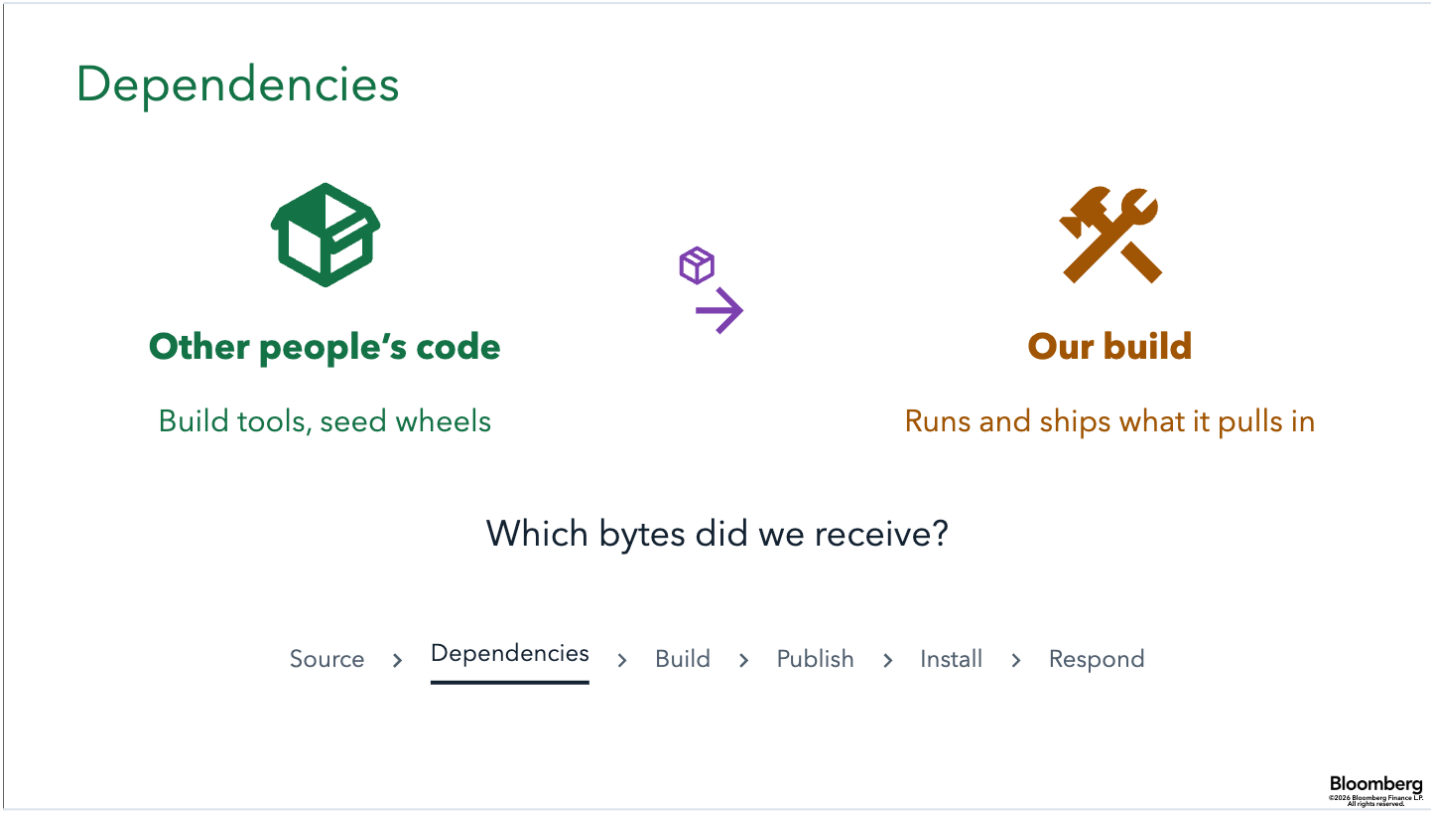
[Click: Keep a failing input as a regression case.]

This test is proposed. Our PowerShell fuzzer checks escaped text without running a shell, which is why it's a row in the guide and not yet code in the repo.



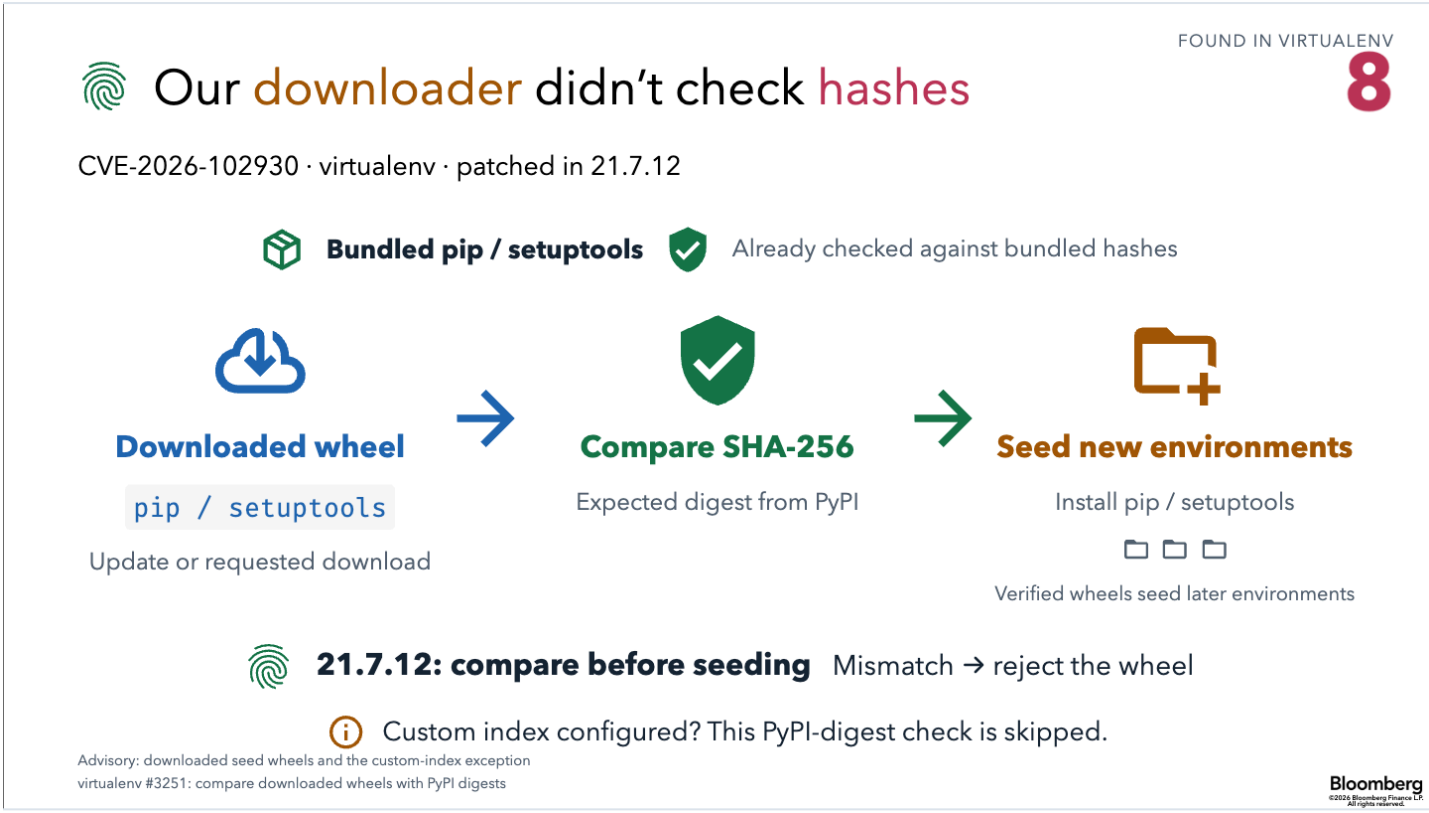
[07:05–07:16]

The source field guide: Ruff, CodeQL, regression examples both ways, fuzzing, every shell at assignment and display. One line: the scanner models what you believe about the code. Keep the inputs that surprised you.



[07:16–07:21]

Next, the code we pull in, and how to know you received the bytes you reviewed.



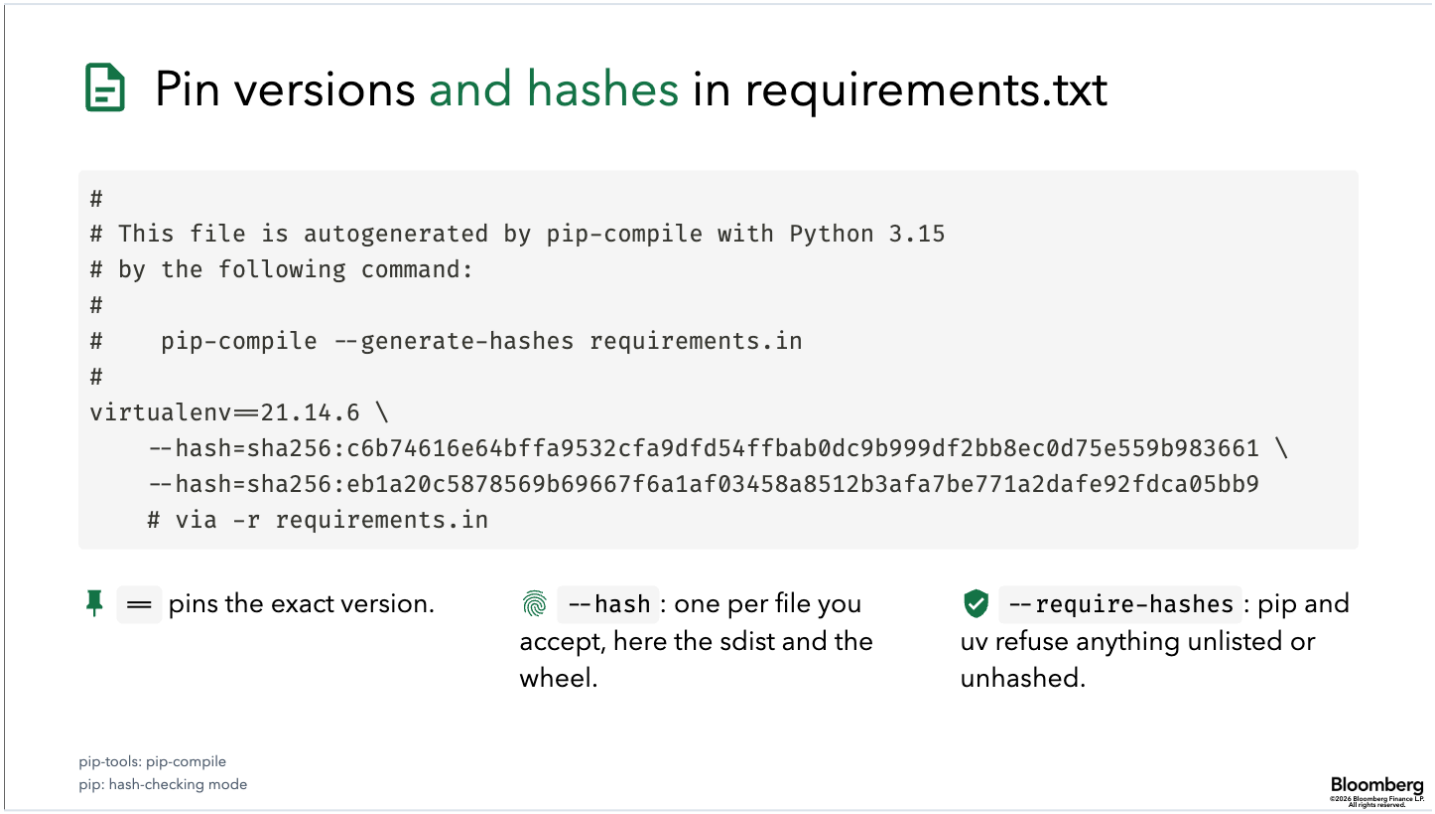
[07:21–08:01]

Then I checked mine: the downloader. virtualenv seeds every new environment with pip and setuptools wheels. The bundled copies ship inside virtualenv and are checked against hashes on every load. The downloaded updates weren't checked at all. Number eight.

An altered download could enter the cache and seed every environment created after it. A familiar filename says nothing about which bytes you received.

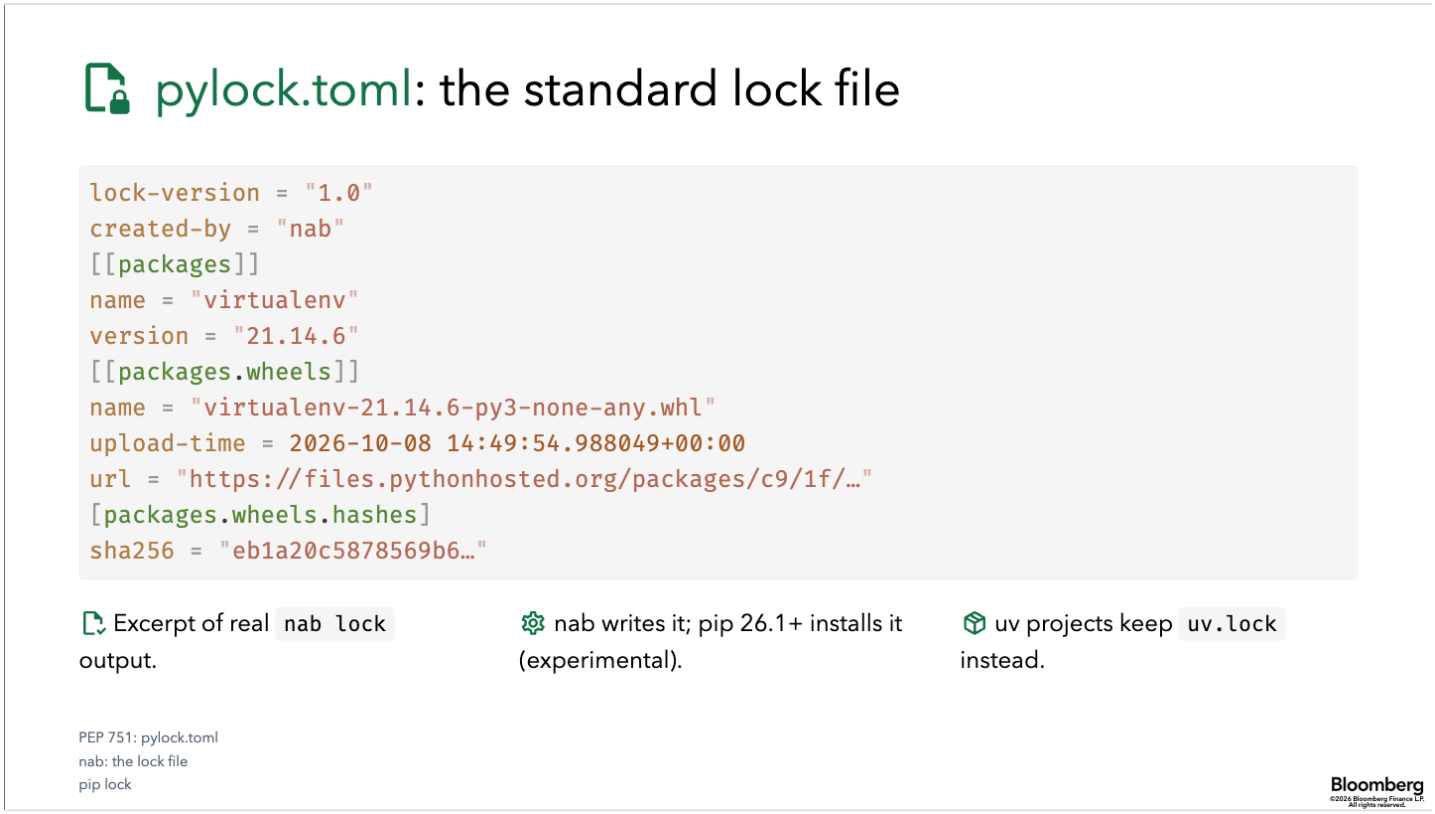
[Click: Put the digest check before seeding.]

The fix compares the download with PyPI's digest for that exact file, whichever index served it. It skips the comparison when a custom index is configured, because private indexes may legitimately serve rebuilt wheels, and that exception needs its own trust policy rather than a silent pass.



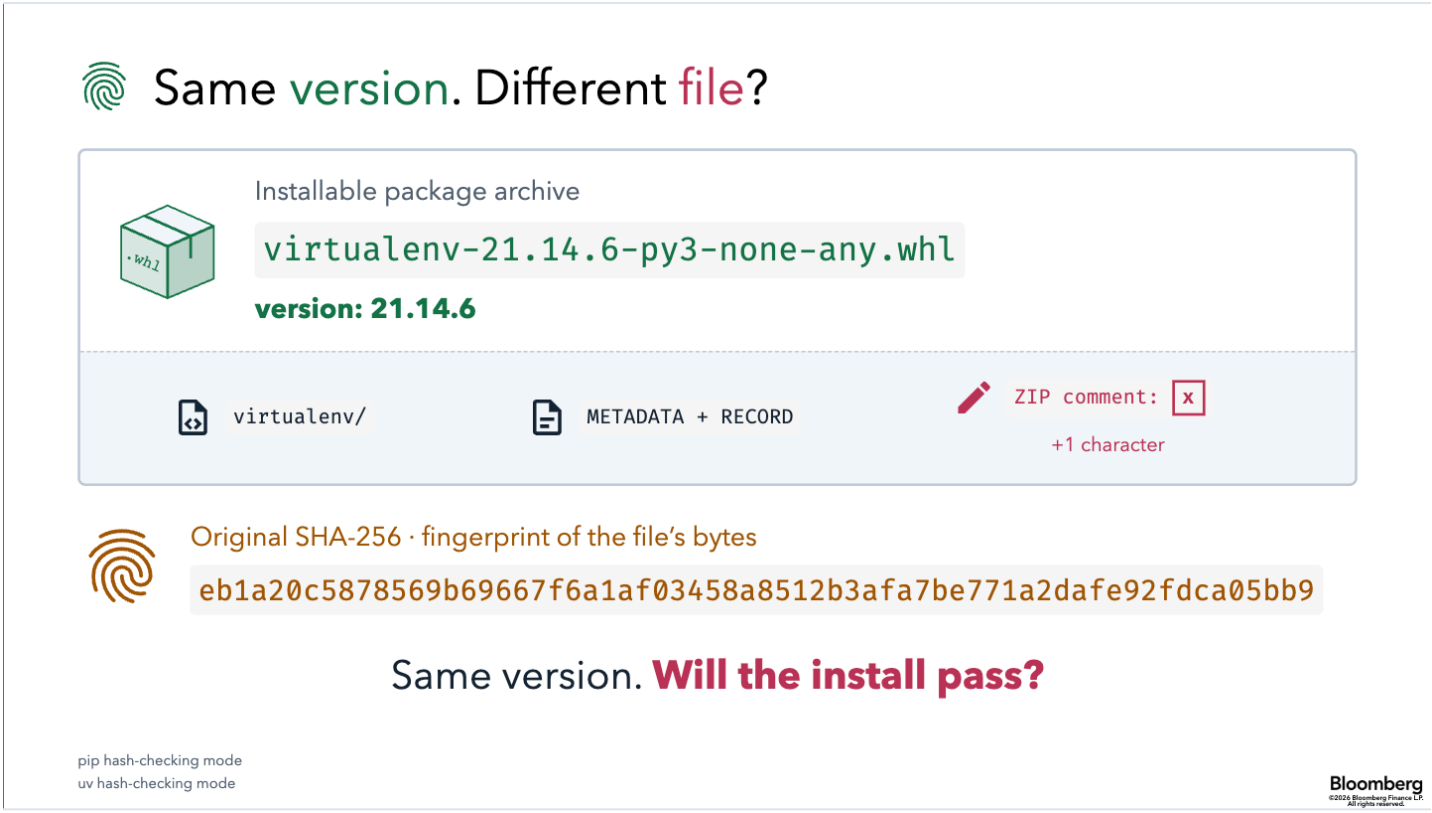
[08:01–08:26]

A requirements file can pin more than a version. Each entry names the exact version and the hash of every file you accept for it. Install with require-hashes and anything without a matching hash is refused, including dependencies you forgot to list.



[08:26–08:51]

pylock.toml is the standard version of the same idea: a TOML file that records, for every package, the version, where the file came from, and its hash. nab writes it with nab lock, and pip 26.1 can install it, though that support is still experimental. uv projects also keep uv.lock, uv's own lock file.



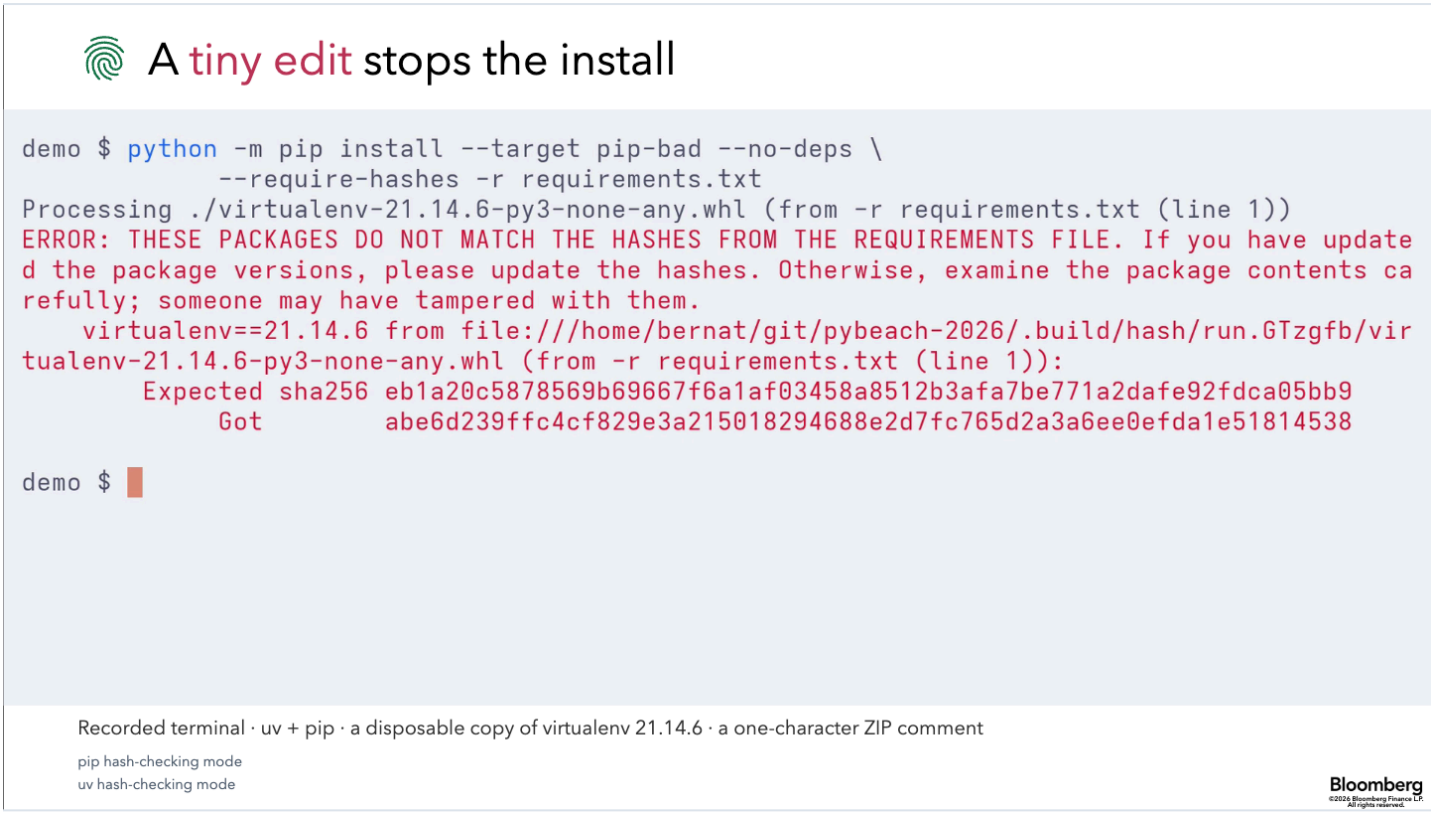
[08:51–09:18]

Let's see what a hash check catches. A wheel is a ZIP archive. PyPI won't let anyone overwrite an uploaded filename, so we'll change a local copy, the way a mirror, a proxy, or that cache could.

[Click: Reveal the question.]

We kept the original file's hash in our requirements. I'll keep the version number, keep the filename, and add a one-character ZIP comment. Will the install pass?

[Pause for three seconds, then advance.]



[09:18–09:52]

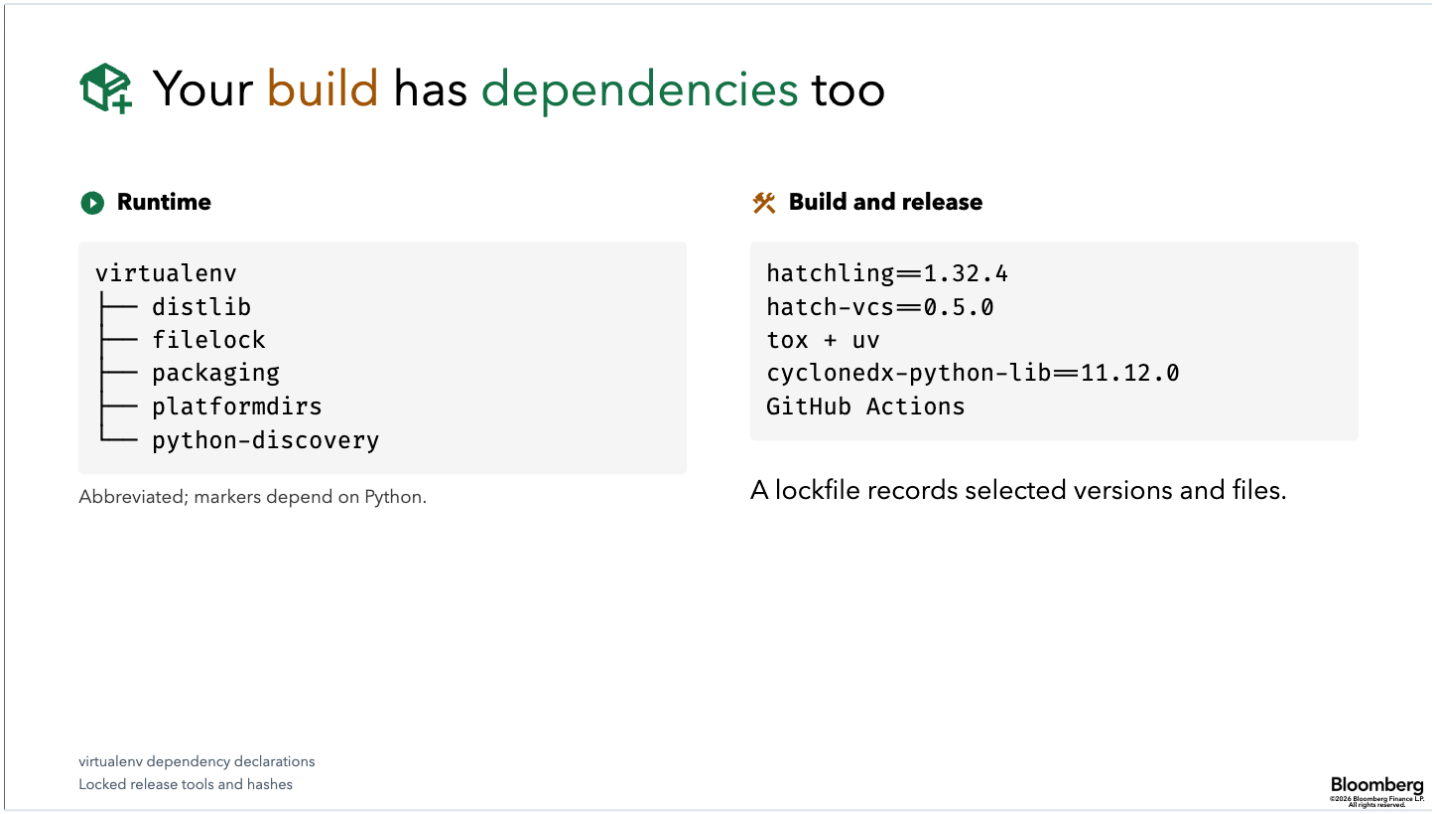
Same requirements file, uv and pip. Both accept the original file against the hash we recorded.

[Point to the successful installs, then the mismatch.]

One ZIP comment, and both reject it. That red error is the result I wanted.

[Pause.]

A hash still matches if you picked a malicious file to begin with; it proves these are the bytes you reviewed, nothing more. So keep the reviewed hash with the release inputs, and when verification fails, find out why. Replace the hash to make the error go away and the check becomes a suggestion.

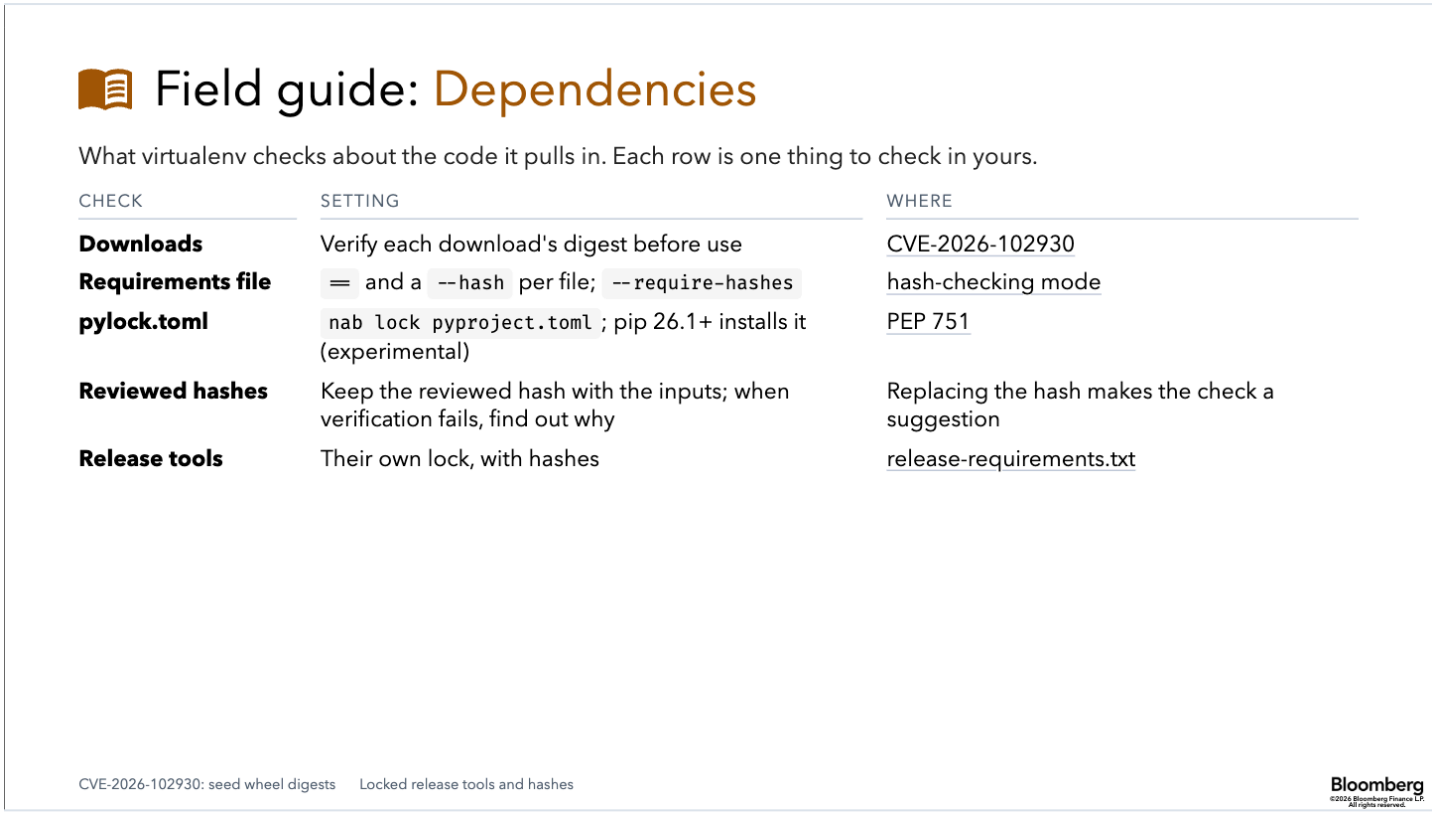


[09:52–10:16]

Building virtualenv runs other people's code: the build backend, the changelog tools, the SBOM generator. None of it lands on your machine; all of it can change the package you receive.

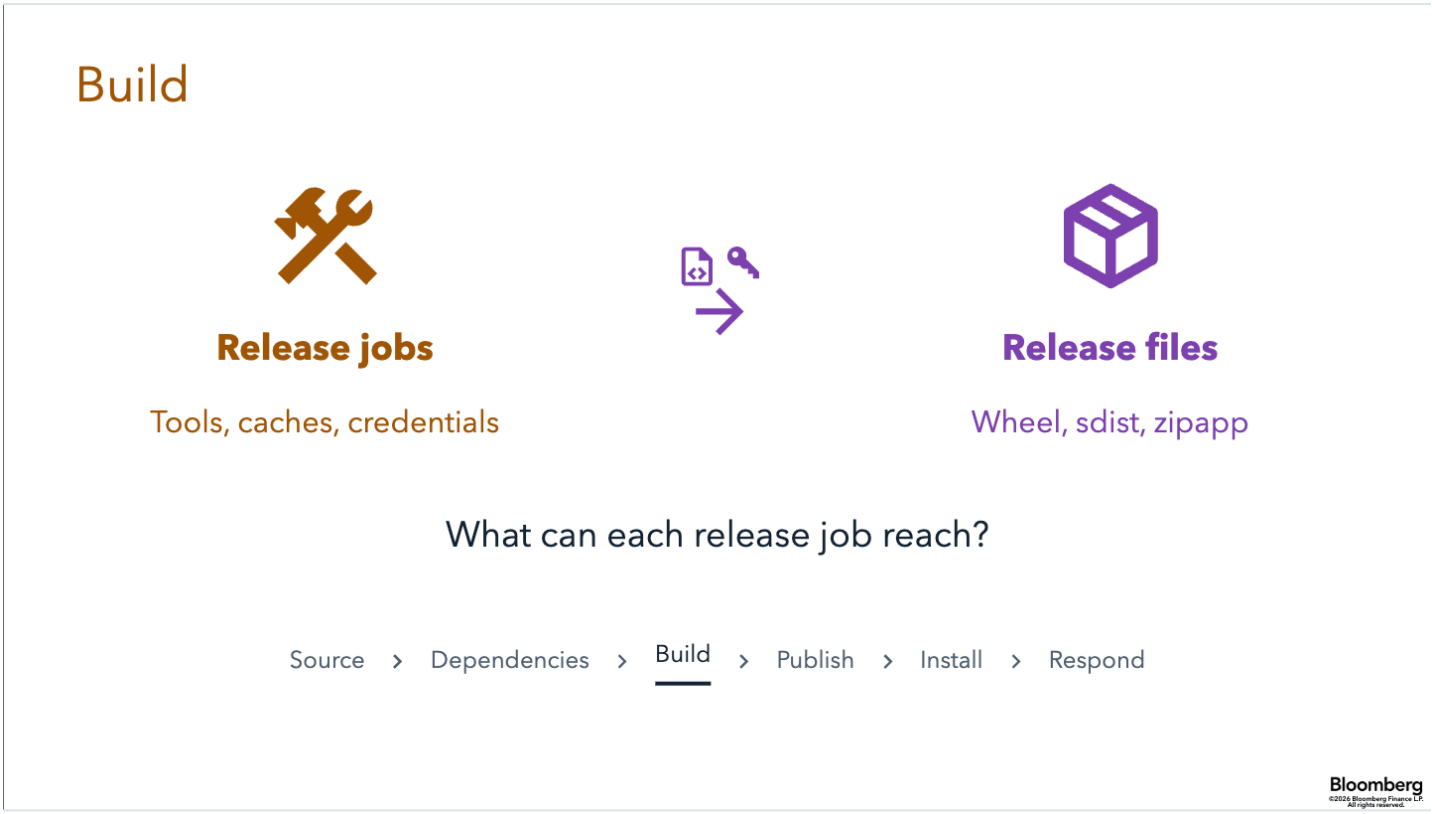
[Click: Reveal the build and release dependencies.]

So release tools get their own lock, with hashes. Calling something a development dependency doesn't limit what it can reach. Where we run it does.



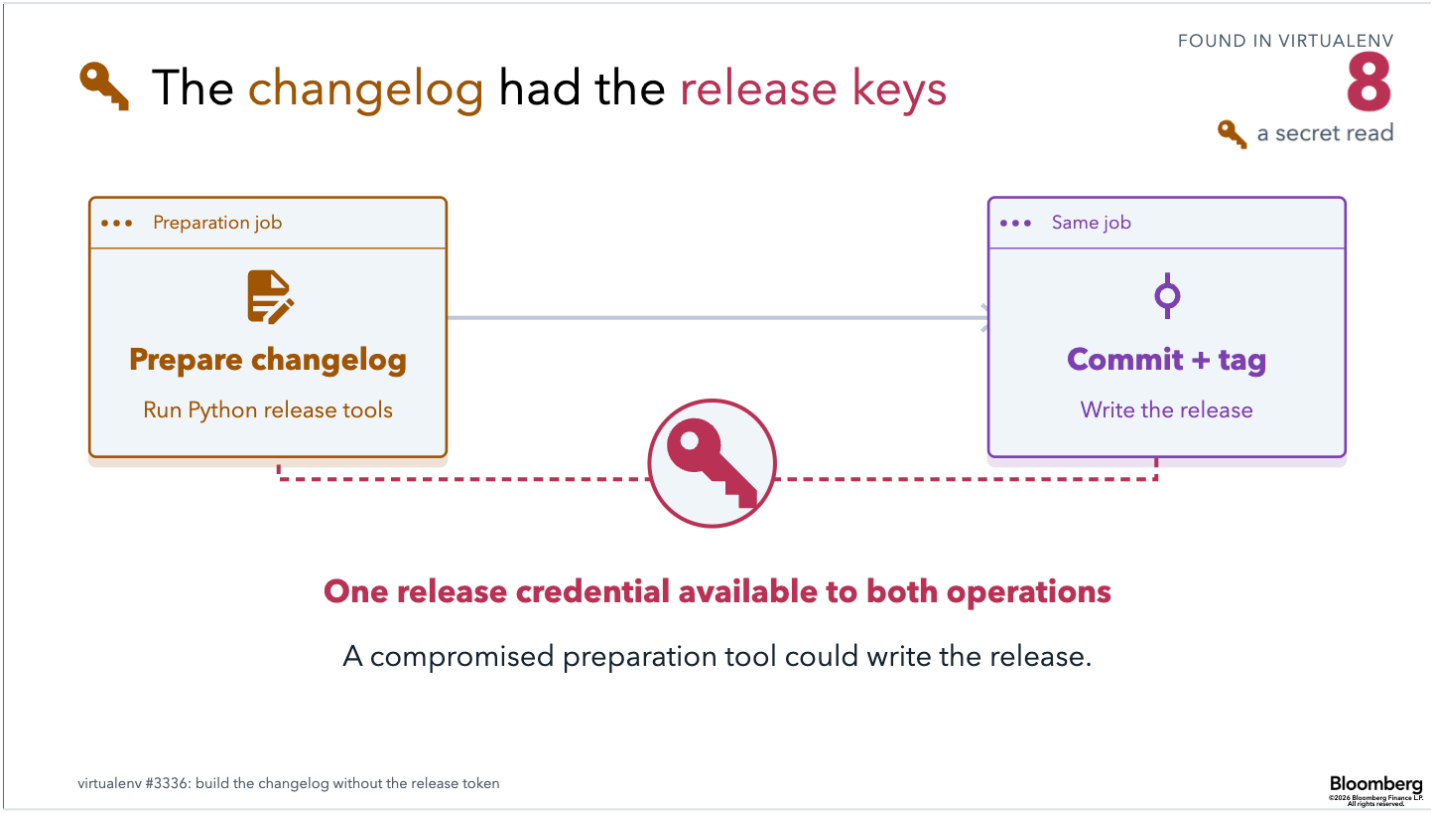
[10:16–10:24]

The dependencies field guide: digest checks on downloads, hashed requirements, pylock.toml, reviewed hashes, a lock for release tools. One line: a hash proves these are the bytes you reviewed, nothing more.



[10:24–10:29]

Now the release workflow, starting with the build.



[10:29–11:09]

Back to the first finding. Our release: generate the changelog, make a commit, and push a tag. Three steps, one job.

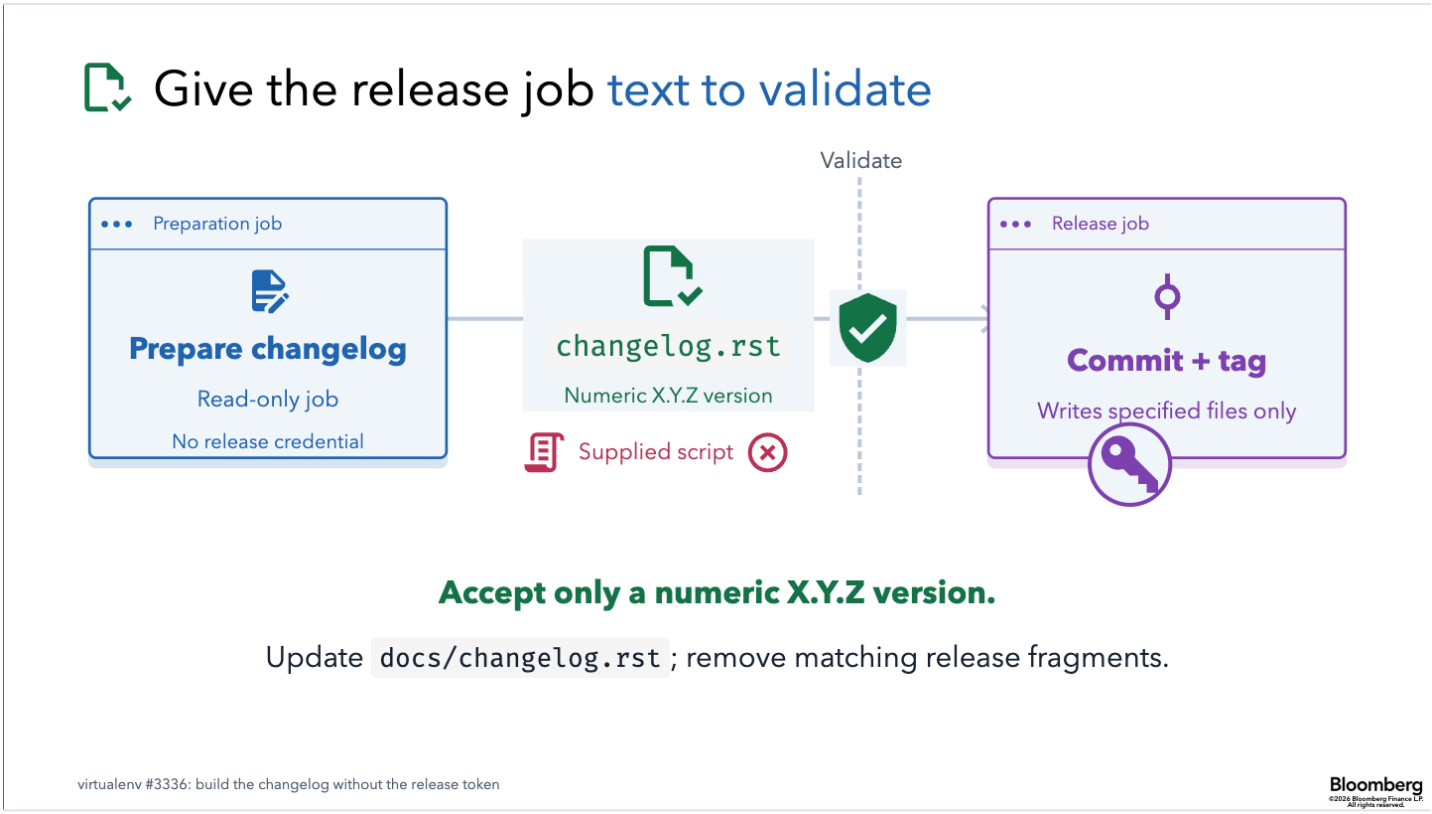
[Click: The keyring connects both operations. Let it settle before the joke.]

The preparation tools ran in a job with access to the credential used to write the release. A compromised one could have used that authority to publish whatever it liked, under our name, with our signature.

We asked it for bullet points and gave it the release keys.

[Pause.]

To generate a changelog, you need source files and somewhere to write text, nothing more. The release credential can go to a much smaller operation, and the preparation tools don't need to know it exists.



[11:09–11:44]

We split the job in two. The first runs the Python tools that generate the notes, without a release credential. It can be compromised; it can't publish.

[Click: The key moves to the release job. Follow the text through validation; the supplied script stays outside.]

It passes a changelog and a version to the privileged job. That job accepts only a numeric X.Y.Z version, and through the GitHub API it updates the changelog file and deletes the matching fragments. Nothing else. They can still write bad release notes. They can't commit code. That's the shape of every fix in this talk: shrink what the untrusted part can reach, then check what crosses the line.

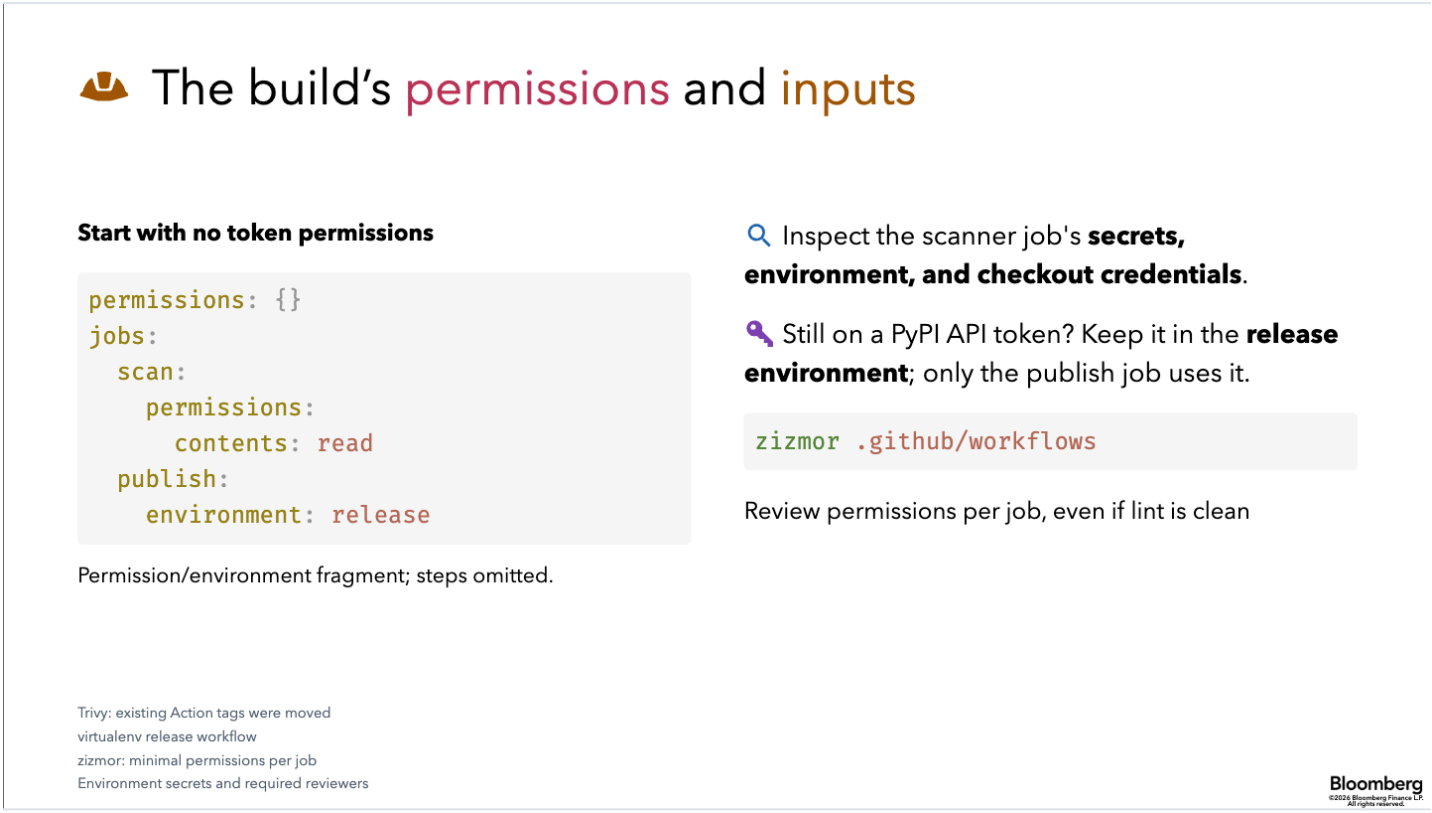


[11:44–12:25]

Elementary's bug is a lint finding, and this is the linter. zizmor checks GitHub Actions workflows for security mistakes. Here, the pull request title appears directly in the run field. GitHub substitutes it before the shell ever parses the script, so a title with a semicolon in it is a command. That's template injection.

[Follow the finding and corrected file.]

The fix puts the title in an environment variable and passes its quoted value to printf. The shell parses a fixed script and treats the contents as data, the same principle as the activation-script fixes. We run zizmor on every workflow change, and it reads the file; it never runs it.



[12:25–12:56]

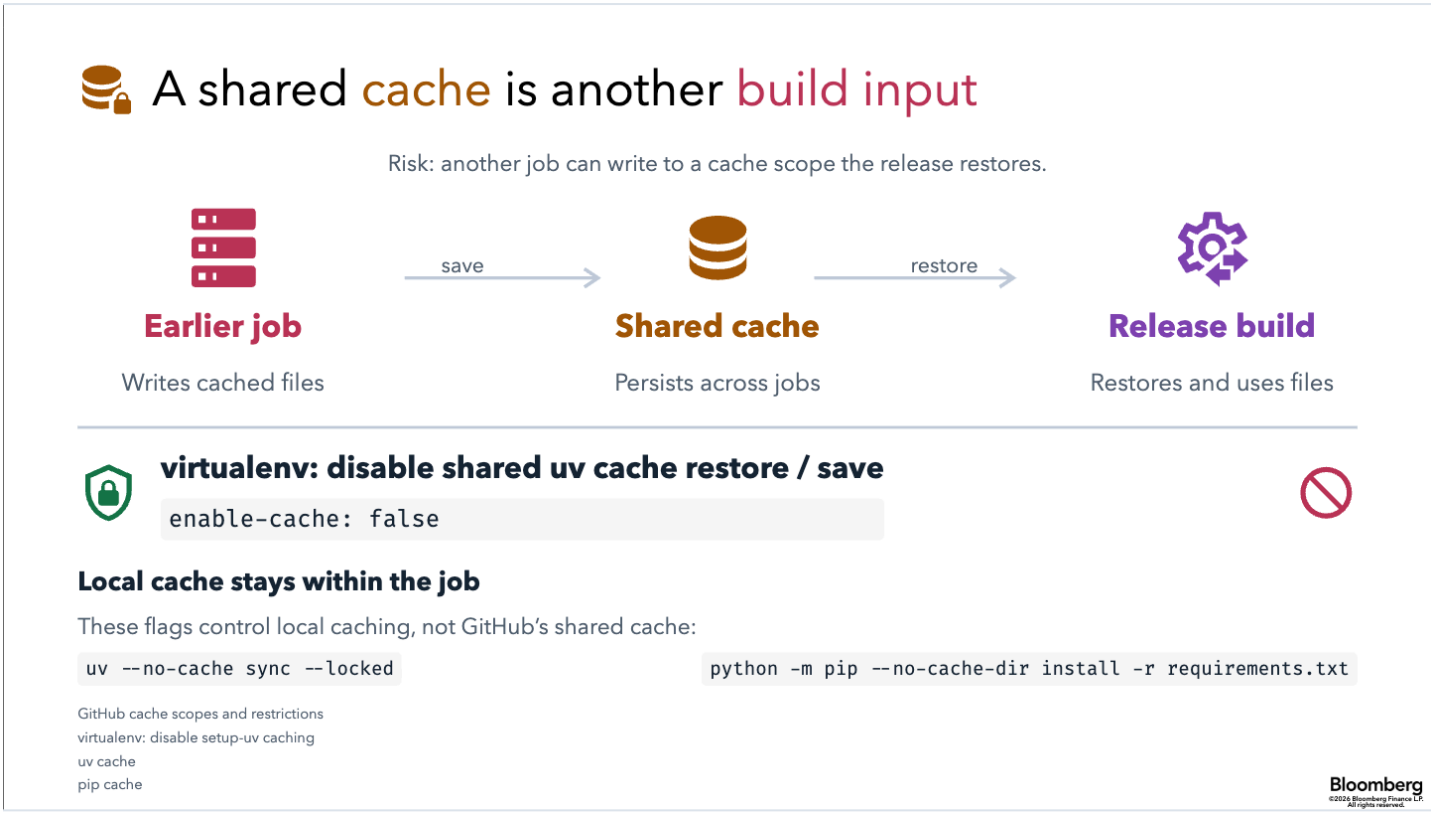
The Trivy campaign also moved trivy-action's version tags to malicious commits; Actions users were hit without changing a line. Pin Actions to reviewed commits, and check what the Action downloads, because a pin promises the same code, not good code.

[Follow the inputs. Click: remove the checkout credential.]

Checkout leaves a credential in Git's configuration; turn that off.

[Click: inspect the scanner's access.]

Then the question from the first slide: open the scanner job, read its secrets, environment, and checkout credentials. Empty permissions at the top, granted per job.

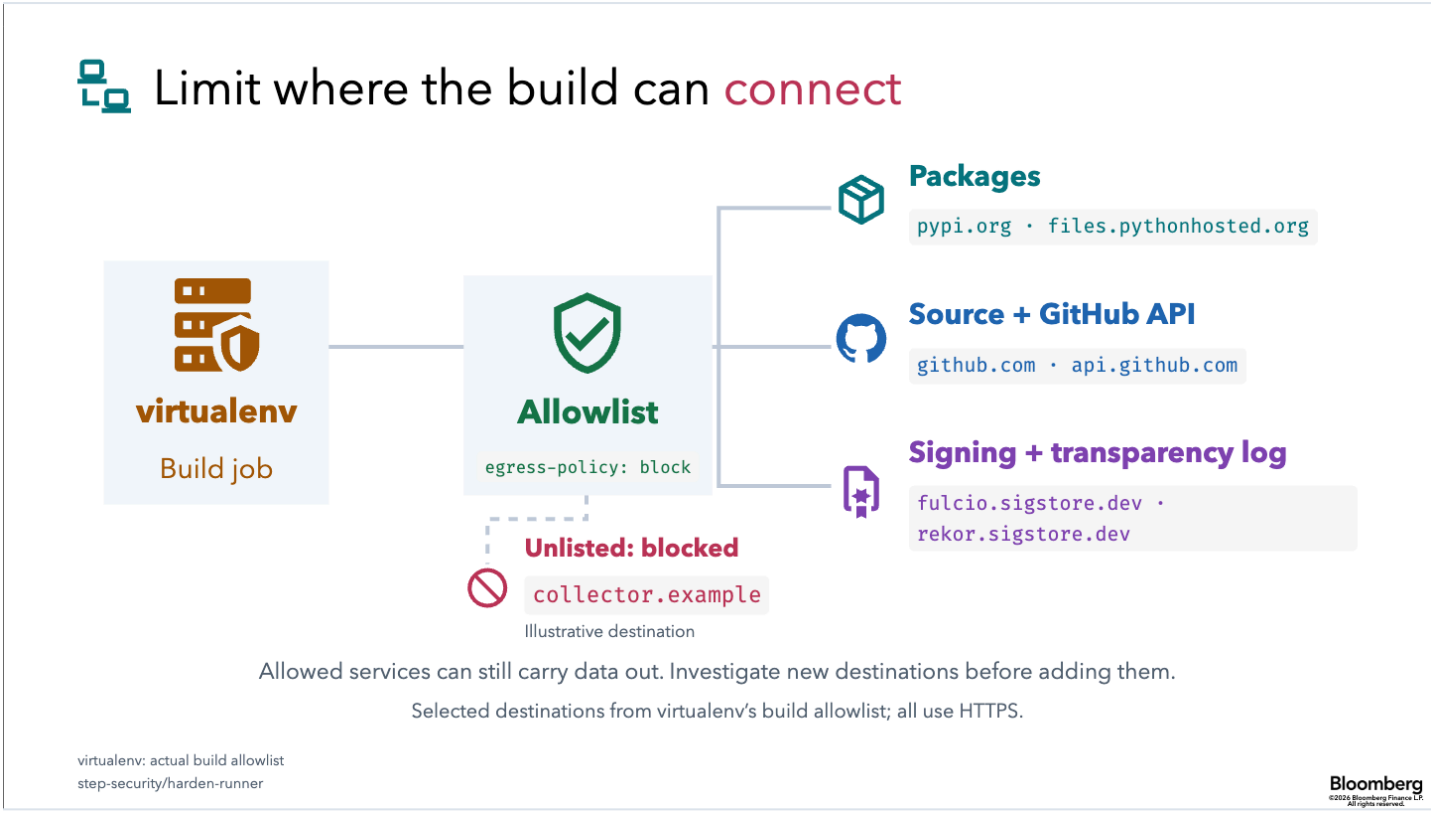


[12:56–13:18]

A cache carries files from one job into the next. If a less-trusted job can write a cache the release restores, it's inside your build. A fork's pull request doesn't get that by default. Ultralytics' did; we'll meet it later.

[Click: stop shared-cache restoration.]

Our release restores no shared cache. The no-cache flags only control uv's local copy, not GitHub's restore step.

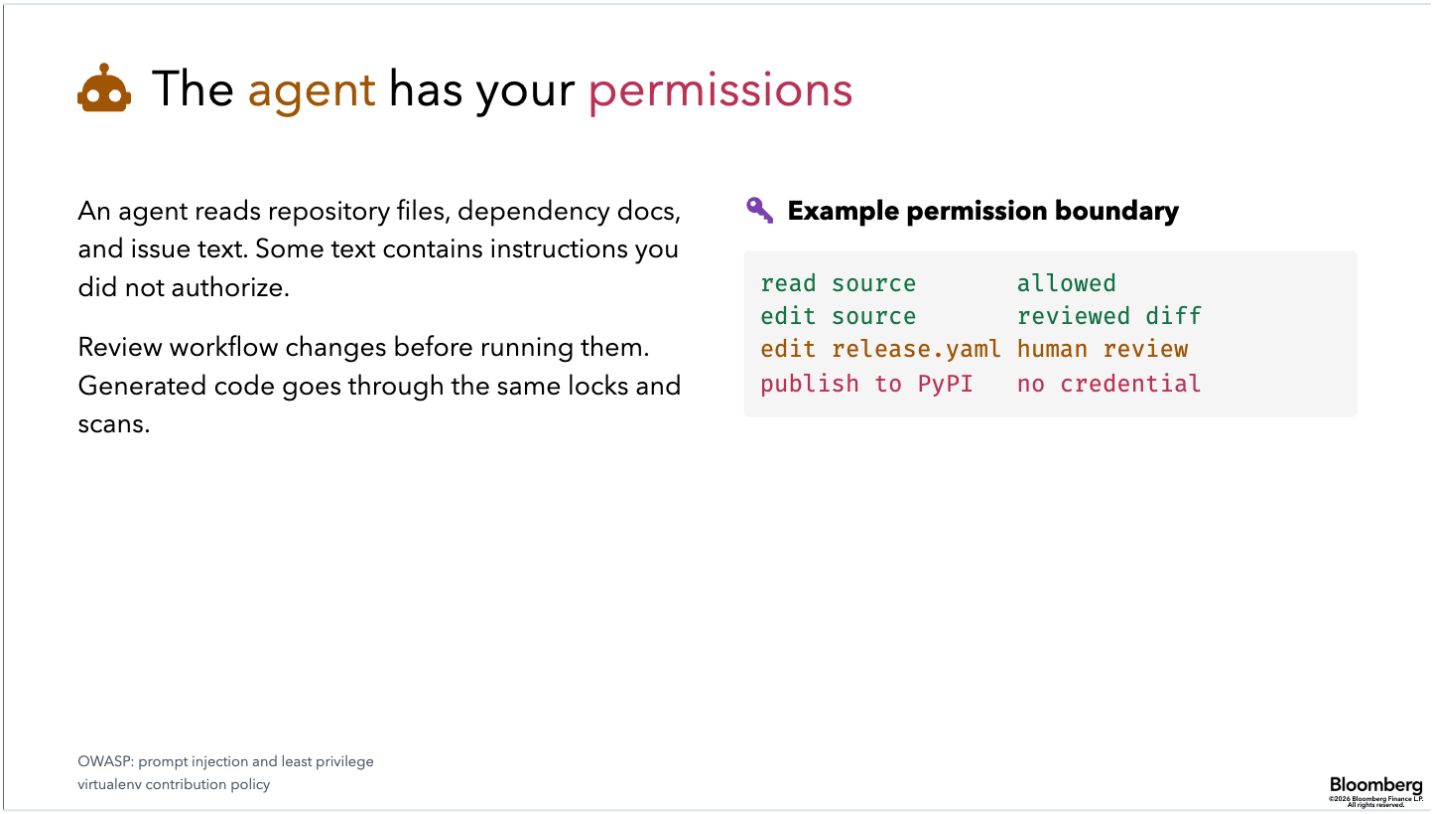


[13:18–13:37]

The build needs PyPI, GitHub, and Sigstore, so the job allows exactly those.

[Follow an allowed request, then the blocked request. Click: reveal the limitation.]

A blocked request is an investigation, not a line to add. Allowed services can still carry data out, so this narrows a compromised build; it doesn't replace the permission work.

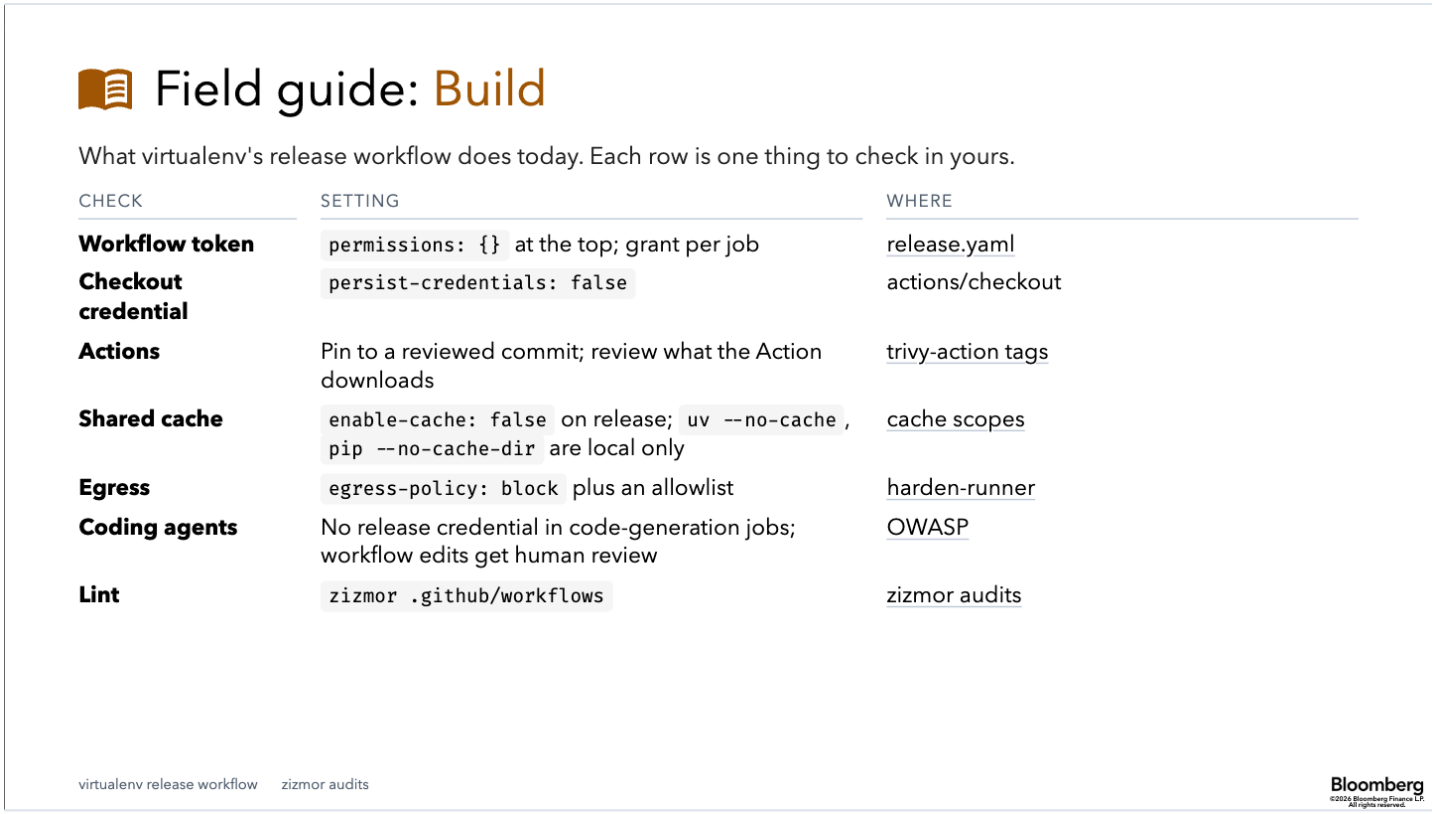


[13:37–13:59]

One more reader of untrusted text with your permissions: a coding agent. Repository files, docs, issue text; some of it contains instructions you didn't write. An agent without a publishing token can't publish. It can open a PR asking for one. Politely.

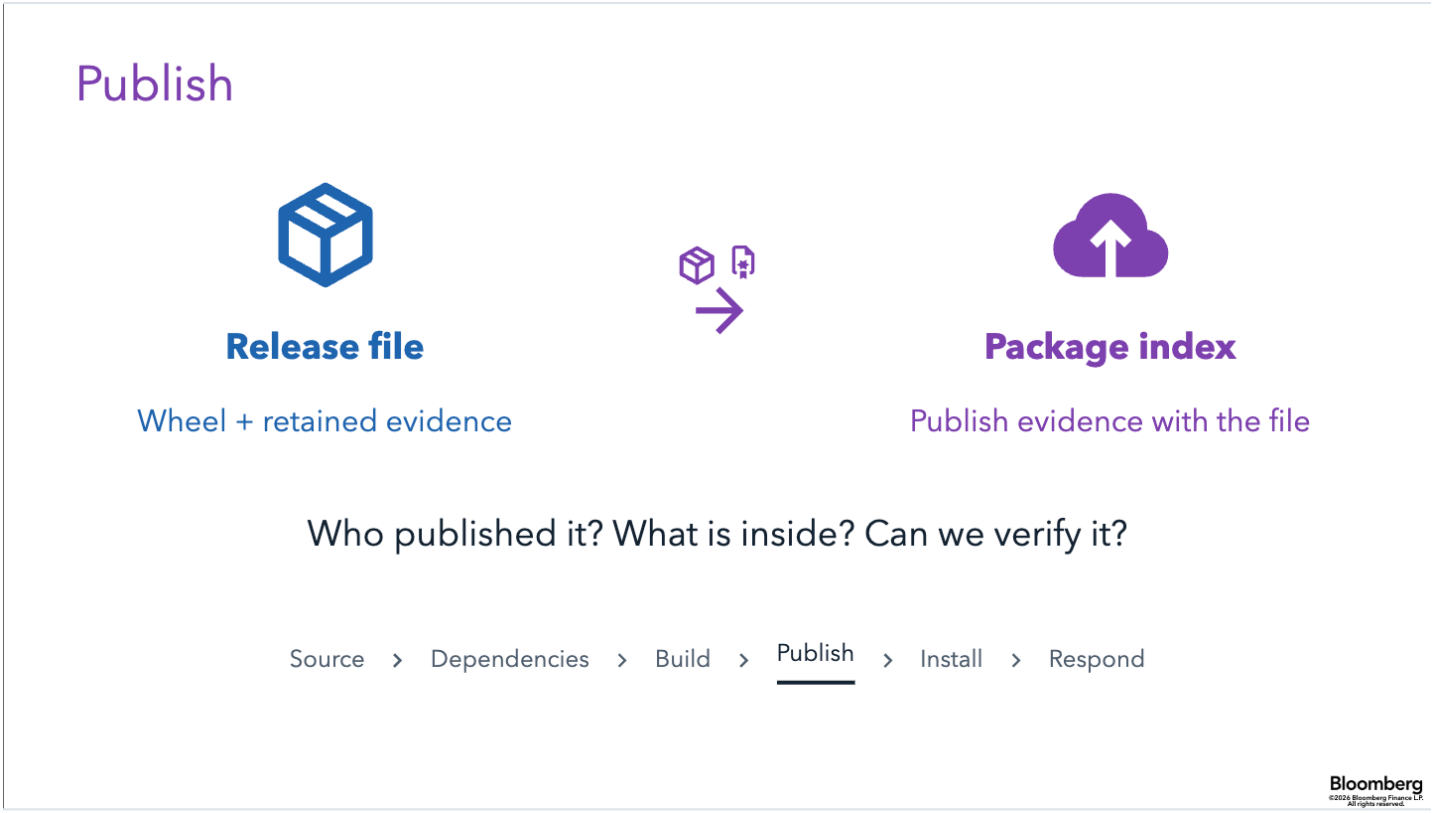
[Click: Reveal the example permission boundary.]

Same boundary as any tool: no release credential, and workflow edits get human review.



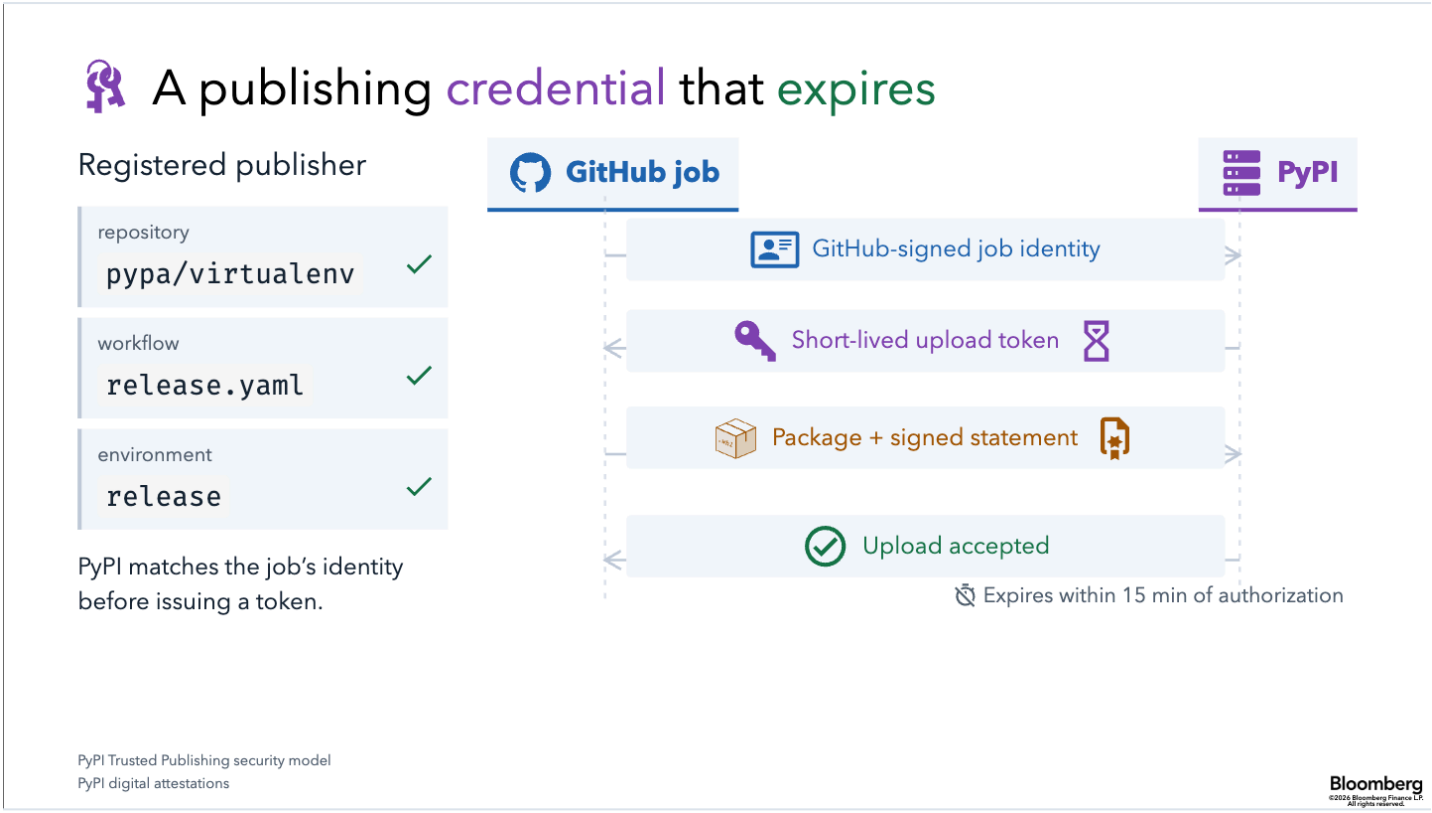
[13:59–14:10]

The build field guide: one row per check, for later. One line: start from permissions-empty and grant per job. Everything else follows from asking what each job can reach.



[14:10–14:15]

Publishing comes next: who can upload, and what users can verify about the file.



[14:15–14:53]

And the token the changelog job could reach? Gone. With Trusted Publishing, the release job proves to PyPI who it is.

[Click: GitHub sends the signed identity statement; the registered fields match.]

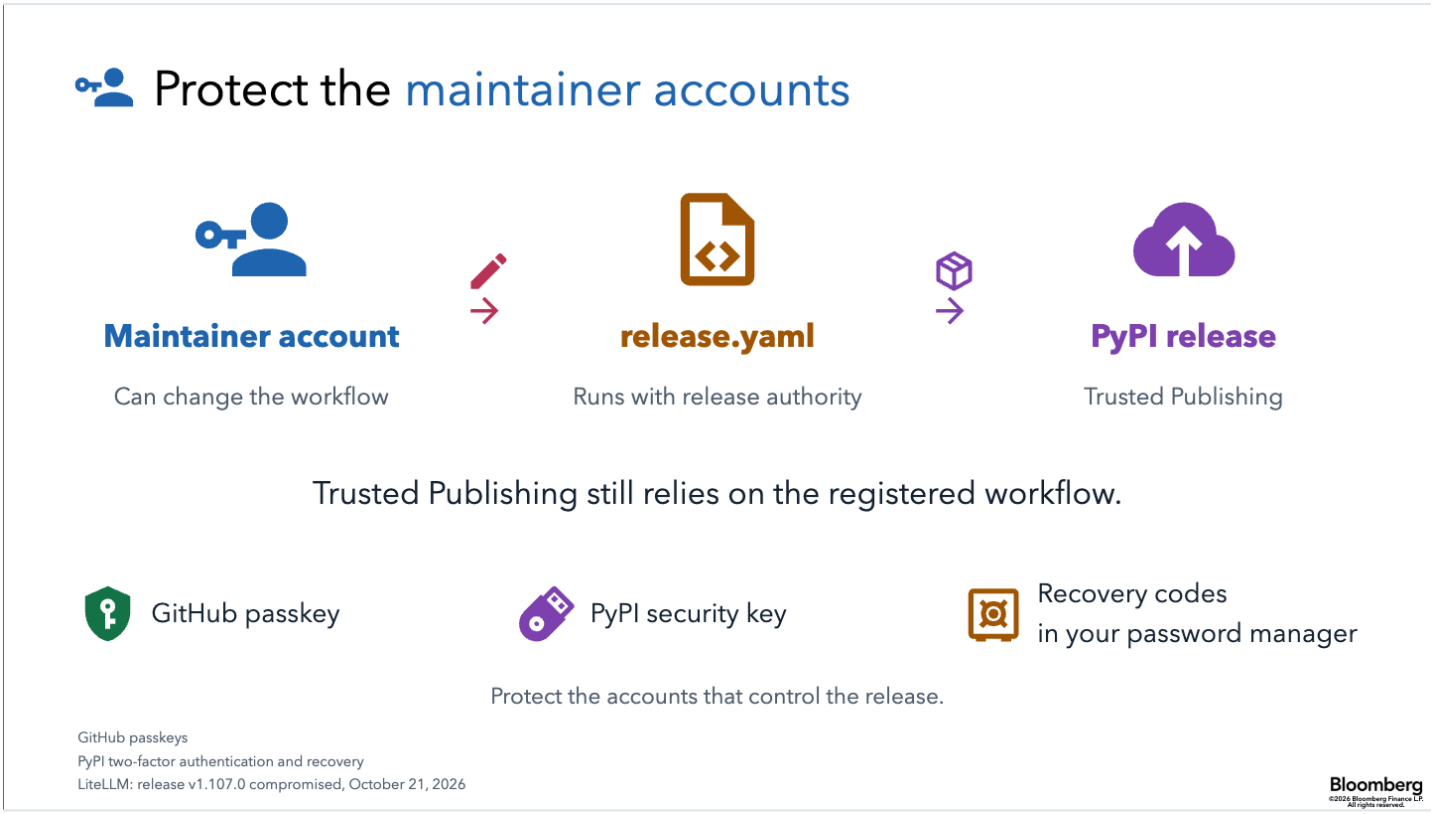
GitHub signs the job's identity through OpenID Connect; PyPI matches it against the repository, workflow, and environment we registered.

[Click: The token comes back; the clock starts.]

It issues an upload credential valid for at most fifteen minutes.

[Click: Upload. Click: Accepted.]

A compromised job can still misuse that token inside its fifteen minutes, which is why the release environment has required reviewers. Naming an environment in YAML isn't configuring approval. Revoke the old tokens.



[14:53–15:08]

Who can change that trusted workflow? Our GitHub accounts. Take over one with write access and Trusted Publishing will sign whatever you publish. That's how LiteLLM fell again on Wednesday: stolen maintainer tokens, branch protection overridden, and Trusted Publishing signed it.

[Click: reveal account protection and recovery.]

Passkey on GitHub, security key on PyPI, recovery codes in the password manager.

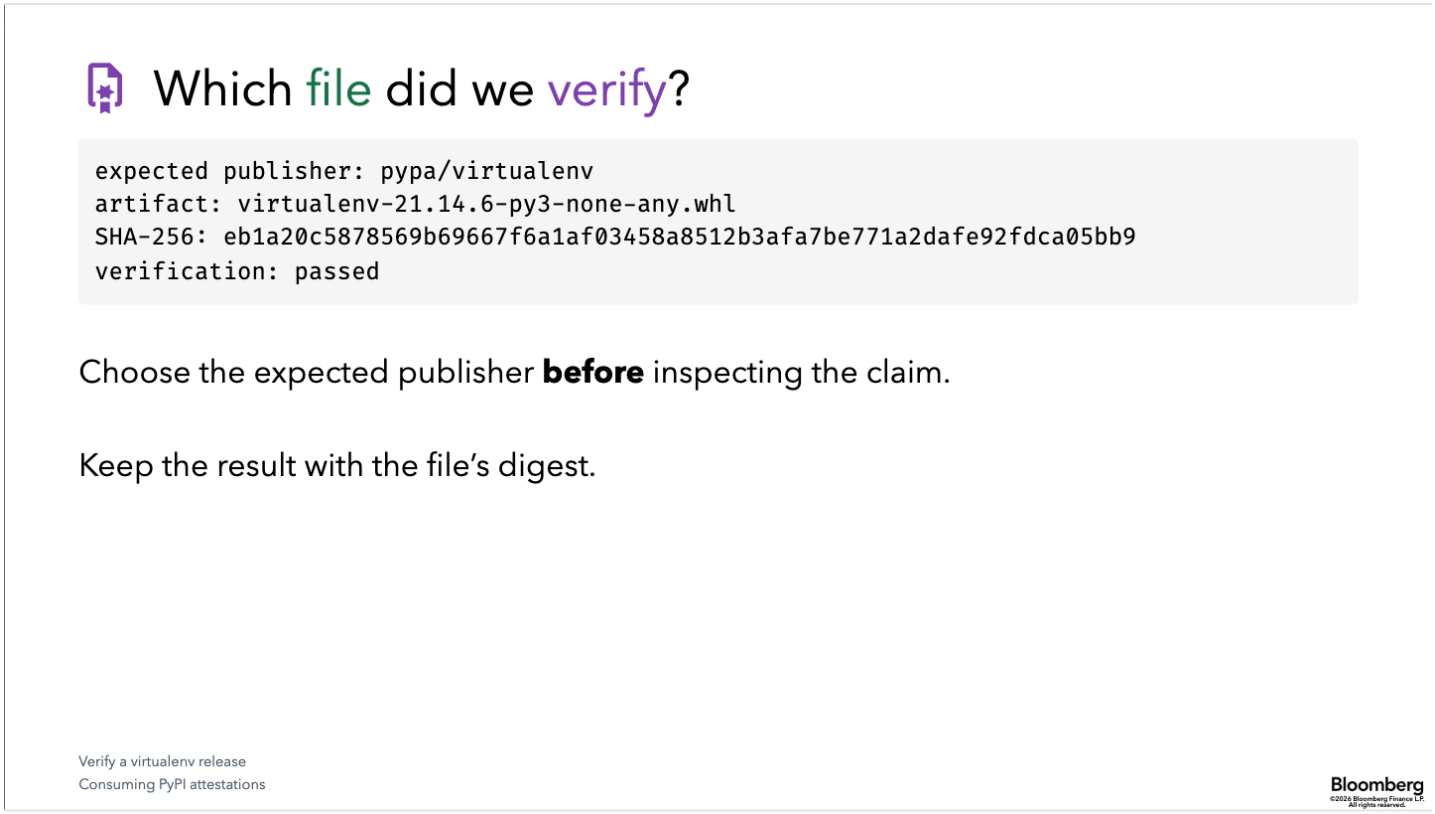


[15:08–15:44]

A hash says these are the bytes I reviewed. The attestation says who built them. I tell the verifier I expect pypa/virtualenv's release workflow; it checks the publisher identity and the file's digest against the statement PyPI stored with the upload.

[Point to the successful result. Follow the changed file to its rejection.]

I change one byte. The signature is fine. It's not for this file. Neither uv nor pip does this during install; it's a separate step, the commands are below, and the important part is choosing the expected publisher before you look at what the file claims. Accept whatever identity the file brings and you've skipped the check.

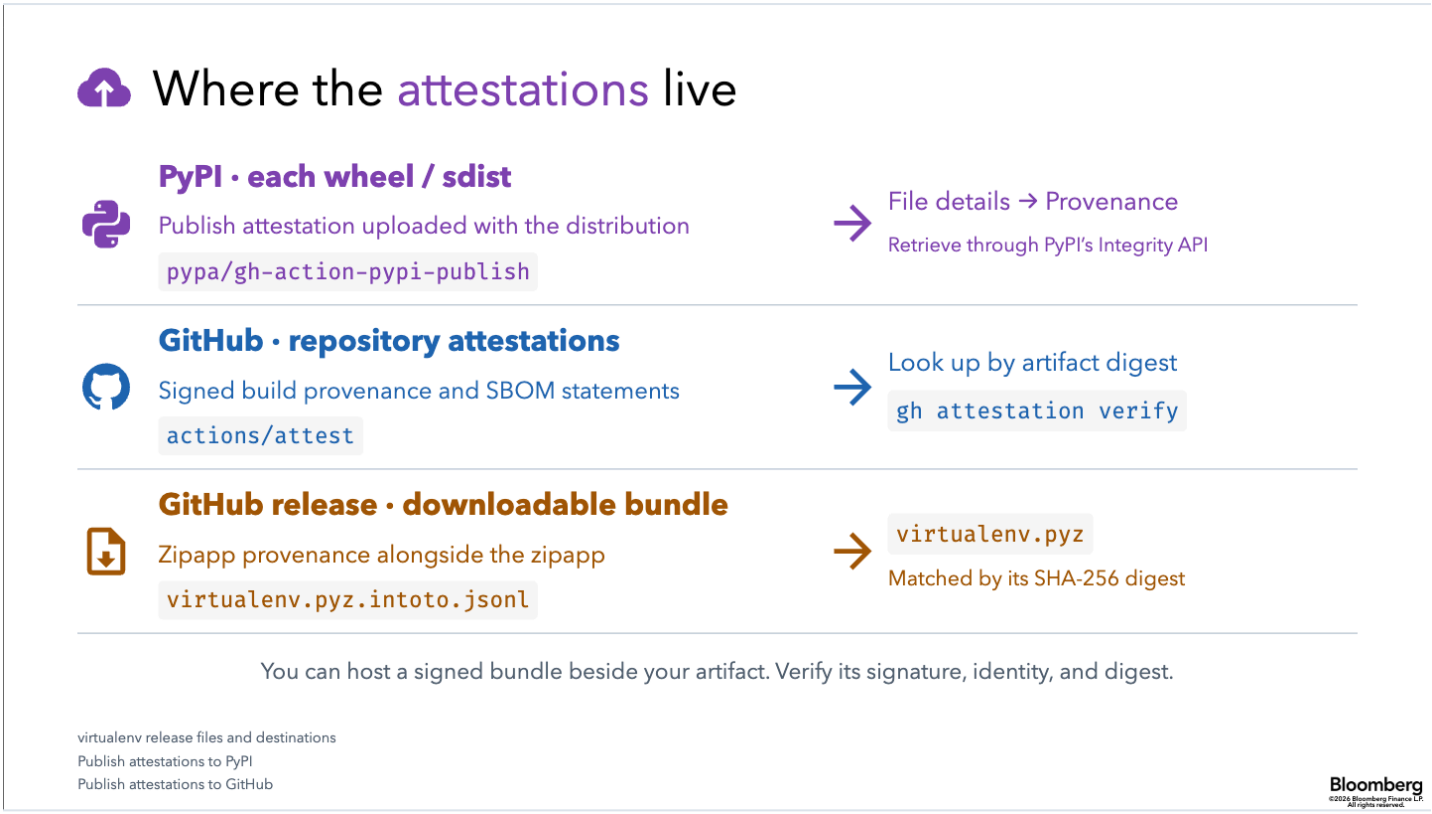


[15:44–16:03]

That result covers one file; another wheel needs its own run.

[Click: Reveal independent publisher selection and retained evidence.]

Name the publisher before you read what the file claims, and keep the result with the digest so later you know which bytes you checked. A valid signature says who built it, not that their dependencies were clean.

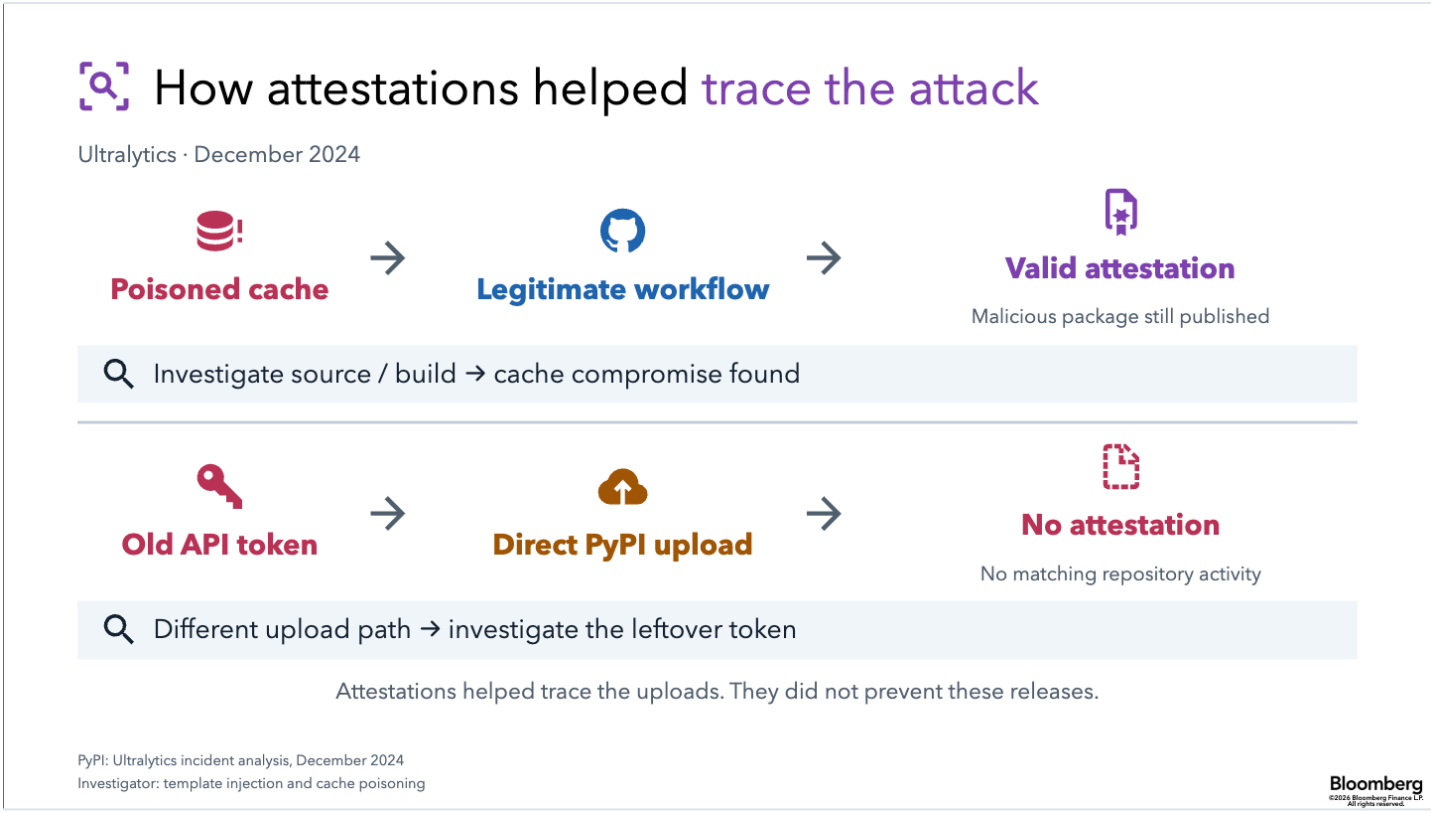


[16:03–16:25]

Where do these statements live? PyPI keeps the publish attestation with each uploaded file, in the provenance details and the API.

[Click: reveal GitHub storage and the downloadable bundle.]

Our build and SBOM attestations live on GitHub, and a bundle sits next to virtualenv.pyz on the release, naming the file by hash. Wherever you got the file, verify the identity and the digest.

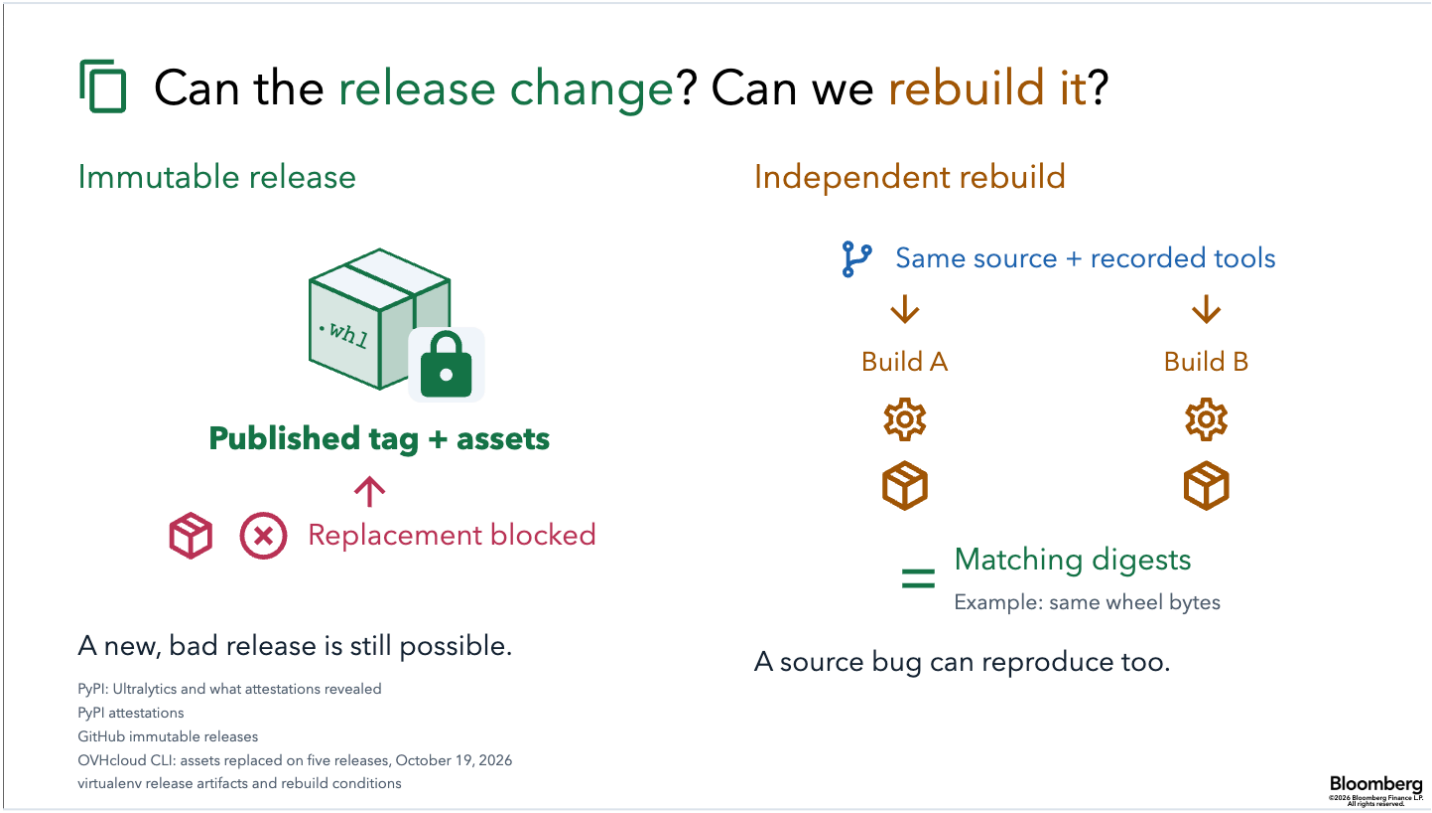


[16:25–17:00]

Does any of this stop an attack? Ultralytics, December 2024: the first malicious releases had valid attestations, because they came from the real workflow with a poisoned build cache behind it, poisoned through the same template injection zizmor flags.

[Click: follow the second upload path.]

The later uploads had no attestation and no matching repository activity. An old API token nobody had revoked. The missing attestation wasn't an alarm; nothing stopped those uploads. It was a receipt nobody had signed, and it told the investigators which of two doors to look at. Evidence, not a gate. The gate is still the permissions.

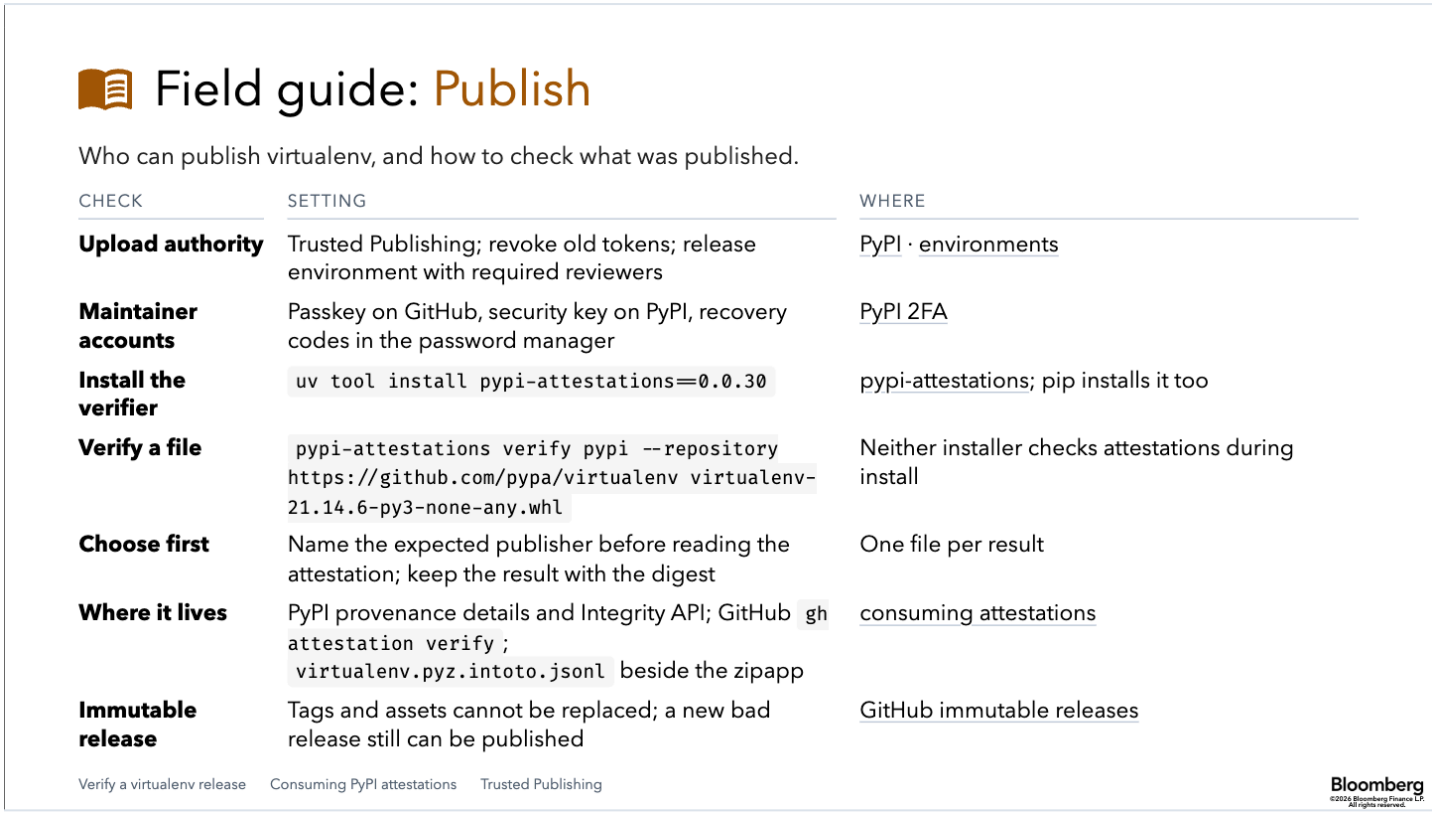


[17:00–17:19]

Could someone change one of our releases, and could we rebuild one? Two checks. An immutable release stops anyone replacing a published tag or asset; a new bad release would have to be published, and that shows up. OVHcloud's CLI releases weren't immutable, and on Monday someone swapped the files on five of them, checksums included.

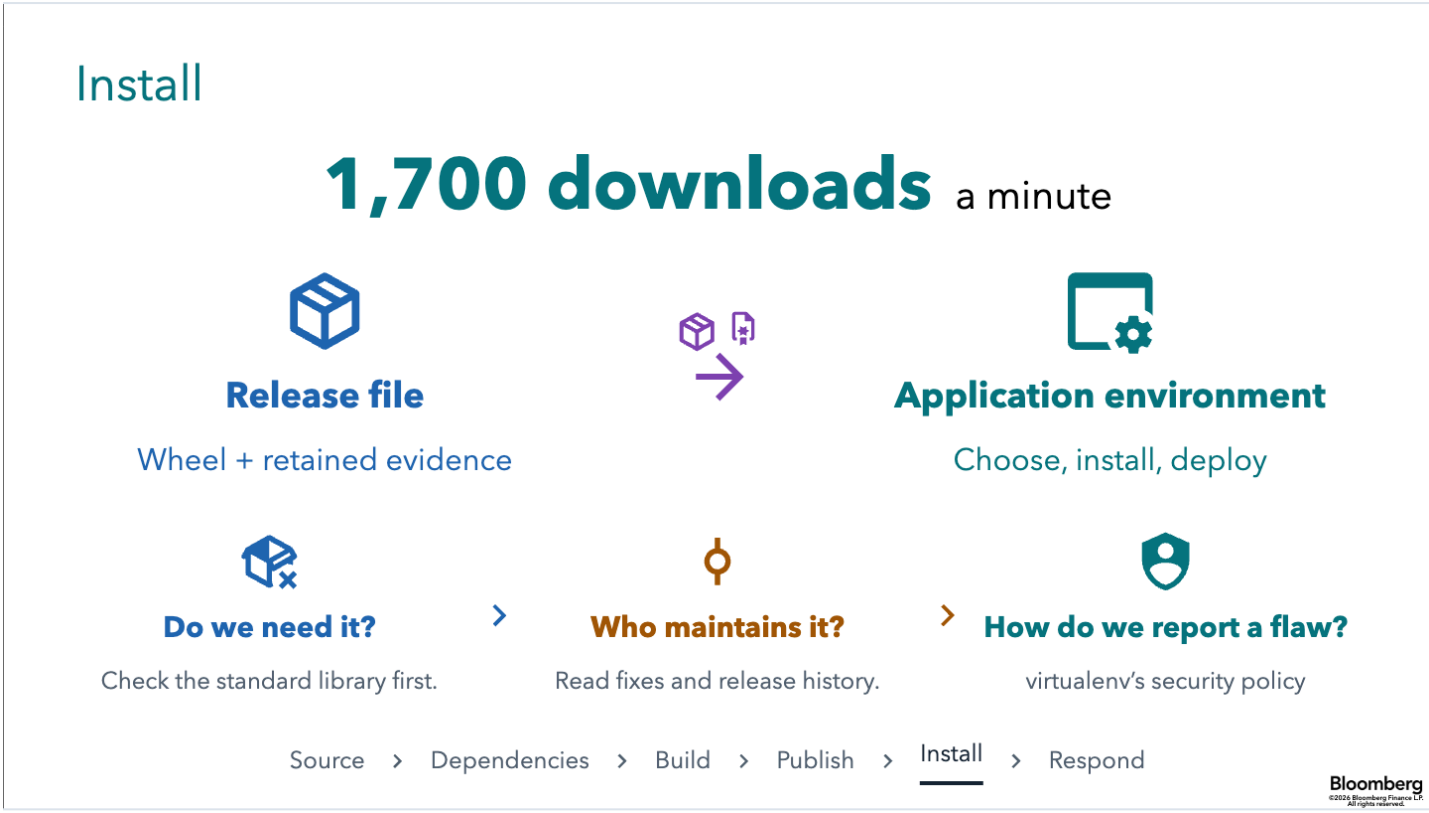
[Let the attempted replacement stop at the lock. Click: follow the same recorded inputs through two builds and compare their digests.]

And a rebuild from recorded inputs should produce the same bytes. Ours didn't; the reason comes with the inventory.



[17:19–17:32]

The publish field guide: upload authority, maintainer accounts, the verifier, choose the publisher first, where the statements live, immutable releases. One line: name the publisher you expect before you read what the file claims.

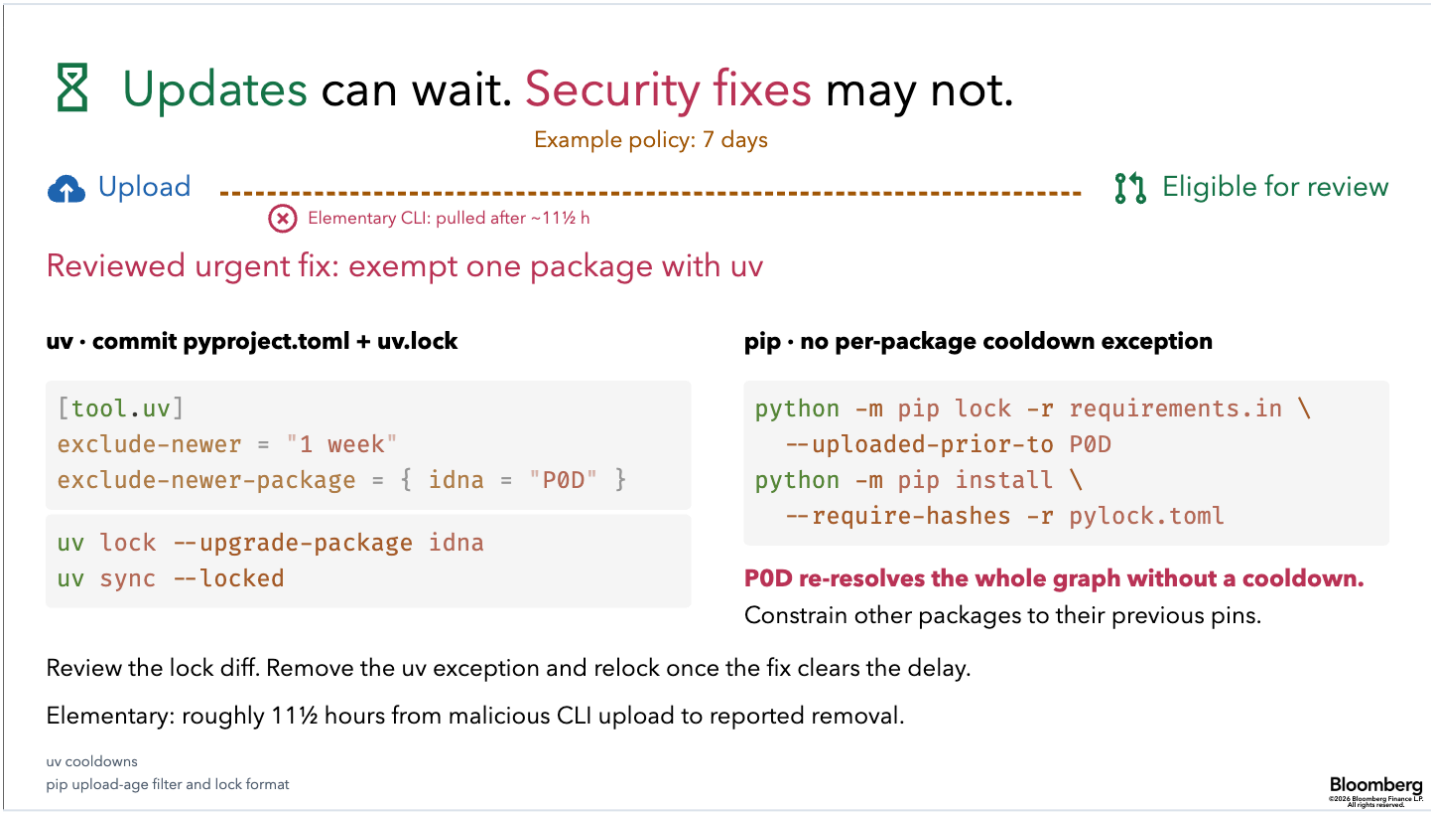


[17:32–17:56]

Now the other side of the download: the 1,700 a minute, which is you.

[Click: Follow the review questions.]

Before adding a dependency: do we need it, or does the standard library already do it? Who maintains it, and how do they handle fixes? Where do we report a flaw, and does anyone answer? Then choose the release.



[17:56–18:36]

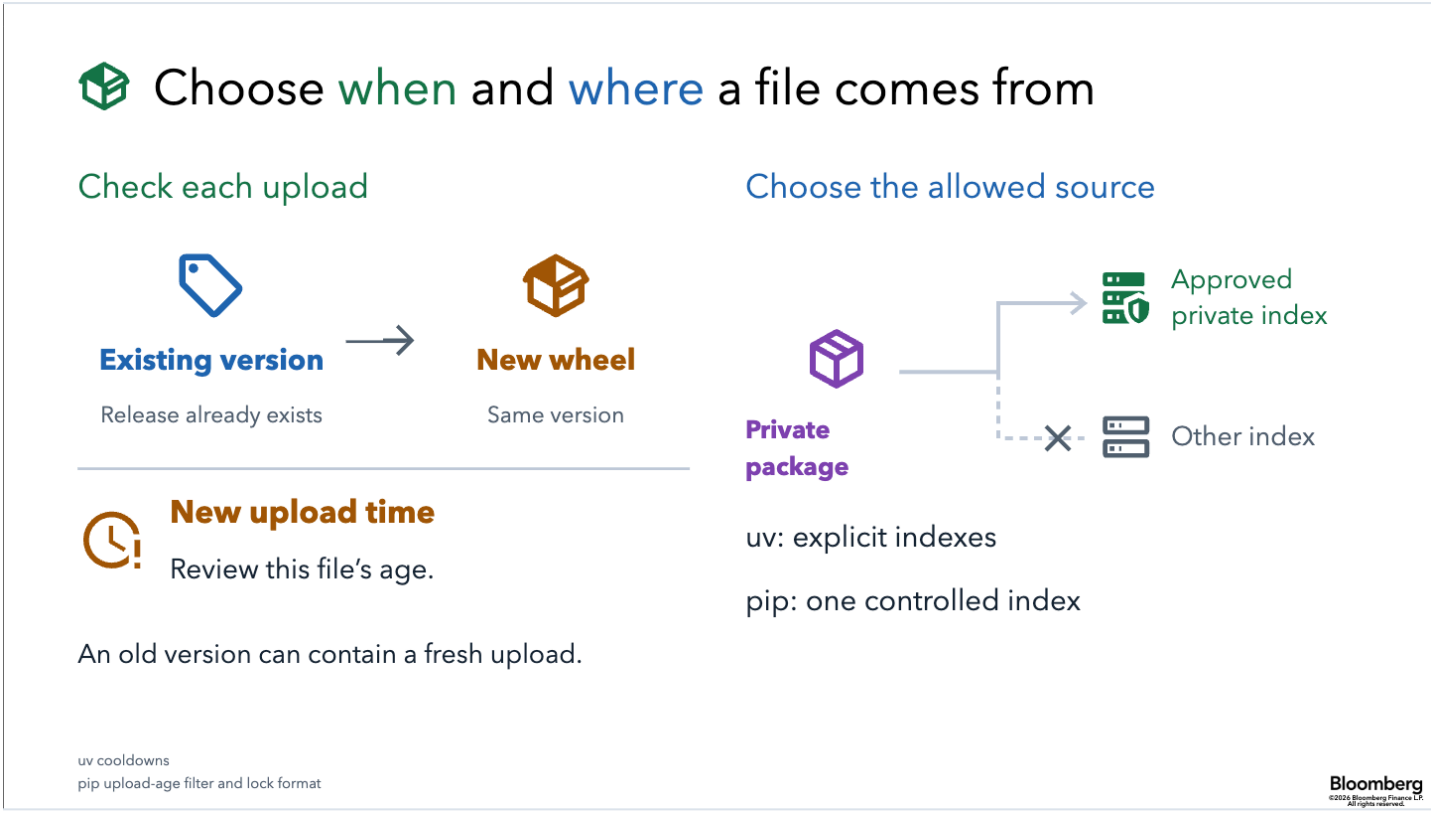
For applications, lock exact versions and read the diff when you move them. Updates can wait for that review; a lockfile is the thing that makes the review possible.

[Click: Reveal both installer workflows.]

A cooldown keeps brand-new uploads out of the lock. Seven days here; PyPI's report suggests three. Elementary's malicious CLI was up for eleven and a half hours, so three would have kept it out.

[Click: Show the scoped exception.]

A security fix can't wait seven days. Exempt that one package, upgrade it, and the rest keep their cooldown; plain locked sync still passes. pip has no per-package exception, and the field guide says what to do instead.

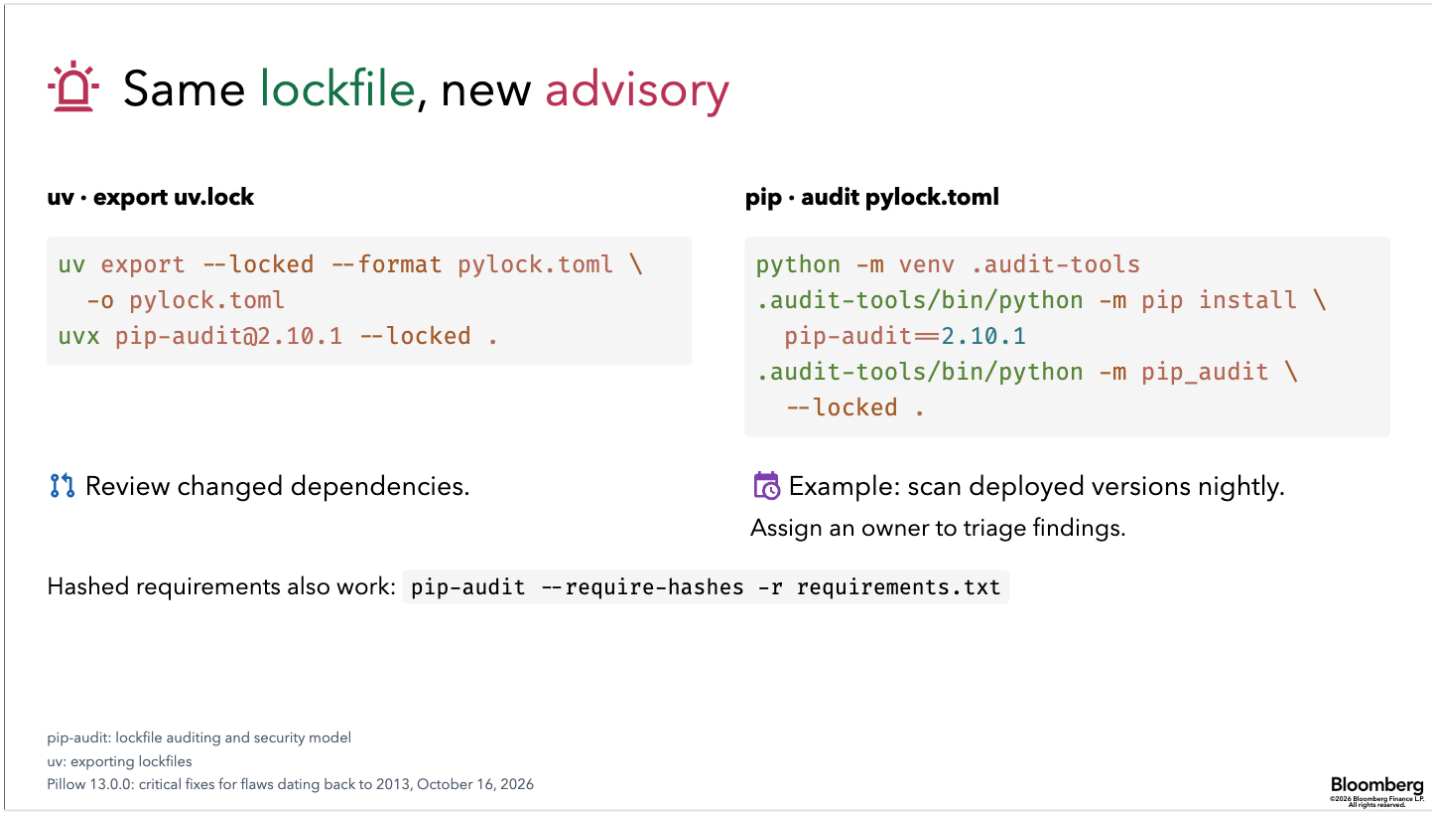


[18:36–18:58]

One more rule for cooldowns: a release can be old and still get a new file, so virtualenv checks each wheel's upload time, not the version's.

[Point to the new wheel arriving under the existing version. Click: follow the private package to its approved source.]

And private packages need an explicit source. extra-index-url adds a second place to look; it doesn't say which wins. Route the package in uv, or one controlled index with pip.

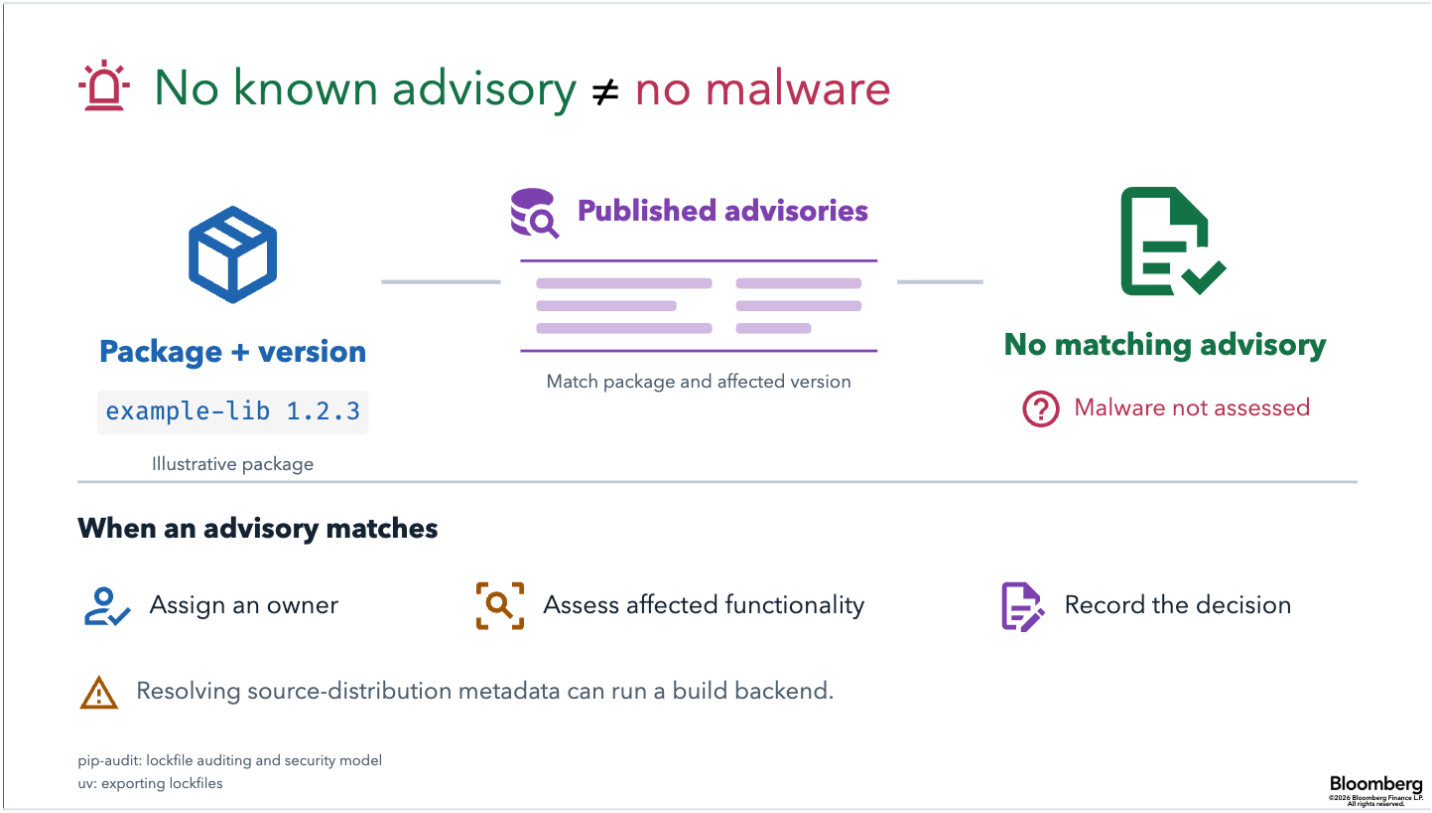


[18:58–19:16]

Even when the files don't change, what we know about them does. Pillow users saw that last week, when one release fixed critical flaws that had shipped for over a decade.

[Click: Reveal when the scan runs.]

pip-audit on the lock when dependencies change, and nightly against what's deployed, with a named owner; nightly without an owner means ignored nightly. Point it at the lockfile, not the environment you installed pip-audit into.

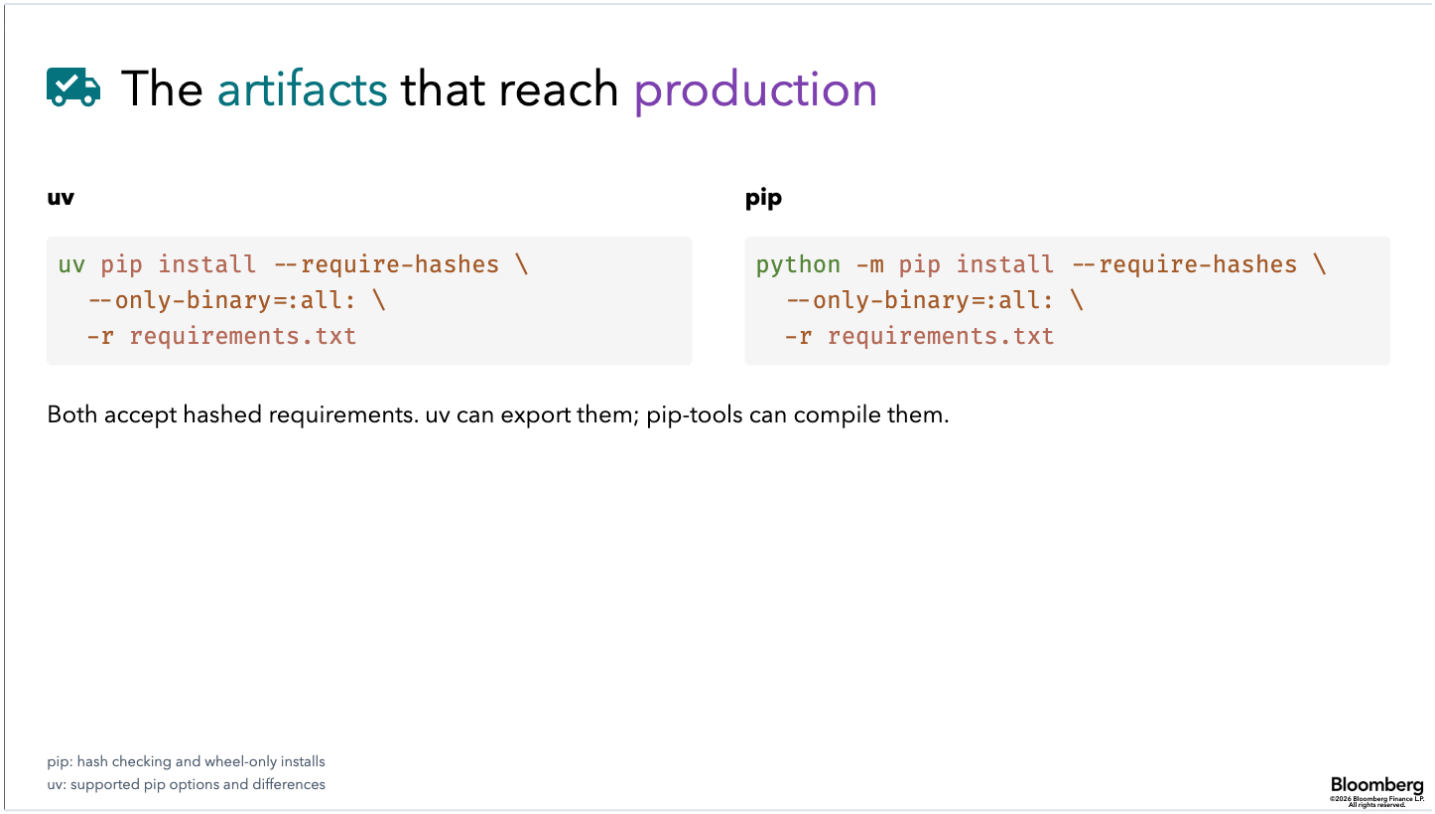


[19:16–19:40]

And read the result for what it is. No matching advisory means nobody has filed one. LiteLLM had none for two and a half hours.

[Follow the lookup and its report. Click: reveal how to handle a matching advisory.]

When something matches, check whether you use the affected code, and write down how you decided. And resolving source-distribution metadata can run a build backend, so scanning an untrusted package isn't a safe way to try it.

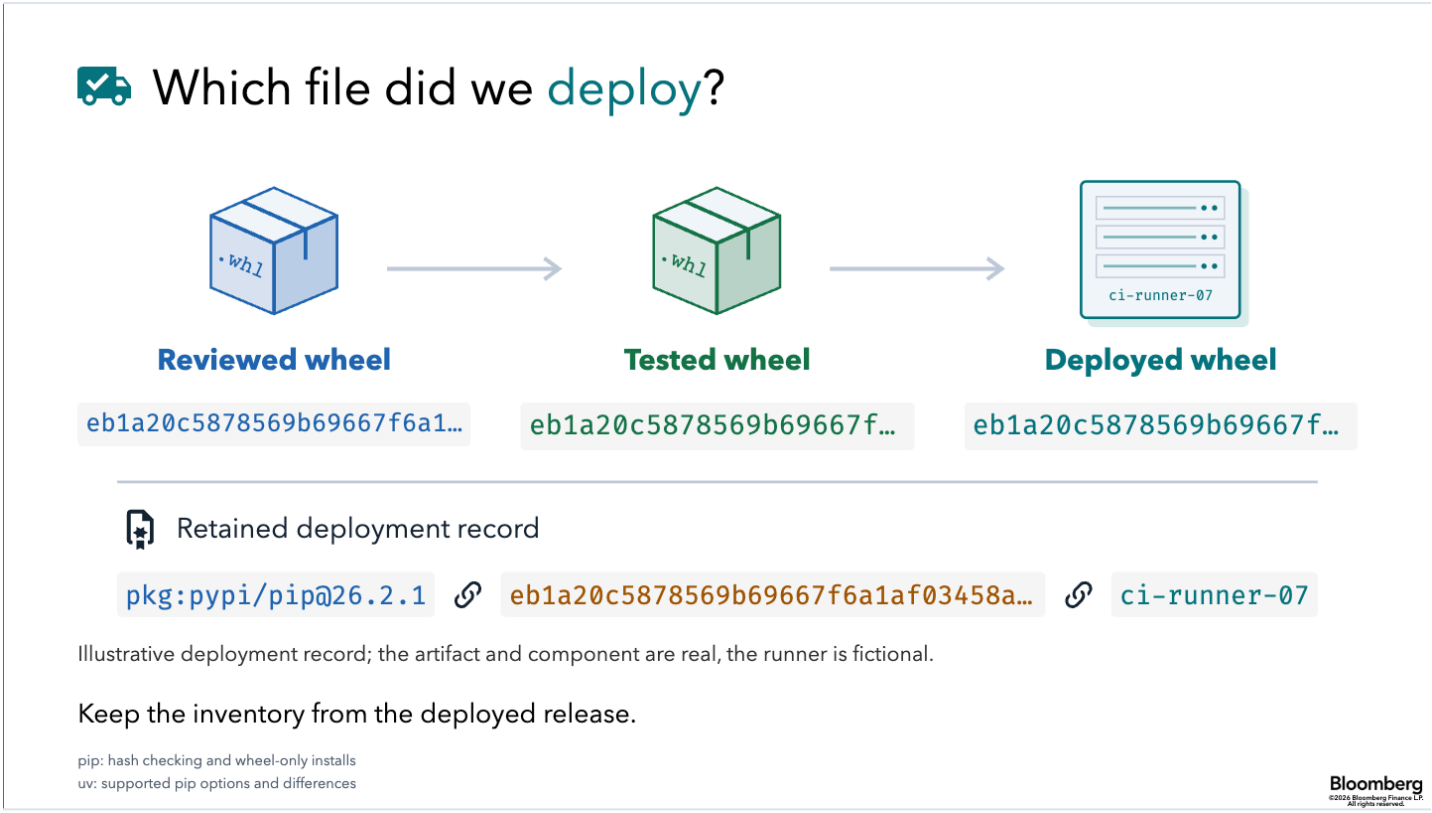


[19:40–19:56]

Production: require hashes and refuse source distributions, so an install can't quietly become a build.

[Click: reveal where the shared requirements file comes from.]

Both read the same hashed requirements; uv exports them, pip-tools compiles them. No compatible wheel, no install. Build and review one in a separate job.

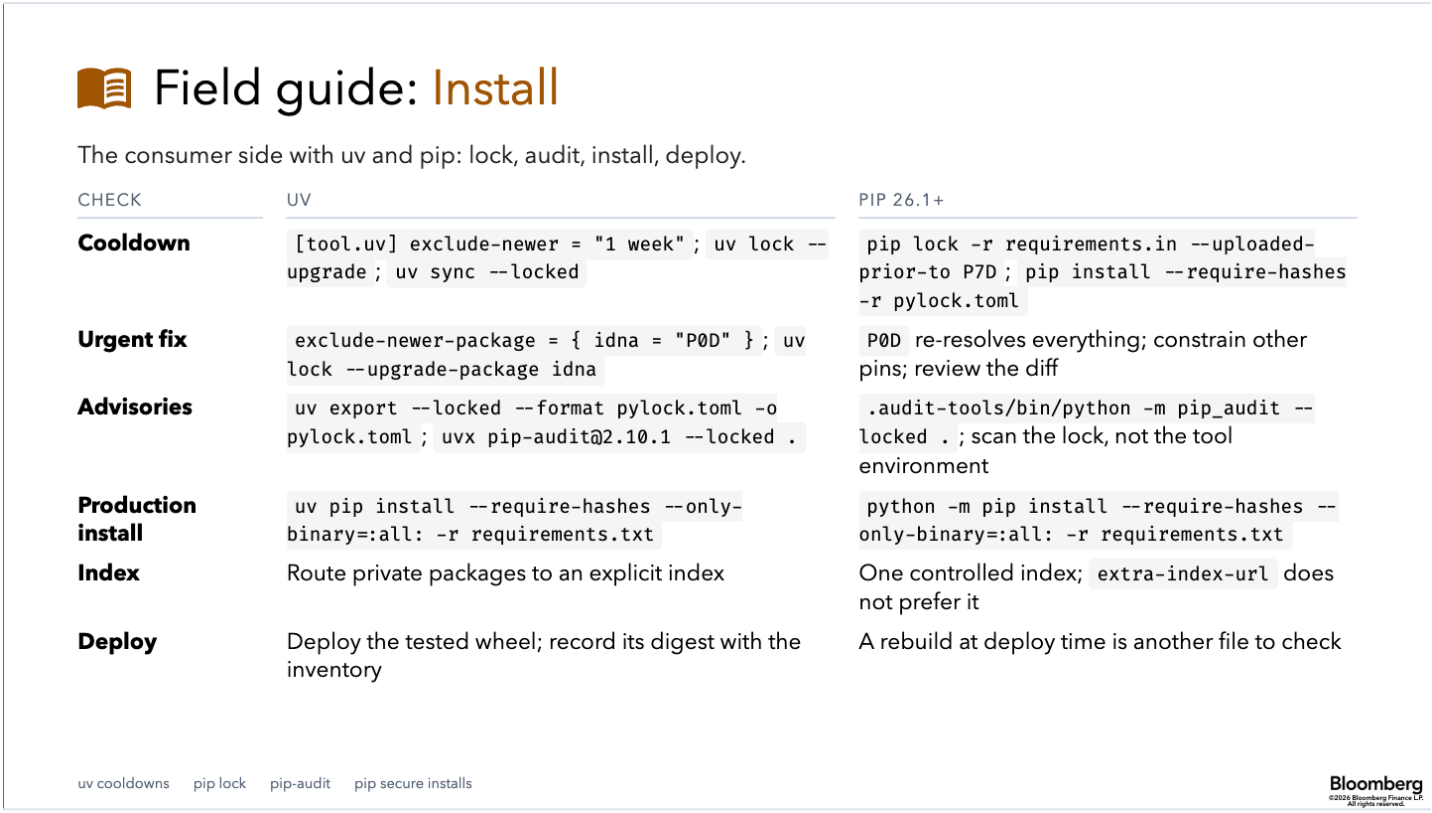


[19:56–20:11]

Deploy the wheel you tested, not a rebuild; a rebuild is another file to check.

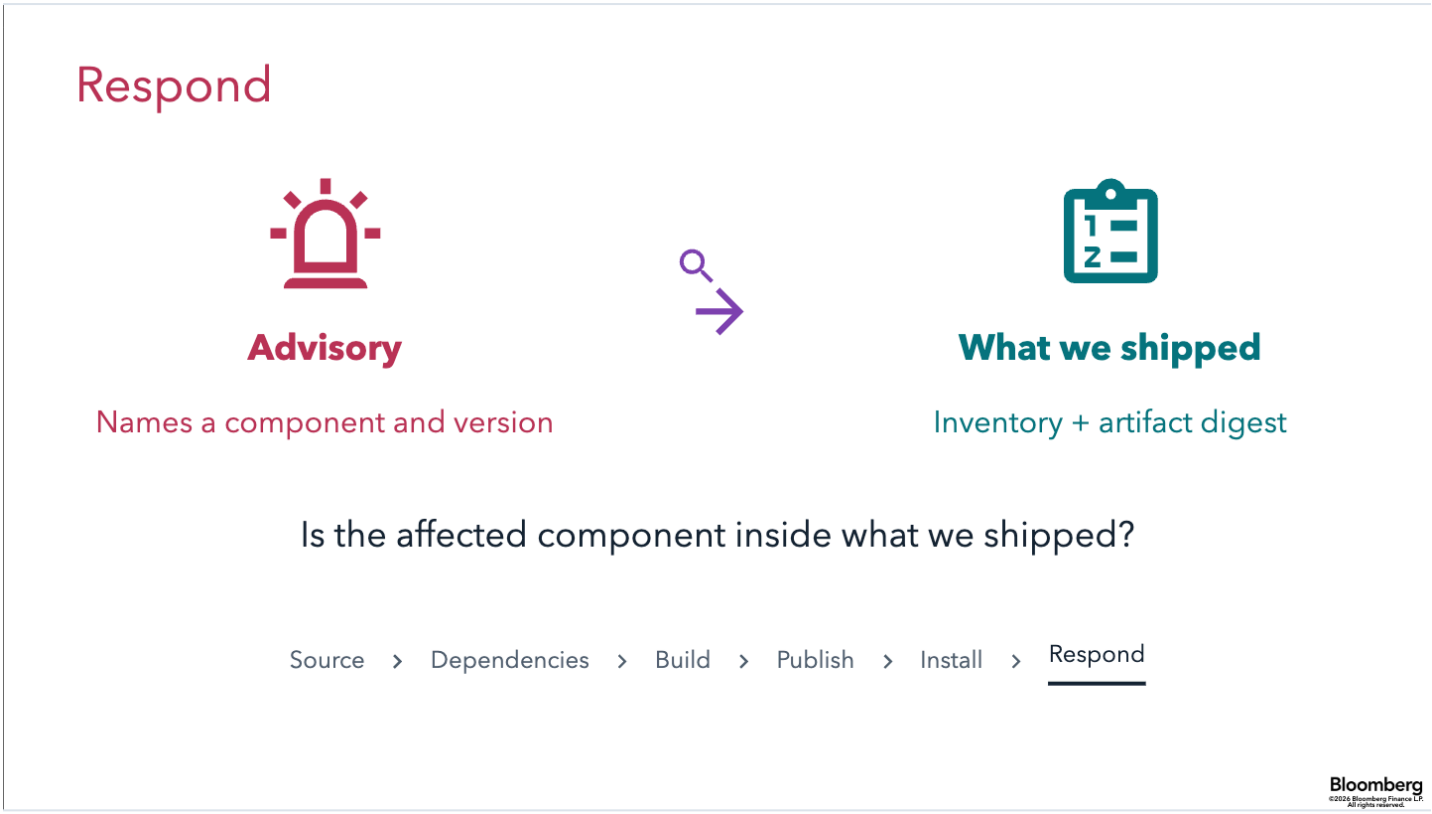
[Follow the wheel through review, test, and deployment; its digest stays attached. Click: Reveal the linked deployment record.]

Record its digest, and keep inventory, digest, and deployment record together.



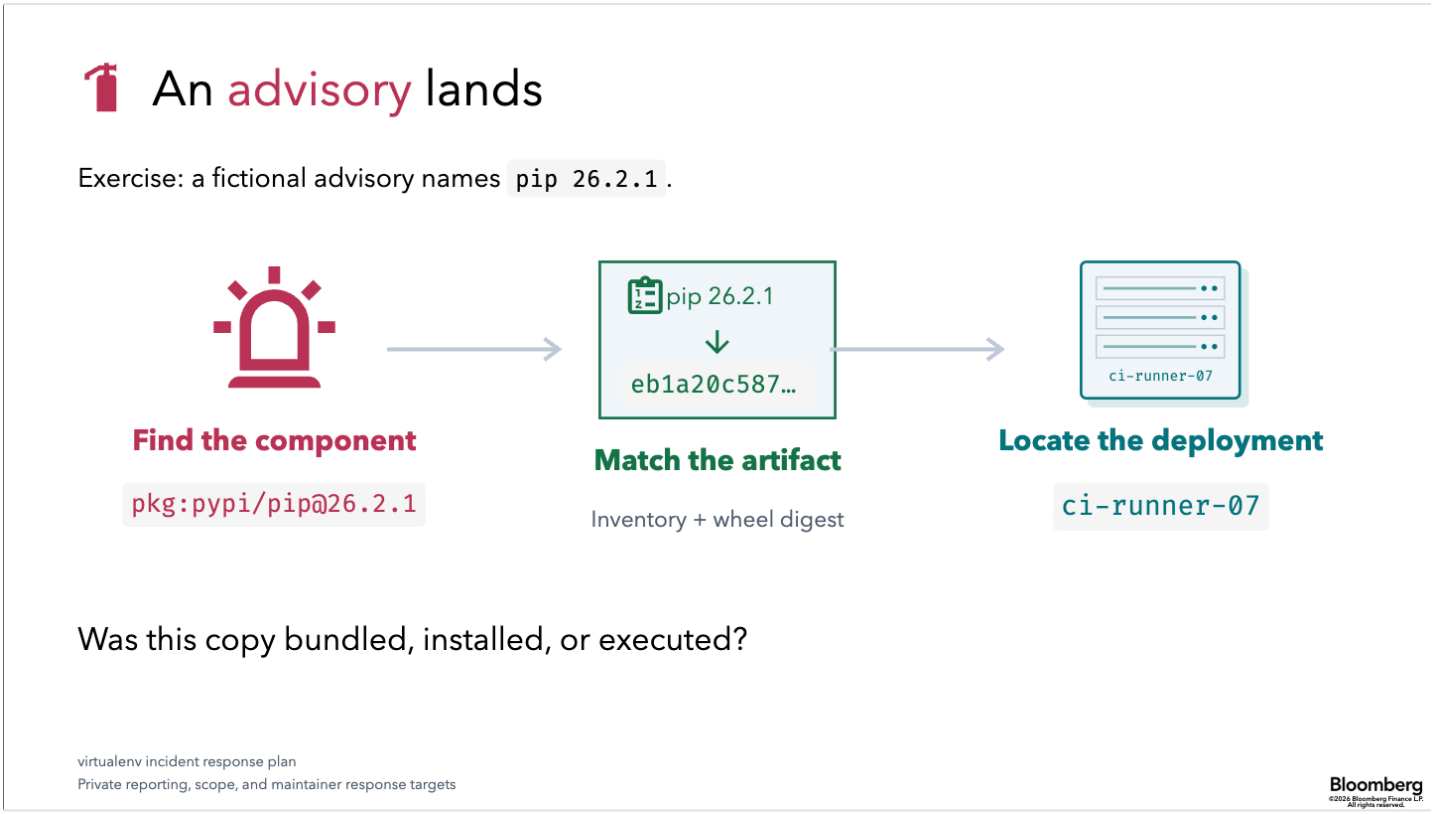
[20:11–20:25]

The install field guide: cooldown and the urgent exception, audit the lock, hashes and wheels only in production, one explicit index, deploy what you tested. One line: no known advisory is not no malware. Someone has to own what the scanner finds.



[20:25–20:30]

Then the day an advisory lands.



[20:30–21:05]

Now the inventory earns its keep. An advisory lands for pip 26.2.1. Fictional advisory, fictional runner, real version. You don't depend on pip. You depend on virtualenv. Are you affected?

[Click: Follow the component identity to the artifact's inventory.]

[Click: Follow the artifact digest to the runner.]

The inventory names the component; the digest names the deployment; the deployment record says which runner got which wheel. That's the chain. But the first link is the one people skip: is pip even in there? pip list says you're fine. Hands up if you believe it.

[Give it three seconds.]



[21:05–21:40]

No. These are the declared dependencies, Requires-Dist in the wheel metadata, what the installer fetches alongside virtualenv. distlib, filelock, packaging, platformdirs. No pip.

[Click: swap to the bundled wheels. Pause for recognition.]

Same wheel, opened. There are pip and setuptools, carried inside virtualenv as wheel files, and those wheels carry copies of other libraries: requests, urllib3, rich. That's vendoring.

I work in packaging. Occasionally, we take that too literally.

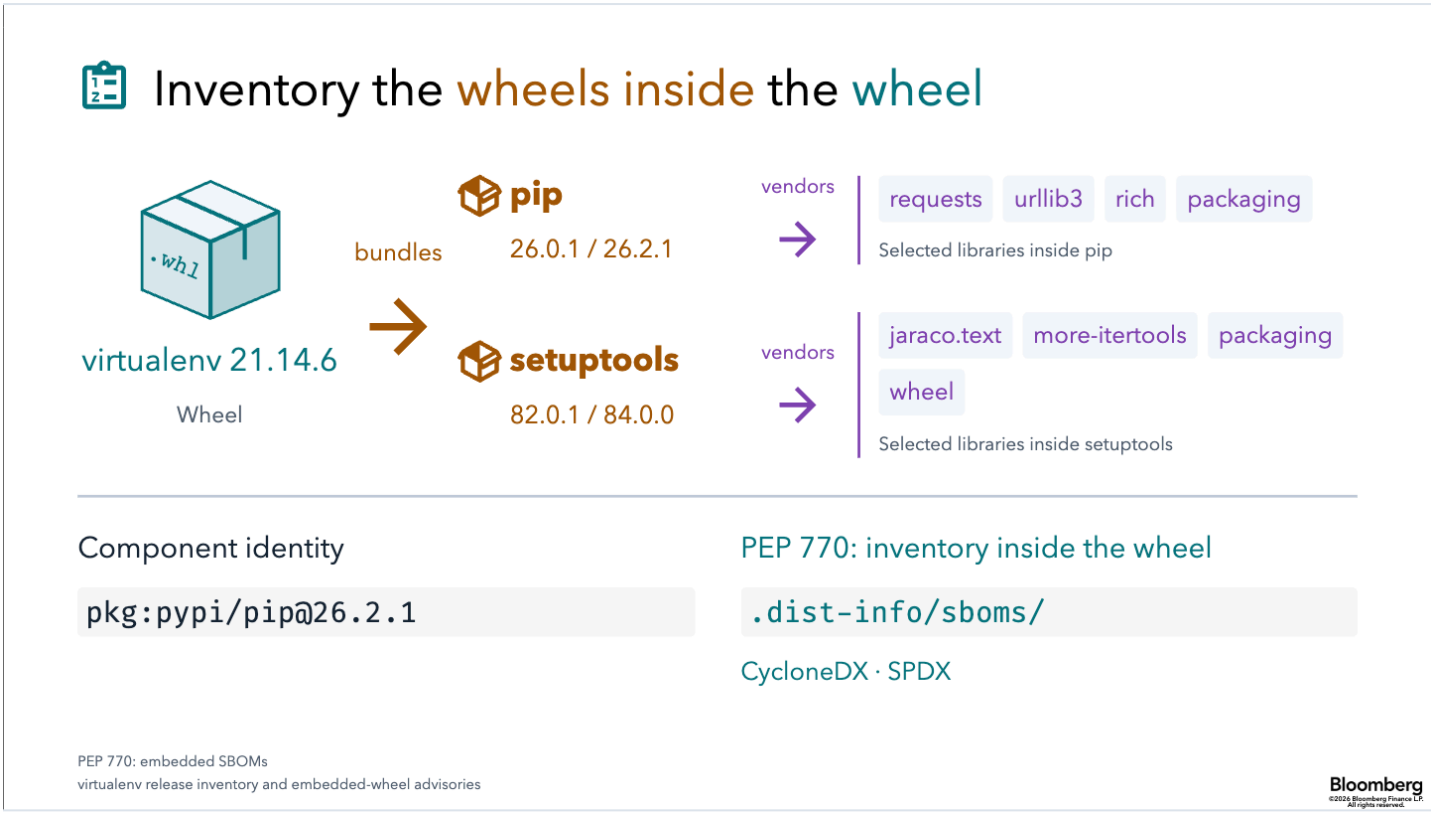
[Give the line a beat.]

Whichever installer put virtualenv here, an environment listing misses everything carried as data. The advisory scanner reading that listing misses it too.



[21:40–21:57]

The SBOM, the software bill of materials, has them. pip, its vendored requests and urllib3, setuptools and its copies, each by name and version, as a component identity a scanner can match. Keep this with the wheel and someone investigating an incident can see what was in the release they received, not the one you're building today.

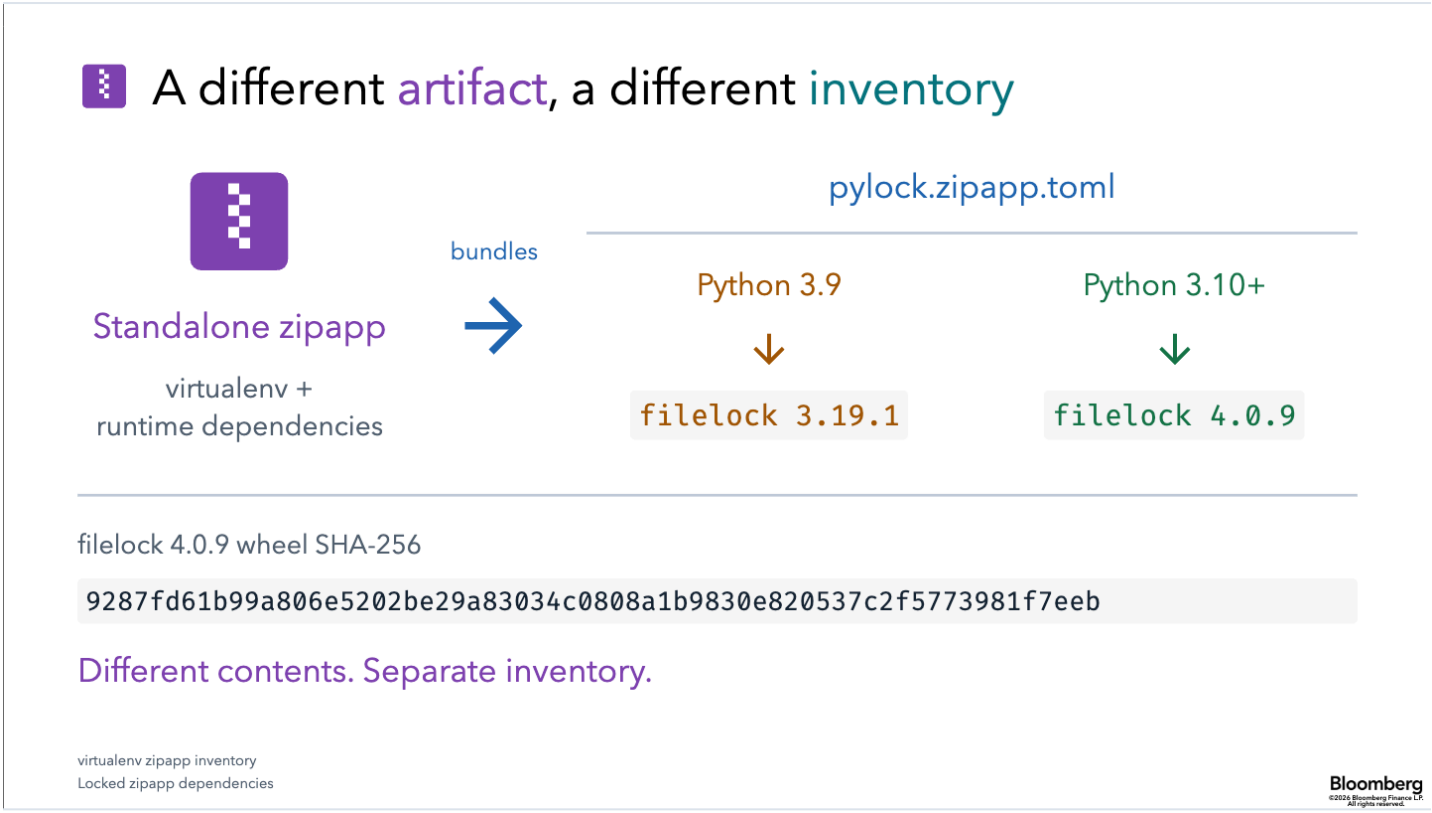


[21:57–22:17]

Follow the nesting: virtualenv carries pip and setuptools, and those carry more. All of it belongs in the inventory.

[Click: show a component identity and the inventory's location.]

This package URL is pip from PyPI at 26.2.1, the identity a scanner matches on. PEP 770 reserves a path inside the wheel for it, so the inventory travels with the file.

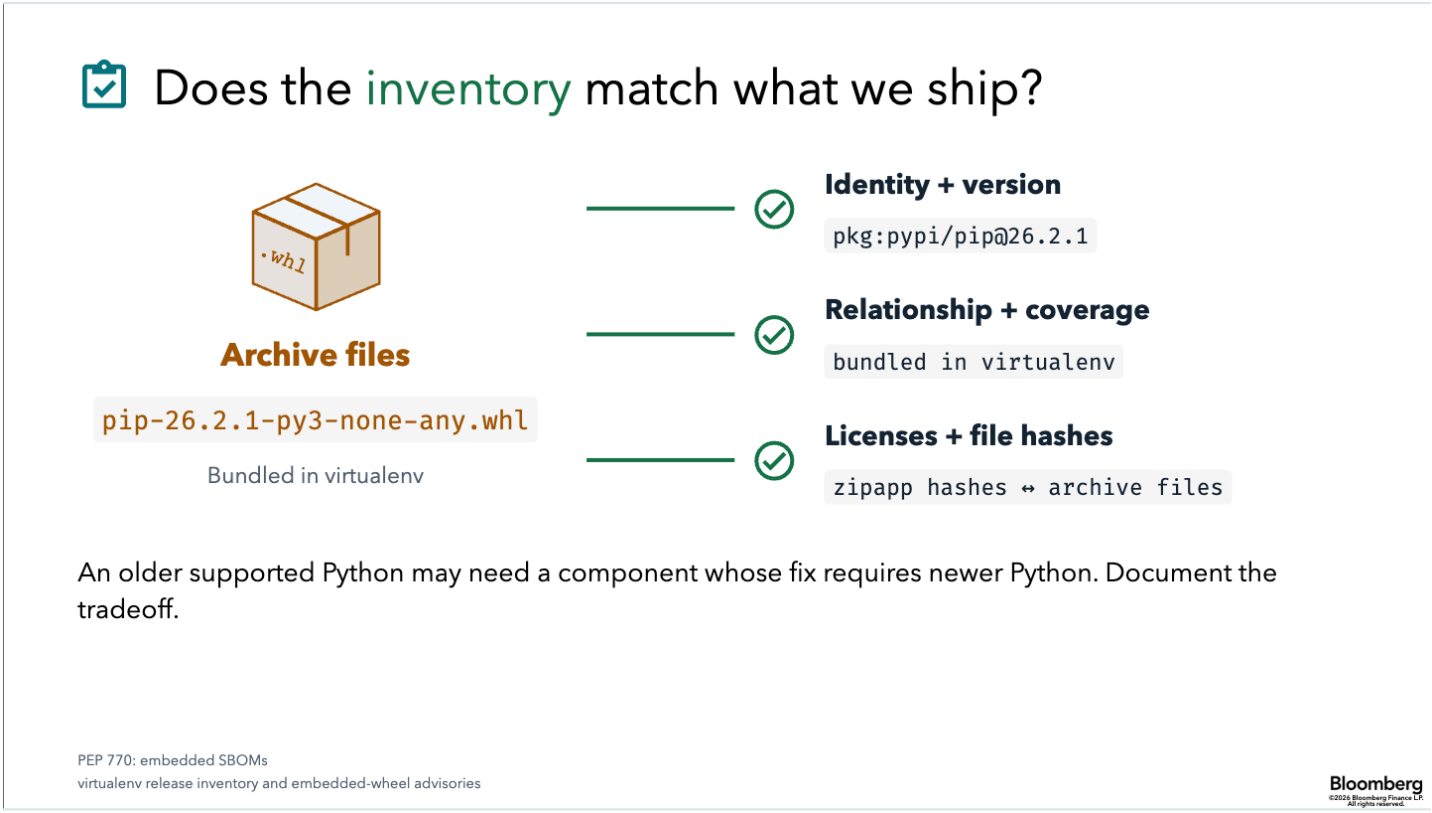


[22:17–22:30]

The zipapp, our standalone archive, bundles runtime dependencies too. Follow filelock: one version for Python 3.9, another for 3.10 and later.

[Click: reveal the wheel hash and the separate-inventory conclusion.]

Different code, different inventory. One per archive.



[22:30–22:54]

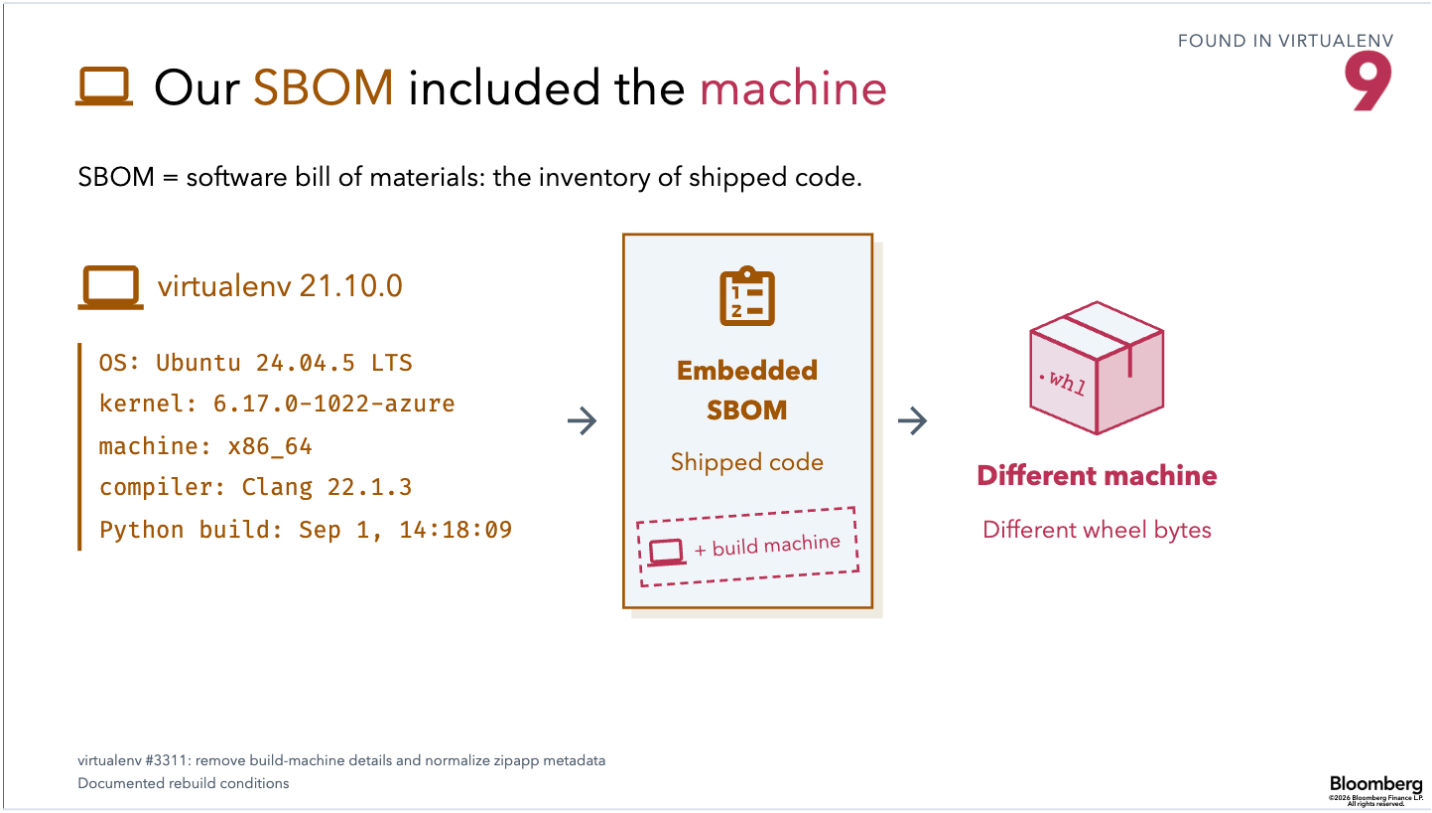
An inventory you don't check is a list you hope is right.

[Click: Follow each connection from archive files to the corresponding inventory check.]

We check the inventory's structure, relationships, and licenses on every build, and for the zipapp, every file and its hash against the archive. Sign an incomplete inventory and you get a signed incomplete inventory.

[Click: Reveal the older-Python tradeoff.]

One honest gap: a bundled tool's fix may need a newer Python than we support. Document it.



[22:54–23:32]

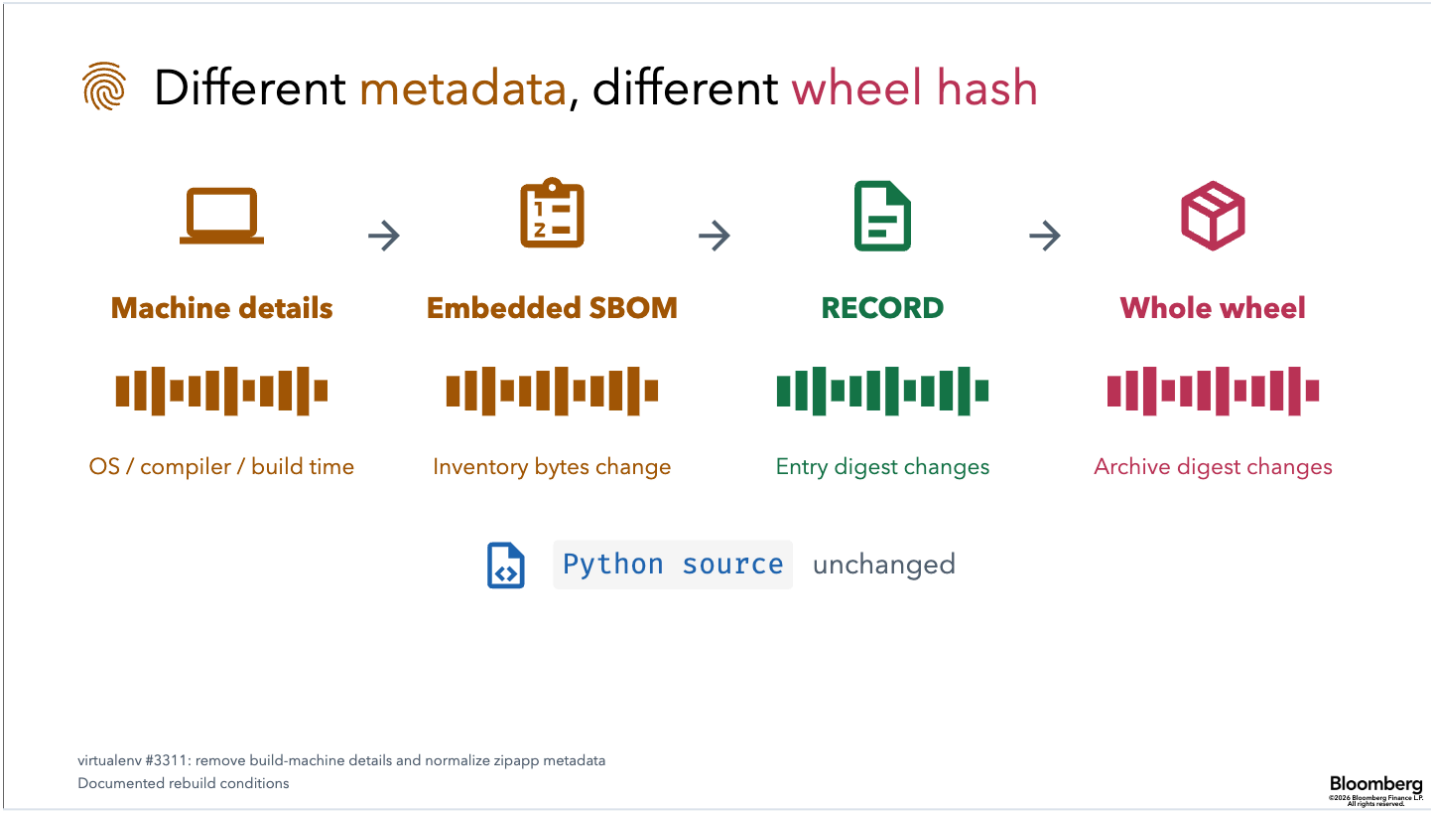
Remember the rebuild that didn't match? Then I checked our SBOM. It recorded the build machine: Ubuntu release, Azure kernel, architecture, compiler build. Number nine, and the last.

[Click: Follow the machine details into the embedded inventory.]

Two machines could build the same source with the same tools and produce different wheels, because these fields differed. Our wheel came with a souvenir from the build machine.

[Pause.]

These fields are from 21.10.0. We deleted them, and fixed archive timestamps and permissions while we were in there, so that a rebuild from the same commit, Python, and tools can match.



[23:32–23:59]

[Follow the change from machine details to inventory, RECORD, and wheel.]

Why does that change the whole wheel? The inventory is a file inside it. Its bytes change, so its hash changes in RECORD, the file that lists every file in the wheel with its hash. RECORD changed, so the wheel's digest changed. Not one byte of Python moved. That's why a mismatch needs a diff before it needs a headline, and why the inventory now records what we shipped and nothing about where.

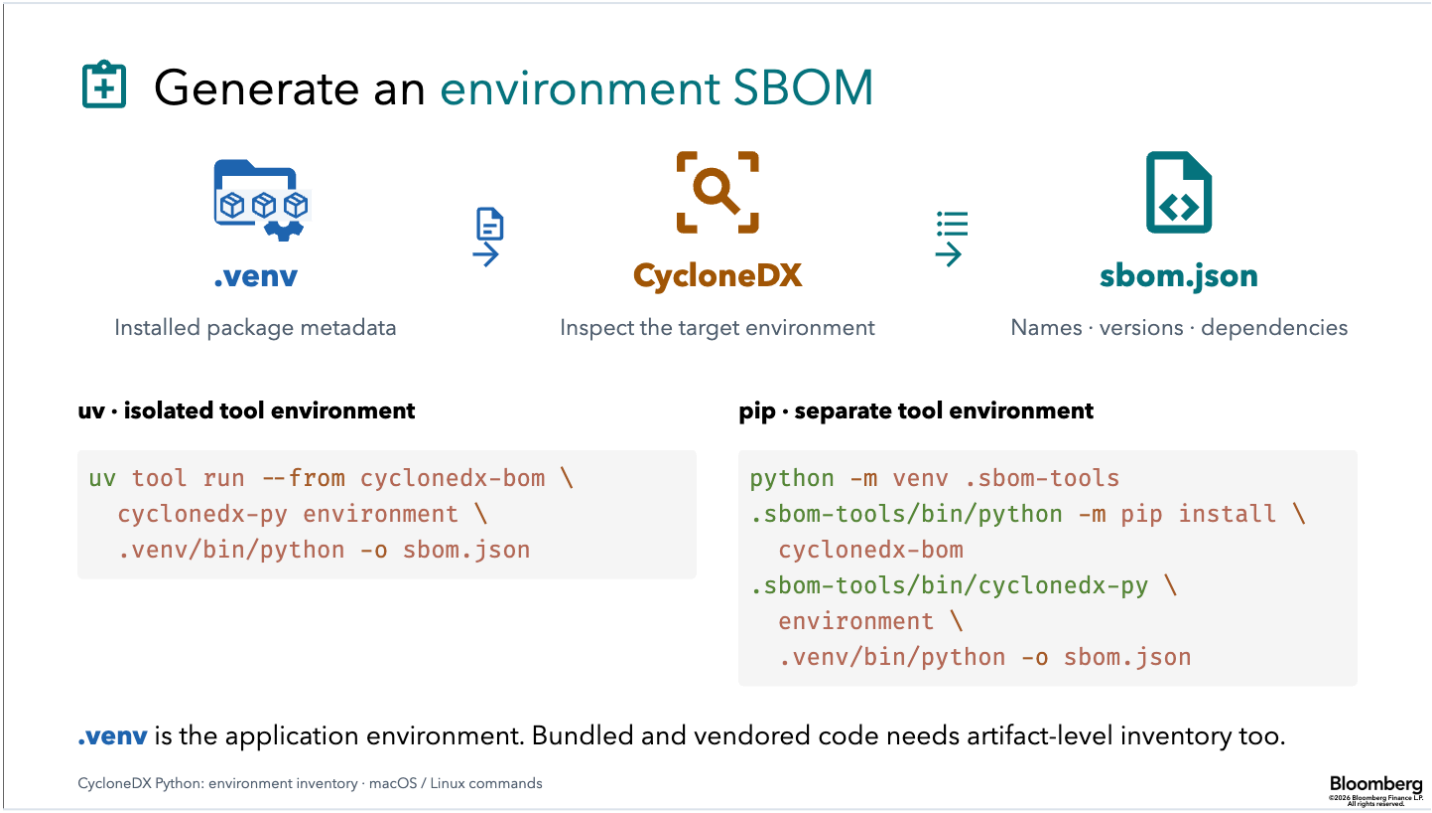


[23:59–24:19]

What does a rebuild need now? The source commit, the Python patch version, the build tools, and a timestamp through SOURCE_DATE_EPOCH. These fields are 21.14.6; the souvenir was 21.10.0, two releases, not two builds.

[Click: Separate the build record from the shipped-code inventory.]

Where the build ran goes in a separate provenance statement that names the wheel by digest.

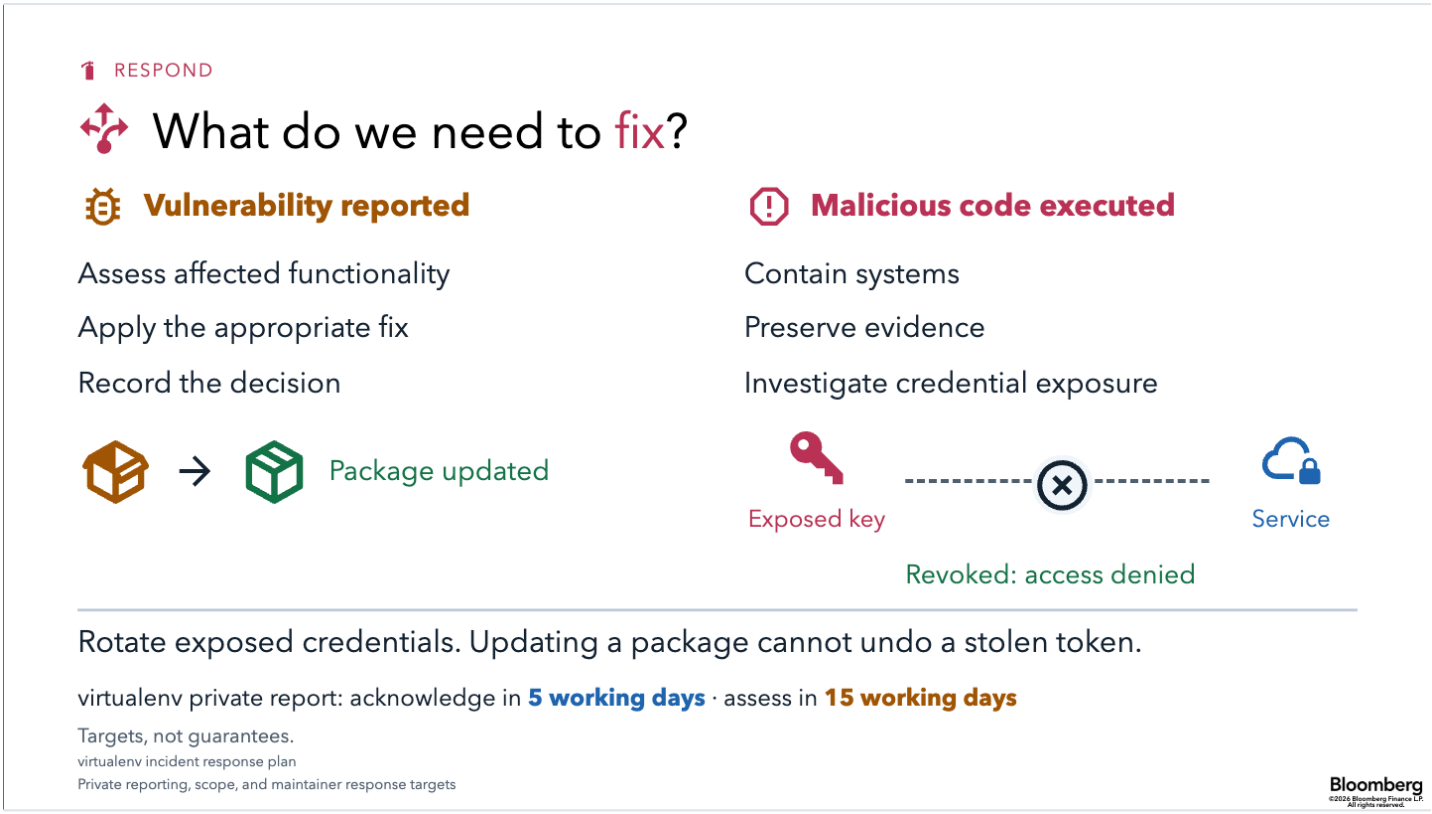


[24:19–24:39]

For your own application, this is the one-liner: CycloneDX reads the environment's metadata and writes the SBOM.

[Click: reveal both commands. Point to the same target interpreter in each.]

Run it from a separate tool environment so its dependencies stay out of your inventory. And remember the show of hands: it reads what's installed. The wheels inside the wheel come from the artifact's inventory.



[24:39–25:12]

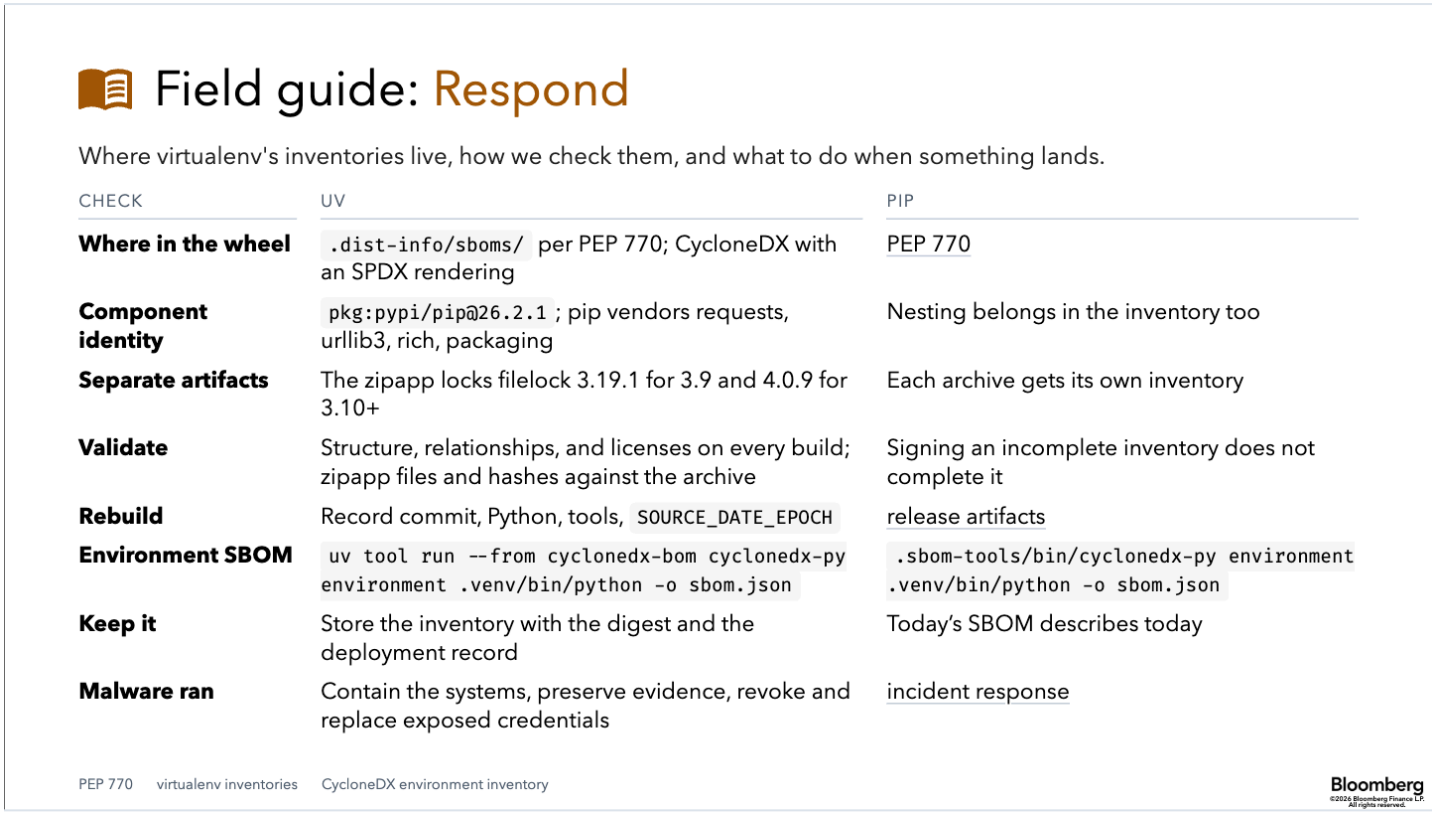
When something does land: for a vulnerability, assess whether we're affected and ship the fix. That alone doesn't tell you whether a credential was stolen.

[Click: Reveal the response to executed malware.]

If malicious code ran, contain the systems, preserve evidence, and investigate credential access. The package is fixed; the exposed key still works, and it will keep working after the all-clear.

[Click: revoke the exposed credential; watch access stop.]

Revoke and replace it. For private reports, virtualenv targets acknowledgment within five working days and assessment within fifteen.



[25:12–25:23]

The respond field guide: where the inventory sits, how components are named, one per artifact, validation, what a rebuild needs, the environment command, and what to do when malware ran. One line: keep the inventory with the digest and the deployment record. Today's SBOM describes today.

Your next release

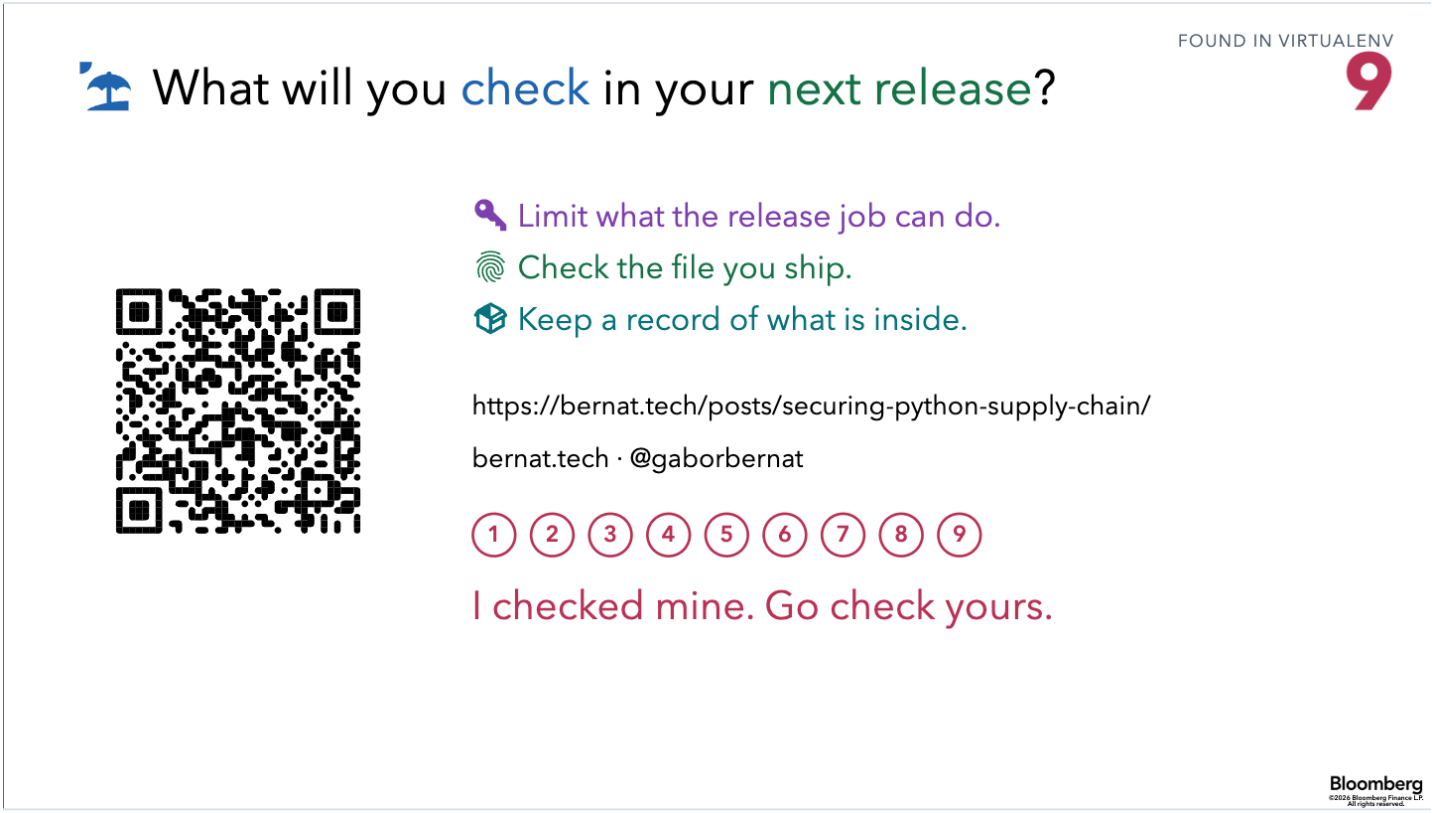
When	Consumer	Publisher
First changes	Keep `uv.lock` / `pylock.toml`; scan known vulnerabilities	`ruff check --select S` `zizmor .github/workflows`
Next release	Review updates and source selection	Restrict release authority; use Trusted Publishing
Then	Retain deployed inventories; rehearse response	Publish inventories and attestations; verify releases

Bloomberg
©2024 Bloomberg Finance L.P.
All rights reserved.

[25:23–25:36]

[Click twice to reveal the table. It is the handout, not the script.]

Tonight, open your release workflow. Find the job that runs your scanner. Read its permissions block. If there isn't one, you have my answer, and you have your first finding.



[25:36–25:50]

Nine, in a project with a threat model, CodeQL, a fuzzer, and a maintainer who gives this talk. Your number isn't zero either. I checked mine. Go check yours. Thank you.

[Leave this conclusion and QR code visible. Open the sources slide only if needed.]

Sources and examples

Python supply chain

- Trivy → LiteLLM / Telnyx, 2026
- Elementary workflow compromise, 2026
- Ultralytics incident analysis
- pip secure installs · pip lock
- LiteLLM maintainer account
- virtualenv activation CVE
- virtualenv 2026 activation advisories
- Downloaded seed wheels: CVE-2026-102930
- uv resolution and cooldowns
- pip-audit security model
- Trusted Publishing
- PyPI attestations
- PEP 770

Maintainer work

- GitHub Actions secure use
- zizmor audits
- virtualenv threat model
- virtualenv release verification
- virtualenv release artifacts
- OWASP prompt injection guidance
- The longer guide

Bloomberg
©2026 Bloomberg Finance L.P.
All rights reserved.

[Reference slide.]

The slide citations link to the sources for each claim. The terminal recordings use virtualenv 21.14.6, and repository examples refer to the inspected bf09c7eb snapshot. The recordings and commands live with the deck so they can be refreshed before the talk.